



智能计算系统学习日志

从神经网络到全栈系统

作者: ZHY

时间: July 16, 2026

版本: 0.1



不要以为抹消过去，重新来过，即可发生什么改变。——比企谷八幡

目录

第 1 章 智能计算系统认知地图	1
1.1 人工智能及其计算系统发展脉络	1
1.2 智能计算系统框架	1
第 2 章 神经网络基础	2
2.1 机器学习经典框架	2
2.2 线性回归模型	3
2.2.1 Representation	3
2.2.2 Evaluation	3
2.2.3 Optimization	3
2.3 神经网络	4
2.3.1 Representation	4
2.3.2 Evaluation	5
2.3.3 交叉验证	5
2.3.4 Optimization	5
第 3 章 深度学习	7
3.1 计算机视觉发展长廊	7
3.1.1 卷积神经网络的结构	7
3.1.1.1 卷积层	7
3.1.1.2 池化层	9
3.1.1.3 全连接层与 Softmax 输出层	9
3.1.2 卷积神经网络的优化	10
3.1.2.1 CNN 网络的前向传播	11
3.1.2.2 CNN 网络的反向传播	11
3.1.2.3 优化技巧	13
3.1.3 应用于图像分类的卷积神经网络	16
3.1.3.1 AlexNet ——深度学习的破冰者	16
3.1.3.2 VGG ——深度与简洁的极致统一	17
3.1.3.3 Inception-v1 (GoogLeNet) ——多尺度卷积的并行探索	17
3.1.3.4 BN-Inception ——批量归一化带来的训练革命	18
3.1.3.5 Inception-v3	19
3.1.3.6 ResNet	20
3.1.4 应用于图像目标检测的卷积神经网络	21
3.1.4.1 背景知识：评测指标	21
3.1.4.2 R-CNN	22
3.1.4.3 Fast R-CNN	23
3.1.4.4 Faster R-CNN	26
3.1.4.5 YOLO	29
3.1.4.6 SSD	30
3.1.5 应用于图像生成的神经网络	30
3.1.5.1 生成式对抗网络 GAN	30

3.1.5.2 扩散模型 (Diffusion Models)	35
3.2 自然语言处理发展长廊	36
3.2.1 循环神经网络 RNN	36
3.2.1.1 网络结构	36
3.2.1.2 正向传播与反向传播	39
3.2.1.3 梯度消失与梯度爆炸	40
3.2.2 长短期记忆网络 LSTM	41
3.2.2.1 LSTM 的核心创新	41
3.2.2.2 LSTM 的数学形式	41
3.2.2.3 LSTM 如何解决梯度消失?	42
3.2.2.4 GRU: LSTM 的轻量级变体	45
3.2.2.5 LSTM 的工程实践建议	45
3.3 从自然语言处理到大语言模型	46
3.3.1 普通的 Seq2Seq 模型	46
3.3.1.1 架构总览: 编码器-解码器 (Encoder-Decoder)	46
3.3.1.2 信息瓶颈问题 (Information Bottleneck)	46
3.3.2 注意力机制 (Attention Mechanism)	46
3.3.2.1 注意力机制的核心公式	46
3.3.2.2 注意力机制的三层递进理解	48
3.3.2.3 自注意力 (Self-Attention) 的引入	48
3.3.3 Transformer	49
3.3.3.1 总体架构	49
3.3.3.2 多头注意力 (Multi-Head Attention)	50
3.3.3.3 位置编码 (Positional Encoding)	50
3.3.3.4 逐位置前馈网络 (Position-wise FFN)	50
3.3.3.5 掩蔽机制 (Masking)	50
3.3.3.6 Transformer 的优势与局限	50
3.3.4 基于 Transformer 的大模型架构	51
3.3.4.1 BERT: 双向编码器表示的里程碑	51
3.3.4.2 GPT 系列: 自回归生成的技术演进	53
3.3.4.3 Gemini: 原生多模态的统一架构	54
3.3.4.4 Claude 系列: 聚焦对齐与安全	54
3.3.5 Transformer 在图像处理中的应用	55
3.3.5.1 ViT: 将图像视为词序列	55
3.3.5.2 DETR: 目标检测的端到端革命	56
3.3.5.3 Swin Transformer: 层次化视觉 Transformer	57
3.3.5.4 DiT: 扩散模型中的 Transformer	58
3.4 多模态大模型	58
3.4.0.1 CLIP: 连接文本与图像的桥梁	59
3.4.0.2 BLIP: 统一理解与生成的视觉-语言模型	59
3.4.0.3 BLIP-2: 高效的多模态预训练范式	60
3.4.1 基于大模型的智能体系统	61
3.4.1.1 智能体的定义与核心架构	61
3.4.1.2 智能体的核心推理范式	61
3.4.1.3 代表性智能体系统	63

3.4.1.4 智能体系统的关键能力维度	64
3.4.1.5 智能体系统的安全与对齐挑战	64
3.4.1.6 智能体系统的未来趋势	64
3.5 神经网络量化	65
3.5.1 量化的动机与基本原理	65
3.5.1.1 为什么需要量化?	65
3.5.1.2 量化的数学本质	65
3.5.2 量化方案的分类体系	66
3.5.2.1 均匀量化 vs. 非均匀量化	66
3.5.2.2 对称量化 vs. 非对称量化	66
3.5.2.3 逐层量化 vs. 逐通道量化	66
3.5.3 量化方法体系: PTQ 与 QAT	66
3.5.3.1 训练后量化 (PTQ)	66
3.5.3.2 量化感知训练 (QAT)	67
3.5.4 极端量化: 二值化神经网络 (BNN)	67
3.5.5 工业级量化最佳实践	67
第 4 章 编程框架使用	69
4.1 PyTorch 具体用法	69
4.1.1 导入与设备检查	69
4.1.2 张量创建与基本操作	69
4.1.3 梯度计算用法	70
4.1.4 模型定义	70
4.1.5 损失函数与优化器	71
4.1.6 数据集与 DataLoader	72
4.1.7 训练、验证与测试模板	72
4.1.8 保存与加载	73
4.1.9 混合精度训练	73
4.2 TensorFlow/Keras 具体用法	74
4.2.1 导入与设备检查	74
4.2.2 张量与变量	74
4.2.3 Keras Sequential 模型	74
4.2.4 Keras 函数式模型	75
4.2.5 子类化模型	75
4.2.6 compile、fit、evaluate、predict	76
4.2.7 损失函数、优化器与指标	76
4.2.8 tf.data 数据流水线	76
4.2.9 GradientTape 自定义训练	77
4.2.10 保存与加载	77
4.2.11 回调函数	78
4.2.12 混合精度训练	78
4.3 常见任务代码模板	79
4.3.1 PyTorch 图像分类模板	79
4.3.2 TensorFlow 图像分类模板	79
4.4 PyTorch 与 TensorFlow API 对照	79

4.5 实践检查清单	79
------------------	----

第 1 章 智能计算系统认知地图

内容提要

智能计算系统发展史

智能计算系统全流程视图

- 本课程本质上探讨一个问题：如何从无到有实现人工智能。
- 该问题源于一个关键预设：人工智能的实现是一个系统工程，今天大家却大部分仅关注算法问题。
- 新的时代对人工智能的需求勃发，新的系统被称为智能计算系统。
 - 智能的核心是算法，算法运行在具体的机器上，机器的运行代价不容忽视。
 - 想要在现实世界中设计优化并运用人工智能算法，必须了解整个智能计算系统。

1.1 人工智能及其计算系统发展脉络

纵观计算机发展百年史，每一代人工智能浪潮，总是经历着先是核心认知思路的改变，再到具体算法不断演进，配套的计算系统开始设计重组，最终人们的生活形成大的变化。

总结如下。

表 1.1: 三代智能计算系统对比

特征维度	第一代 (1980s)	第二代 (2010s)	第三代
核心认知	符号主义	连接主义	二者融合
方法论	形式逻辑、专家系统	深度学习、反向传播	探索中
计算载体	Prolog 机、LISP 机等专用计算机	GPU、AI 专用芯片 (TPU/NPU)	探索中
核心瓶颈	逻辑规则难以穷尽	算力与能耗瓶颈	探索中

1.2 智能计算系统框架

从系统层面观察一个大语言模型，其简要架构如下图所示。本课程核心目标就是打通智能系统全流程。

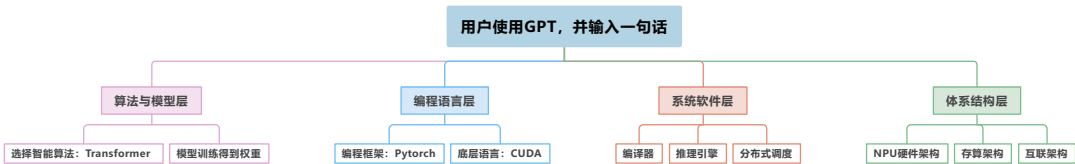


图 1.1: NPU 推理全栈流程图

第2章 神经网络基础

内容提要

- ❑ 线性回归模型
- ❑ 正则化思路

- ❑ 交叉验证方法
- ❑ 神经网络基础

本章内容看上去杂多，实则只是讨论了两个子问题。

- 以线性回归为例，讨论了机器学习的基本流程。
- 以神经网络为例，讨论了深度学习的基本流程。

二者同属一派，关系如下图所示。

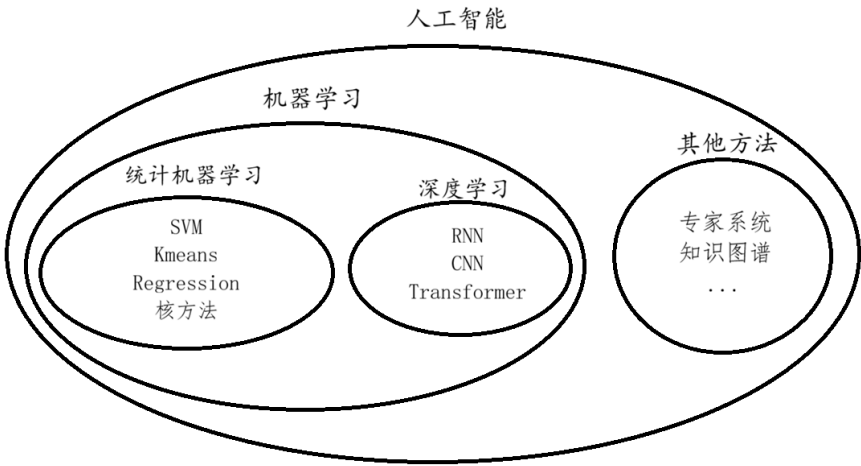


图 2.1: 机器学习、深度学习与人工智能的关系

2.1 机器学习经典框架

所有的机器学习算法本质上都是在某一输入空间和输出空间中，寻找一个函数模型，其可以最好地描述输入数据和输出数据的关系。因此所有的机器学习算法必然由三部分组成。

- **Representation:** 选择一个函数模型，作为输入和输出的映射关系。
- **Evaluation:** 选择一个评价函数，作为衡量模型好坏的标准。
- **Optimization:** 选择一个优化算法，作为寻找最优模型的手段。

上述分析思路我们称之为 **R-E-O** 框架。接下来，我们将以此为线索讨论两个经典模型。

2.2 线性回归模型

2.2.1 Representation

命题 2.1 (线性回归模型的输入输出空间)

线性回归模型在下述空间中被定义。

输入: $\mathbf{x} = [1, x_1, \dots, x_m]^T \in \mathbb{R}^{m+1}$

输出: $y \in \mathbb{R}$

数据集: $\mathcal{S} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$

核心假设:

- 输入 $\mathbf{x}$ 是随机变量 $\mathcal{X}$ 的采样结果
- 输出 y 是随机变量 $\mathcal{Y}$ 的采样结果
- $(\mathbf{x}, y)$ 是联合分布 $\mathcal{D}$ 的采样结果
- 数据集 $\mathcal{S}$ 是联合分布 $\mathcal{D}^n$ 的独立同分布采样结果

定义 2.1 (线性回归模型)

线性回归模型在函数空间 $\mathcal{F}$ 中试图寻找一个线性映射 f_w

$$\mathcal{F} = \{f_{\mathbf{w}}(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} \mid \mathbf{w} = [w_0, w_1, \dots, w_m]^T \in \mathbb{R}^{m+1}\} \quad (2.1)$$

对于给定的 $\mathbf{w}$, 有 $f_{\mathbf{w}}(\mathbf{x}) = w_0 + w_1 x_1 + \dots + w_m x_m$ 。

注 上述论述是线性回归模型的形式化定义。一言以蔽之, 线性回归模型假设 $\mathbf{x}$ 和 y 之间的关系是线性的, 并且通过参数 $\mathbf{w}$ 来描述这种线性关系。

2.2.2 Evaluation

在线性回归中, 评估函数衡量模型预测值 $f_{\mathbf{w}}(\mathbf{x})$ 与真实值 y 之间的差距。最常用的选择是均方误差 (Mean Squared Error, MSE)。

定义 2.2 (线性回归的损失函数)

给定数据集 $\mathcal{S} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$, 线性回归模型的损失函数定义为:

$$\mathcal{L}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n \left(f_{\mathbf{w}}(\mathbf{x}^{(i)}) - y^{(i)} \right)^2 \quad (2.2)$$

等价地, 可以写成矩阵形式:

$$\mathcal{L}(\mathbf{w}) = \frac{1}{n} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 \quad (2.3)$$

其中 $\mathbf{X} \in \mathbb{R}^{n \times (m+1)}$ 为设计矩阵, $\mathbf{y} = [y^{(1)}, \dots, y^{(n)}]^T$ 。

注 MSE 作为评估函数的合理性源于其数学性质: 在误差服从均值为零的高斯分布假设下, 最小化 MSE 等价于极大似然估计。这一性质为线性回归提供了概率论层面的解释。

2.2.3 Optimization

线性回归的优化目标是 minimize 损失函数 $\mathcal{L}(\mathbf{w})$, 即求解:

$$\mathbf{w}^* = \arg \min_{\mathbf{w}} \mathcal{L}(\mathbf{w}) \quad (2.4)$$

由于 $\mathcal{L}(\mathbf{w})$ 是凸函数, 存在闭式解 (解析解) 和数值解两种求解路径。

命题 2.2 (线性回归的闭式解)

令 $\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}) = 0$ ，可得：

$$\mathbf{w}^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} \quad (2.5)$$

该解被称为普通最小二乘（**Ordinary Least Squares, OLS**）估计量。



注 闭式解的求解需要 $\mathbf{X}^T \mathbf{X}$ 可逆。当特征数 m 很大 ($m > n$) 或存在多重共线性时，矩阵不可逆，此时必须引入正则化项。

在实际应用中，由于矩阵求逆运算复杂度很高，通常采用**梯度下降（Gradient Descent）**进行迭代求解。

命题 2.3 (线性回归: 梯度下降法)

从几何直观上理解，梯度是函数上升最快的方向，因此沿着梯度的反方向更新参数 $\mathbf{w}$ ，可以逐步逼近最优解：

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}) \quad (2.6)$$

其中 η 为学习率，梯度计算公式为：

$$\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}) = \frac{2}{n} \mathbf{X}^T (\mathbf{X} \mathbf{w} - \mathbf{y}) \quad (2.7)$$



注 从 R-E-O 框架审视线性回归：线性函数族完成了表示，MSE 完成了评估，而解析解或梯度下降完成了优化。三者合在一起，构成了一个有完整数学闭合形式的机器学习算法。

2.3 神经网络

如果说线性回归是最简单的机器学习模型，那么神经网络则代表了另一条截然不同的路线：通过组合基本非线性单元来逼近任意复杂函数。

2.3.1 Representation

定义 2.3 (人工神经元)

一个神经元是一个计算单元，接收输入向量 $\mathbf{x} \in \mathbb{R}^d$ ，输出标量值：

$$a = \sigma(\mathbf{w}^T \mathbf{x} + b) \quad (2.8)$$

其中 $\mathbf{w} \in \mathbb{R}^d$ 为权重向量， $b \in \mathbb{R}$ 为偏置， $\sigma(\cdot)$ 为激活函数（如 ReLU、Sigmoid、Tanh）。

**定义 2.4 (神经网络的前向传播)**

一个 L 层前馈神经网络通过逐层复合的方式定义函数：

$$f(\mathbf{x}) = f_L \circ f_{L-1} \circ \cdots \circ f_1(\mathbf{x}) \quad (2.9)$$

其中每一层 f_ℓ 是一个仿射变换 ($\mathbf{z}_\ell = \mathbf{W}_\ell \mathbf{a}_{\ell-1} + \mathbf{b}_\ell$) 后接激活函数 σ_ℓ 。

记第 ℓ 层的神经元数为 d_ℓ ，输入层 $d_0 = m$ ，输出层 $d_L = k$ 。则第 ℓ 层的参数矩阵为：

$$\mathbf{W}_\ell \in \mathbb{R}^{d_\ell \times d_{\ell-1}}, \quad \mathbf{b}_\ell \in \mathbb{R}^{d_\ell} \quad (2.10)$$

整个网络的参数集合为 $\theta = \{\mathbf{W}_\ell, \mathbf{b}_\ell\}_{\ell=1}^L$ 。



定理 2.1 (通用近似定理, Universal Approximation Theorem)

设 $\sigma(\cdot)$ 为一非常数、有界、单调递增的连续激活函数（如 Sigmoid 或 Tanh）。令 I_m 表示 m 维单位超立方体 $[0, 1]^m$, $C(I_m)$ 表示定义在 I_m 上的所有连续函数构成的空间。则对于任意目标函数 $f \in C(I_m)$ 和任意精度 $\varepsilon > 0$, 存在整数 N 、实数常数 $v_i, b_i \in \mathbb{R}$ 以及实向量 $\mathbf{w}_i \in \mathbb{R}^m$ ($i = 1, \dots, N$), 使得如下形式的单隐层前馈神经网络:

$$F(\mathbf{x}) = \sum_{i=1}^N v_i \sigma(\mathbf{w}_i^T \mathbf{x} + b_i) \quad (2.11)$$

可以作为 f 的近似函数, 满足:

$$|F(\mathbf{x}) - f(\mathbf{x})| < \varepsilon, \quad \forall \mathbf{x} \in I_m \quad (2.12)$$



注 该定理表明, **宽度** (即单层神经元数) 足够大的前馈神经网络具有万能逼近能力。它说明神经网络的表示能力在理论上是完备的——任何连续函数都可以用单隐层网络以任意精度逼近。但该定理具有以下重要局限:

1. **未保证学习效率**: 未说明达到给定精度所需的最小神经元数 N 。
2. **未保证可学习性**: 不保证梯度下降等优化算法能找到它。
3. **未保证泛化性**: 需要额外的正则化与验证。

表示、评估和优化三个环节各自的理论边界, 决定了实际系统的性能上限。即使表示能力极强也无济于事。

2.3.2 Evaluation

神经网络的评估函数取决于任务类型, 通常采用极大似然估计框架进行统一理解。

定义 2.5 (神经网络中的损失函数)

根据任务类型, 常见的损失函数有:

- 回归任务: 均方误差 (MSE), $\mathcal{L} = \frac{1}{n} \sum_{i=1}^n \|f(\mathbf{x}^{(i)}) - \mathbf{y}^{(i)}\|_2^2$
- 二分类任务: 二元交叉熵 (Binary Cross-Entropy), $\mathcal{L} = -\frac{1}{n} \sum_{i=1}^n [y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]$
- 多分类任务: 交叉熵 (Cross-Entropy), $\mathcal{L} = -\frac{1}{n} \sum_{i=1}^n \sum_{j=1}^k y_j^{(i)} \log \hat{y}_j^{(i)}$



2.3.3 交叉验证

2.3.4 Optimization

不同于线性回归的凸优化问题, 神经网络的损失函数是高度非凸的, 不存在闭式解。优化依赖于**反向传播 (Backpropagation)**和梯度下降及其变体。

定义 2.6 (反向传播算法)

对于第 ℓ 层的参数, 梯度通过链式法则从输出层向输入层逐层传递:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_\ell} = \delta_\ell \cdot \mathbf{a}_{\ell-1}^T \quad (2.13)$$

其中 $\delta_\ell = \frac{\partial \mathcal{L}}{\partial \mathbf{z}_\ell}$ 为第 ℓ 层的误差项, $\mathbf{a}_{\ell-1}$ 为前一层激活值。

**定义 2.7 (神经网络的参数更新)**

采用随机梯度下降 (SGD) 或其自适应变体进行参数更新:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \cdot \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla_{\boldsymbol{\theta}} \mathcal{L}^{(i)}(\boldsymbol{\theta}) \quad (2.14)$$

其中 $\mathcal{B}$ 为 mini-batch, $B = |\mathcal{B}|$ 为批大小。

实际训练中常用更先进的优化器 (如 Adam、AdamW), 其核心是自适应调整每个参数的学习率。



命题 2.4 (神经网络优化的核心挑战)

神经网络的优化面临以下关键挑战：

- 非凸性：损失函数存在大量局部极小值和鞍点，梯度下降无法保证收敛到全局最优。
- 梯度消失/爆炸：在深层网络中，梯度逐层相乘可能导致数值不稳定，梯度消失使浅层无法更新，梯度爆炸导致训练发散。
- 过拟合：由于参数量巨大（大模型参数量可达千亿级别），神经网络极易过拟合，必须依赖正则化技术：

$$\mathcal{L}_{\text{reg}}(\theta) = \mathcal{L}(\theta) + \lambda \Omega(\theta) \quad (2.15)$$

其中 $\Omega(\theta)$ 为参数范数惩罚（L1/L2），Dropout、早停等同样为有效手段。



注 从 R-E-O 框架审视神经网络：网络的深度和宽度决定了 Representation，损失函数与评价指标共同构成 Evaluation，而反向传播 + 梯度下降则构成了 Optimization。

命题 2.5 (R-E-O 框架的统一视角)

线性回归和神经网络虽然是复杂度迥异的两个模型，但在 R-E-O 框架下呈现出统一的结构：

	Representation	Evaluation	Optimization
线性回归	线性函数 $\mathcal{F} = \{\mathbf{w}^T \mathbf{x}\}$	MSE	解析解 / 梯度下降
神经网络	非线性复合函数	交叉熵 / MSE	反向传播 + SGD/Adam

机器学习算法的演变本质上是这三部分各自独立演进与协同创新的历史：表示的复杂化（从线性到非线性）、评估任务的多元化（从回归到生成）、优化技术的精进。理解了三者之间的相互作用，就理解了机器学习的核心方法论——而这也正是《智能计算系统》课程的理论起点。



第3章 深度学习

内容提要

□ 用于图像处理的深度学习网络

□ 用于文本和语音处理的深度学习网络

课程的这一章节，实际上进入了漫长的深度学习发展史，从计算机视觉，自然语言处理（大语言模型）两条线索出发，梳理了一个学科发展长廊中的璀璨明珠。

3.1 计算机视觉发展长廊

计算机视觉的发展必须要从 CNN 卷积神经网络说起。

3.1.1 卷积神经网络的结构

卷积神经网络（Convolutional Neural Network, CNN）是计算机视觉任务的基石。其核心设计思想源于生物视觉系统的两个重要特性：**局部感受野**（Local Receptive Field）和**权重共享**（Weight Sharing）。与全连接网络（Fully Connected Network, FCN）相比，CNN 通过显式地编码图像的空间结构先验，大幅减少了参数数量，从而使得深层网络的训练成为可能。

3.1.1.1 卷积层

定义 3.1 (卷积神经网络的基本组成)

一个典型的卷积神经网络由若干基本功能层堆叠而成，按信息流动顺序可划分为以下四类核心组件：

- **卷积层（Convolutional Layer）**：通过滑动卷积核提取输入数据的局部空间特征，是 CNN 中参数共享与稀疏连接的实现载体。
- **池化层（Pooling Layer）**：对特征图进行下采样，起到降维、增大感受野和引入平移不变性的作用。
- **全连接层（Fully Connected Layer）**：将提取到的高层语义特征展平并进行非线性组合，完成最终的决策或回归任务。
- **Softmax 层**：将全连接层的输出分数映射为概率分布，常作为分类任务的输出端。

命题 3.1 (卷积层的数学形式)

设输入特征图为 $\mathbf{X} \in \mathbb{R}^{H \times W \times C}$ ，输出特征图为 $\mathbf{Y} \in \mathbb{R}^{H' \times W' \times C'}$ ，卷积核权重为 $\mathbf{W} \in \mathbb{R}^{k \times k \times C \times C'}$ ，偏置为 $\mathbf{b} \in \mathbb{R}^{C'}$ ，则卷积操作可形式化为：

$$\mathbf{Y}_{i,j,c'} = \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} \sum_{c=1}^C \mathbf{X}_{i+u,j+v,c} \cdot \mathbf{W}_{u,v,c,c'} + \mathbf{b}_{c'} \quad (3.1)$$

其中 k 为卷积核尺寸， C 与 C' 分别为输入与输出通道数。

注[CNN 的设计哲学] 在未采用 CNN 之前，人们设想将输出的每一个信息与输入的每一个信息通过全连接层相连。但是参数爆炸，无法训练。而 CNN 的诞生则源于一下的简单直觉。

- 图像中表示同一特征的像素往往相互邻接。
- 图像中出现过的特征往往会在其他区域以相似的统计特性出现。

而卷积核的设计正是基于上述直觉：通过局部感受野捕捉邻接像素的相关性，通过参数共享捕捉全局特征的重复性。

命题 3.2 (卷积层的设计技巧)

卷积层的实际应用效果不仅取决于其数学定义，还高度依赖于三个核心超参数：边界扩充（Padding）、卷积步长（Stride）和空洞率（Dilation）。三者分别从“边缘处理”、“采样密度”和“视野跨度”三个维度调控特征图的输出特性。

名称	作用	具体做法
Padding	防止边缘信息丢失，控制输出尺寸	在输入特征图外侧填充零值或其他常数，再进行卷积
Stride	控制卷积核滑动的步长，实现下采样	每移动 s 个像素进行一次卷积运算
Dilation	在不增加参数量的前提下扩大感受野	在卷积核各元素间插入 0（空洞）

引入上述超参数后，卷积层输出尺寸的通用计算公式为：

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2p - [k + (k - 1)(d - 1)]}{\text{stride}} \right\rfloor + 1 \tag{3.2}$$

其中 k 为原始卷积核尺寸， d 为空洞率（ $d = 1$ 时退化为标准卷积，括号项还原为 k ）。

注[卷积神经网络的训练参数] **卷积层的可训练参数总结：** 对于一个卷积层，其可训练参数包括卷积核权重和偏置项，总量为：

$$\text{参数量} = (k^2 \times C_{\text{in}} + 1) \times C_{\text{out}} \tag{3.3}$$

其中 C_{in} 和 C_{out} 分别为输入和输出通道数。值得注意的是，该参数量与输入特征图的空间尺寸 $H_{\text{in}} \times W_{\text{in}}$ 完全无关——这正是参数共享机制带来的核心优势。由于偏置项 +1 相对于权重项 $k^2 \times C_{\text{in}}$ 通常可忽略，故常简化记为 $O(k^2 \cdot C_{\text{in}} \cdot C_{\text{out}})$ 。

命题 3.3 (深度可分离卷积)

深度可分离卷积（Depthwise Separable Convolution, DSC）将标准卷积分解为两个连续步骤，在保持相近表征能力的前提下显著降低参数量与计算量，是 MobileNet、Xception、EfficientNet 等轻量化网络的核心构建模块。

- **深度卷积（Depthwise Convolution）**——逐通道空间滤波。为每个输入通道独立分配一个 $k \times k$ 卷积核，通道间不发生混合，输出为 $\mathbf{Y}_{\text{dw}} \in \mathbb{R}^{H' \times W' \times C}$ 。参数量与计算量均为 $k^2 \times C \times H' \times W'$ 。
- **逐点卷积（Pointwise Convolution）**——跨通道特征组合。采用 1×1 卷积对深度卷积的输出进行通道维度的线性组合，生成 C_{out} 个输出通道。参数量与计算量均为 $C \times C_{\text{out}} \times H' \times W'$ 。

深度可分离卷积的总参数量与标准卷积之比为：

$$\frac{\text{Param}_{\text{DSC}}}{\text{Param}_{\text{std}}} = \frac{k^2 C + C C_{\text{out}}}{k^2 C C_{\text{out}}} = \frac{1}{C_{\text{out}}} + \frac{1}{k^2} \tag{3.4}$$

取 $k = 3$ 、 $C_{\text{out}} = 512$ 时，参数量约为标准卷积的 $1/512 + 1/9 \approx 11\%$ ，计算量缩减比例相同。

	标准卷积	深度可分离卷积	缩减比例
参数量	$k^2 C C_{\text{out}}$	$k^2 C + C C_{\text{out}}$	$\approx 1/k^2$ (C_{out} 较大时)
计算量	$k^2 H' W' C C_{\text{out}}$	$k^2 H' W' C + H' W' C C_{\text{out}}$	同上

注 深度可分离卷积的有效性依赖于“空间特征提取与通道特征组合可解耦”这一假设，该假设在自然图像中大致成立。当通道间存在高度复杂的非线性交互时（如注意力机制），标准卷积仍具优势。现代高效架构（如 MobileNetV2、EfficientNet）在此基础上引入倒置残差结构与线性瓶颈作为补充。

注[与 R-E-O 框架的关联] 深度可分离卷积是纯粹的表示（Representation）层面改进——改变了网络层的参数化结构，未改变评估准则或优化方法。表示的效率优化是模型走向实用化的关键驱动力。

3.1.1.2 池化层

池化层（Pooling Layer）是卷积神经网络中紧随卷积层之后的标准组件，其本质是一种固定的下采样算子——它不含可训练参数，仅通过确定的数学规则对特征图进行空间维度的压缩。池化层的设计动机源于三个层面的考量：降低计算复杂度、扩大感受野、以及引入对轻微形变的鲁棒性。

定义 3.2 (池化操作)

池化层将输入特征图 $\mathbf{X} \in \mathbb{R}^{H \times W \times C}$ 划分为若干不重叠（或可重叠）的局部窗口 $\mathcal{N}_{i,j}$ ，对每个窗口内的激活值进行聚合统计，输出压缩后的特征图 $\mathbf{Y} \in \mathbb{R}^{H' \times W' \times C}$ 。设池化窗口尺寸为 $p \times p$ ，步长为 s ，则输出尺寸满足：

$$H' = \left\lfloor \frac{H - p}{s} \right\rfloor + 1, \quad W' = \left\lfloor \frac{W - p}{s} \right\rfloor + 1 \tag{3.5}$$

通道数 C 在池化过程中保持不变——池化独立作用于每个通道。



命题 3.4 (池化的主要类型)

按聚合函数的不同，池化层可分为两类经典变体：

名称	数学定义	特性	典型用途
最大池化	取局部窗口内的最大值作为输出	对显著特征敏感	主流 CNN 的默认选择
平均池化	取局部窗口内所有值的均值作为输出	保留背景信息	全局池化的基础

- 最大池化设计哲学：在一个局部区域内，最强烈的激活信号最能代表该区域的特征。可以抑制噪声。



注[池化层的核心作用] 池化层的贡献可归纳为以下三个层面：

1. 降维与计算量压缩：池化操作使特征图的空间尺寸减半（典型配置 $p = 2, s = 2$ ）
2. 增大感受野：后续卷积层的一个卷积核能覆盖输入图像上更大的空间范围，有利于高层语义特征的提取。
3. 平移不变性：最大池化对局部区域内的微小位移具有不敏感性。若一个特征在窗口内任意位置出现，池化后都会被“浓缩”到同一个输出位置。

注[池化的“等价性”辨析] 需注意区分两个概念：

- 平移等变性（Equivariance）：输入平移 $\rightarrow$ 输出平移（卷积层具备）。特征的位置信息被保留，只是发生了相应的位移。
- 平移不变性（Invariance）：输入平移 $\rightarrow$ 输出不变（池化层具备）。特征的具体位置信息被丢弃，只保留“该特征是否出现”。

池化层的平移不变性，靠的不是单层窗口的大范围容忍，而是通过层层累积的“粗粒化”，将底层几像素的位移在顶层稀释为可忽略的误差。卷积提供的是“位置感知”，池化提供的是“位置鲁棒性”。两者互补，共同构成了 CNN 对空间结构的建模能力。

3.1.1.3 全连接层与 Softmax 输出层

在卷积层与池化层完成对输入图像的特征提取与空间压缩之后，网络需要将这些分布式的高层语义特征汇聚为最终的决策输出。这一职责由全连接层与 Softmax 输出层共同承担——前者负责特征的全局组合与高阶抽象，后者负责将组合结果翻译为直观的概率分布。

定义 3.3 (全连接层的数学形式)

全连接层（Fully Connected Layer, FC）将前层输出的多维特征图展平（Flatten）为一维向量 $\mathbf{a} \in \mathbb{R}^D$ ，并通过仿射变换实现特征的非线性组合：

$$\mathbf{z} = \mathbf{W}\mathbf{a} + \mathbf{b}, \quad \mathbf{W} \in \mathbb{R}^{D' \times D}, \quad \mathbf{b} \in \mathbb{R}^{D'} \tag{3.6}$$

其中 D 为输入特征维度， D' 为输出特征维度。全连接层的核心特性在于：输出层的每一个神经元都与输入层的所有神经元相连接，即“全连接”之名由此而来。

全连接层在 CNN 中扮演着“特征决策者”的角色——它将卷积层提取的局部特征（边缘、纹理、部件）进行全局性的加权组合，形成完整的语义判断。但是却由于参数量巨大，伴随着过拟合，参数爆炸等风险。因此，计算机视觉中另一种可能的处理方法是全局平均池化。

全局平均池化 = 对每个特征图取全局均值，把“空间位置”全部抹平，每个通道只留下一个数，以此替代全连接层的海量参数。

定义 3.4 (全局平均池化)

全局平均池化 (Global Average Pooling, GAP) 对特征图 $\mathbf{X} \in \mathbb{R}^{H \times W \times C}$ 的每个通道进行全局平均：

$$\mathbf{Y}_c = \frac{1}{H \times W} \sum_{i=1}^H \sum_{j=1}^W \mathbf{X}_{i,j,c}, \quad c = 1, \dots, C \tag{3.7}$$

输出为 $\mathbf{Y} \in \mathbb{R}^C$ 。GAP 将每个通道的空间信息压缩为单一标量，使得网络可接受任意尺寸的输入，且参数量降为零——其本质是“强制性的特征选择”：每个通道对应一种语义模式，其全局平均响应强度直接反映该模式在整张图像中的显著性。

注[全连接层 vs 全局平均池化]

对比维度	全连接层	全局平均池化
参数量	$D \times D'$ (数百万至数千万)	0
可解释性	权重难以可视化	每个通道对应一类语义响应
典型应用	AlexNet、VGG	ResNet、DenseNet、MobileNet

定义 3.5 (Softmax 层)

在分类任务中，全连接层（或 GAP）输出的得分向量 $\mathbf{z} \in \mathbb{R}^K$ （其中 K 为类别数）被称为 **Logits**，其取值可为任意实数，缺乏直观的概率解释。Softmax 层将 Logits 映射为概率单纯形上的分布：

$$\hat{y}_i = \text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}, \quad i = 1, \dots, K \tag{3.8}$$

该映射具有三条优良性质：

1. 非负性： $\hat{y}_i > 0$ ，符合概率的公理化定义。
2. 归一性： $\sum_{i=1}^K \hat{y}_i = 1$ ，保证输出为一个合法的概率分布。
3. 保序性：Softmax 是单调递增函数，得分越高者概率越大，分类决策不受影响。

命题 3.5 (交叉熵损失)

Softmax 层通常与交叉熵损失 (Cross-Entropy Loss) 配合使用。设真实标签的独热编码为 $\mathbf{y} \in \{0, 1\}^K$ （其中仅正确类别处为 1，其余为 0），预测概率分布为 $\hat{\mathbf{y}}$ ，则交叉熵损失定义为：

$$\mathcal{L}_{\text{CE}}(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_{i=1}^K y_i \log \hat{y}_i = - \log \hat{y}_{\text{true}} \tag{3.9}$$

直观上：当模型对正确类别给出的概率越接近 1 时，损失趋近于 0；当概率趋近于 0 时，损失趋近于正无穷。交叉熵直接度量了两个概率分布之间的“距离”。

3.1.2 卷积神经网络的优化

神经网络的优化与线性回归等凸优化问题有着本质区别。深度网络的损失函数是高度非凸的，不存在闭式解析解，核心依赖于反向传播 (Backpropagation) 计算梯度，并结合梯度下降及其变体进行参数迭代更新。

命题 3.6 (优化的核心挑战)

深度神经网络的优化面临以下三重困境：

- 非凸性：损失函数存在大量局部极小值和鞍点。鞍点（Saddle Point）尤为棘手——其梯度为零，但既非极大也非极小，SGD 在此处会陷入停滞。高维空间中鞍点的数量远多于局部极小值，是深度网络优化的主要障碍。
- 梯度不稳定：在深层网络中，梯度逐层相乘（链式法则）可能导致数值不稳定。梯度消失（Gradient Vanishing）使浅层参数几乎不更新；梯度爆炸（Gradient Explosion）则导致参数更新发散。
- 过拟合风险：参数量远超样本量，网络极易记忆训练集噪声而非学习泛化规律。

3.1.2.1 CNN 网络的前向传播

前向传播是信息在 CNN 中从输入到输出的逐层流动过程。图像数据依次经过特征提取、特征整合与分类决策三个阶段，最终输出类别概率分布。

命题 3.7 (CNN 分类器的完整前向传播)

一个用于图像分类的 CNN，其前向传播路径可分解为三个阶段：

1. 特征提取阶段（卷积与池化）：输入图像 $\mathbf{X} \in \mathbb{R}^{H \times W \times 3}$ 依次经过 n 组“卷积 + 池化”模块，逐层降低空间分辨率、增加通道数，最终输出高层语义特征图：

$$\mathbf{X} \xrightarrow{\text{Conv}_1 \rightarrow \text{Pool}_1 \rightarrow \dots \rightarrow \text{Conv}_n \rightarrow \text{Pool}_n} \mathbf{A} \in \mathbb{R}^{h \times w \times C} \quad (3.10)$$

其中单个卷积层的前向计算为（第 ℓ 层输入为 $\mathbf{A}_{\ell-1}$ ）：

$$\mathbf{Z}_\ell = \mathbf{W}_\ell * \mathbf{A}_{\ell-1} + \mathbf{b}_\ell, \quad \mathbf{A}_\ell = f_\ell(\mathbf{Z}_\ell) \quad (3.11)$$

$*$ 表示卷积运算， f_ℓ 为激活函数（通常为 ReLU）。

2. 特征整合阶段（全连接层）：将特征图 $\mathbf{A}$ 展平或全局平均池化为一维向量 $\mathbf{a} \in \mathbb{R}^D$ ，经全连接层映射为类别得分：

$$\mathbf{A} \xrightarrow{\text{Flatten / GAP}} \mathbf{a} \in \mathbb{R}^D \xrightarrow{\mathbf{W}_{fc}\mathbf{a} + \mathbf{b}_{fc}} \mathbf{z} \in \mathbb{R}^K \quad (3.12)$$

其中 K 为类别总数， $\mathbf{z}$ 为各类别的未归一化得分（Logits）。

3. 概率输出阶段（Softmax）：将得分向量 $\mathbf{z}$ 映射为概率单纯形上的分布：

$$\mathbf{z} \xrightarrow{\text{Softmax}} \hat{\mathbf{y}} \in \Delta^{K-1}, \quad \hat{y}_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \quad (3.13)$$

完整路径可概括为：

$$\mathbf{X} \xrightarrow{\text{卷积 + 池化} \times n} \mathbf{A} \xrightarrow{\text{展平/全局池化}} \mathbf{a} \xrightarrow{\text{全连接}} \mathbf{z} \xrightarrow{\text{Softmax}} \hat{\mathbf{y}} \quad (3.14)$$

训练阶段， $\hat{\mathbf{y}}$ 与真实标签 $\mathbf{y}$ （one-hot 编码）计算交叉熵损失：

$$\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = - \sum_{i=1}^K y_i \log \hat{y}_i \quad (3.15)$$

3.1.2.2 CNN 网络的反向传播

反向传播是链式法则在神经网络中的具体实现。它从损失函数出发，将误差信号逐层向输入端传递，计算损失对每一层参数的梯度，为参数更新提供依据。

命题 3.8 (CNN 反向传播：核心公式与操作)

符号约定： $\mathbf{A}_{\ell-1}$ ：输入 ($H_{\ell-1} \times W_{\ell-1} \times C_{\ell-1}$)， $\mathbf{W}_\ell$ ：卷积核 ($k \times k \times C_{\ell-1} \times C_\ell$)， $\mathbf{b}_\ell$ ：偏置 (C_ℓ)， $\mathbf{Z}_\ell$ ：卷积输出， $\delta_\ell = \partial \mathcal{L} / \partial \mathbf{Z}_\ell$ ：误差项。超参数：步长 s ，填充 p 。

3.1.2.2.1 输出层 (Softmax + 交叉熵)

$$\delta_{\text{out}} = \hat{\mathbf{y}} - \mathbf{y}$$

其中 $\hat{\mathbf{y}}$ 为预测概率向量, $\mathbf{y}$ 为真实标签的 one-hot 向量, δ_{out} 为输出层误差项。

3.1.2.2.2 全连接层

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{\text{fc}}} = \mathbf{a} \cdot \delta^T, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}_{\text{fc}}} = \delta, \quad \delta_{\text{prev}} = \mathbf{W}_{\text{fc}} \cdot \delta$$

其中 $\mathbf{a}$ 为全连接层输入向量, δ 为上游误差, $\mathbf{W}_{\text{fc}}$ 为权重矩阵, $\mathbf{b}_{\text{fc}}$ 为偏置。

3.1.2.2.3 池化层

- 最大池化: 误差只传回前向传播时窗口内最大值所在位置, 其余位置置零。
- 平均池化: 误差在窗口内均匀分配, 即 $\delta_{\text{prev}} = \frac{1}{p^2} \delta_{\text{pool}}$, 其中 p 为池化窗口尺寸。

3.1.2.2.4 卷积层 (核心)

3.1.2.2.4.1 偏置梯度

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}_{\ell}} = \sum_{i,j} \delta_{\ell}[i, j, :]$$

即对误差张量的空间维度 (高度 i 和宽度 j) 求和。由于偏置梯度的个数等于卷积核的个数, 每一个卷积核作用在输入特征图上得到一个输出特征图, 因此每一个卷积核与一个误差矩阵对应, 对该误差矩阵求和即得到该卷积核的偏置梯度。

3.1.2.2.4.2 权重梯度 直观理解: 权重来源于卷积核。当卷积核在输入特征图上不断游走时: 每移动一次, 每一个权重会对应到一个输入特征图中的元素, 然后卷积, 形成一个输出元素。

于是权重梯度就等于 sum (该权重作用过的每一个输入元素 * 作用时形成的输出元素的梯度)

当考虑张量 (batch 和输入频道有多个) 时, 只要一一对应即可。

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{\ell}[u, v, c, c']} = \sum_{i,j} \delta_{\ell}[i, j, c'] \cdot \mathbf{A}_{\ell-1}[is + u - p, js + v - p, c]$$

其中:

- u, v : 卷积核内的空间位置 ($0 \leq u, v < k$)
- c : 输入通道索引
- c' : 输出通道索引
- i, j : 输出特征图的空间位置
- $is + u - p, js + v - p$: 输入特征图上对应的位置

3.1.2.2.4.3 误差回传 直观理解: 一个元素通过多个卷积核进行卷积计算, 并形成多个输出特征图时: 它直接对应多个卷积核中的多个权重, 并通过计算, 得到多个输出特征图的对应元素。

一个输入元素对应多个权重, 每个权重唯一对应一个输出元素。所以将所有输出元素的梯度乘上对应权重并求和, 便得到输入元素的梯度。

$$\delta_{\ell-1}[x, y, c] = \sum_{c'} \sum_{\substack{i, j \\ (x, y) \in \text{window}(i, j)}} \delta_{\ell}[i, j, c'] \cdot \mathbf{W}_{\ell}[x - is + p, y - js + p, c, c']$$

其中：

- x, y : 输入特征图上的位置
- $\text{window}(i, j)$ 表示输出位置 (i, j) 的卷积窗口所覆盖的输入区域
- 条件 $(x, y) \in \text{window}(i, j)$ 等价于：

$$is - p \leq x < is - p + k, \quad js - p \leq y < js - p + k$$

- $x - is + p, y - js + p$: 输入位置 (x, y) 在卷积核中的相对位置

一句话：误差回传 = 把每个输出位置的误差”按权重”分配回所有看到它的输入位置。一个输入位置若被多个输出位置看到，则累加所有贡献。

3.1.2.2.5 激活函数（ReLU） 已在上一章介绍。等价地，逐元素写为：

$$\delta_{\ell}[i, j, c] = \begin{cases} \delta_{\ell}^{\text{upstream}}[i, j, c], & \mathbf{Z}_{\ell}[i, j, c] > 0 \\ 0, & \mathbf{Z}_{\ell}[i, j, c] \leq 0 \end{cases}$$

即只有前向传播时激活（正值）的位置，误差才能继续回传。

命题 3.9 (梯度消失与爆炸的根源)

由误差递推式可知，当网络加深时，梯度由一系列矩阵相乘构成：

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{\ell}} \propto \prod_{j=\ell}^{L-1} (\mathbf{W}_{j+1}^T \cdot \text{diag}(f'_j)) \quad (3.16)$$

若矩阵的谱范数 > 1 ，梯度指数级增长（爆炸）；若 < 1 ，梯度指数级衰减（消失）。

- **Sigmoid/Tanh**: 导数值域为 $(0, 1]$ ，连乘后极易趋近于 0，是梯度消失的主要诱因。
- **ReLU**: 正半轴导数为 1，负半轴为 0，在一定程度上缓解了梯度消失，但引入了“神经元死亡”问题（负半轴永不激活）。

在 CNN 中，若特征提取阶段的卷积层无法获得有效梯度，网络将退化为“仅全连接层可学、卷积层停滞”的状态——这正是 Xavier/Kaiming 初始化、Batch Normalization 等技巧试图解决的核心问题。

3.1.2.3 优化技巧

参数初始化

参数初始化是训练深度网络的第一步，也是最容易被忽视却至关重要的环节。初始化不当会导致梯度消失或爆炸，使网络从一开始就无法有效学习。

命题 3.10 (Xavier 初始化)

Xavier 初始化（Glorot 初始化）由 Glorot & Bengio (2010) 提出，其设计目标是使每一层的输入和输出方差保持一致，从而避免信号在前向传播中过度放大或衰减。其核心思想基于方差一致性原则：

- 适用激活函数：Sigmoid、Tanh 等关于原点对称、在零点附近近似线性的激活函数。
- 初始化方式：从均匀分布或正态分布中采样，满足：

$$\text{Uniform: } \mathbf{W} \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right) \quad (3.17)$$

$$\text{Normal: } \mathbf{W} \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}} + n_{\text{out}}}\right) \quad (3.18)$$

其中 n_{in} 和 n_{out} 分别为该层的输入和输出神经元数量。

Xavier 初始化的推导基于线性激活假设：若 $\text{Var}(x) = \text{Var}(y)$ ，则需要 $\text{Var}(W) = 2/(n_{\text{in}} + n_{\text{out}})$ 。这一策略使得信号在前向传播和反向传播中均能保持稳定的方差尺度。

命题 3.11 (Kaiming 初始化)

Kaiming 初始化（何恺明初始化）由 He et al. (2015) 提出，专门针对 ReLU 激活函数及其变体设计。由于 ReLU 在负半轴输出为 0，会截断约一半的激活值，使得输出的方差约为输入方差的一半。为补偿这一衰减，Kaiming 初始化将方差放大一倍：

- 适用激活函数：ReLU 及其变体（Leaky ReLU、PReLU 等）。
- 初始化方式：

$$\mathbf{W} \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}}}\right) \quad (3.19)$$

或从均匀分布采样：

$$\mathbf{W} \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{\text{in}}}}, \sqrt{\frac{6}{n_{\text{in}}}}\right) \quad (3.20)$$

初始化方法	适用激活函数	方差设置
Xavier	Sigmoid、Tanh	$\text{Var}(W) = 2/(n_{\text{in}} + n_{\text{out}})$
Kaiming	ReLU 及其变体	$\text{Var}(W) = 2/n_{\text{in}}$

Kaiming 初始化是 ResNet 等极深网络能够成功训练的关键因素之一——它确保了在上百层的网络中，梯度仍能稳定传播。

注[初始化方法的选择原则] 实践中的通用准则：

- 使用 ReLU 激活函数 → 默认选择 Kaiming 初始化。
- 使用 Sigmoid/Tanh → 选择 Xavier 初始化。
- 现代框架（PyTorch、TensorFlow）的默认初始化通常是 Kaiming Uniform，因其对 ReLU 系网络最为友好。

优化器

参数初始化后，需通过优化器（Optimizer）迭代更新参数以降低损失。优化器的选择直接影响收敛速度与最终性能。以下按历史演进梳理各类优化算法。

命题 3.12 (批量梯度下降与随机梯度下降)

批量梯度下降（**Batch Gradient Descent**）每次更新使用全部训练数据计算梯度，梯度准确但计算开销巨大，且易陷入局部极小值。

随机梯度下降（**Stochastic Gradient Descent, SGD**）每次更新仅使用单个样本计算梯度，更新频繁且引入了随机性，有助于跳出局部极小。然而，其梯度估计存在较大方差，收敛路径震荡剧烈。

小批量随机梯度下降（**Mini-batch SGD**）是两者的折中，每次从训练集中随机抽取 B 个样本（通常 $B = 32, 64, 128, 256$ ）计算梯度：

$$\mathbf{g}_t = \frac{1}{B} \sum_{i \in \mathcal{B}_t} \nabla_{\boldsymbol{\theta}} \mathcal{L}^{(i)}(\boldsymbol{\theta}_{t-1}) \quad (3.21)$$

参数更新为：

$$\boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \eta \cdot \mathbf{g}_t \quad (3.22)$$

其中 η 为学习率。Mini-batch SGD 兼顾了计算的稳定性和效率，是深度学习的标准实践。

命题 3.13 (动量法 (Momentum))

SGD 在峡谷形曲面（一个方向陡峭、另一方向平缓）中易产生剧烈震荡。动量法引入“惯性”概念，将历史梯度的指数加权平均纳入更新：

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + \mathbf{g}_t, \quad \boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \eta \cdot \mathbf{v}_t \quad (3.23)$$

其中 $\beta \in (0, 1)$ 为动量系数（通常取 0.9）。 $\mathbf{g}_t$ 为反向传播时计算的梯度。动量法的效果如同一个滚下山坡的球体：在平坦方向上持续加速，在陡峭方向上累积的震荡因方向交替而相互抵消，从而加速收敛。

命题 3.14 (NAG (Nesterov Accelerated Gradient))

NAG 是对动量法的改进，其核心思想是“超前看”——先按照当前动量方向走一步，再计算该“未来位置”的梯度：

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + \nabla \mathcal{L}(\boldsymbol{\theta}_{t-1} - \eta \beta \mathbf{v}_{t-1}), \quad \boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \eta \cdot \mathbf{v}_t \quad (3.24)$$

NAG 具有“矫正”作用：当动量方向即将冲过头时，超前梯度能提前修正方向，因此在凸优化中具有更快的收敛保证。实践中，NAG 通常比标准动量法表现更优。

命题 3.15 (自适应学习率方法)

SGD 及其动量变体对所有参数使用统一的学习率，这在处理稀疏特征或不同尺度的参数时效率低下。自适应方法为每个参数单独维护学习率。

AdaGrad：对频繁出现的特征使用较小的学习率，对稀疏特征使用较大的学习率。维护梯度平方的累加和：

$$\mathbf{s}_t = \mathbf{s}_{t-1} + \mathbf{g}_t^2, \quad \boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \frac{\eta}{\sqrt{\mathbf{s}_t + \epsilon}} \odot \mathbf{g}_t \quad (3.25)$$

优点：无需手动调节学习率，适合稀疏数据。缺点： $\mathbf{s}_t$ 单调递增，学习率最终趋近于 0，导致训练提前停止。

RMSProp：引入指数加权平均替代累加和，解决 AdaGrad 学习率归零的问题：

$$\mathbf{s}_t = \gamma \mathbf{s}_{t-1} + (1 - \gamma) \mathbf{g}_t^2, \quad \boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \frac{\eta}{\sqrt{\mathbf{s}_t + \epsilon}} \odot \mathbf{g}_t \quad (3.26)$$

其中 γ 通常取 0.9。RMSProp 在非凸优化中表现稳健，尤其适合 RNN 等动态网络。

AdaDelta：RMSProp 的延伸，进一步去除了对全局学习率 η 的依赖，使用参数更新量的均方根值进行缩放：

$$\boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \frac{\sqrt{\Delta \boldsymbol{\theta}_{t-1}^2 + \epsilon}}{\sqrt{\mathbf{s}_t + \epsilon}} \odot \mathbf{g}_t \quad (3.27)$$

其中 $\Delta \boldsymbol{\theta}_{t-1}^2$ 为历史更新量的指数加权平均。

命题 3.16 (Adam (Adaptive Moment Estimation))

Adam 是目前最广泛使用的优化器，它融合了动量法（一阶矩）和 RMSProp（二阶矩）的思想，并加入了偏差校正以应对初始阶段矩估计的偏差。

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t \quad (\text{一阶矩：梯度的均值}) \quad (3.28)$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2 \quad (\text{二阶矩：梯度的未中心化方差}) \quad (3.29)$$

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t} \quad (\text{偏差校正}) \tag{3.30}$$

$$\boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \eta \cdot \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} \tag{3.31}$$

Adam 的超参数默认值为： $\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$ 。

注[优化器选择指南]

- **SGD + Momentum**: 泛化性能通常最佳，尤其适合计算机视觉任务，但需精细调节学习率。
- **Adam**: 收敛快，超参数鲁棒，适合大多数任务，尤其是自然语言处理和生成模型。
- **AdamW**: Adam 的改进版，将权重衰减与梯度更新解耦，在大规模 Transformer 训练（如 BERT、GPT）中成为标配。

3.1.3 应用于图像分类的卷积神经网络

卷积神经网络自 2012 年以来经历了快速的架构演进，每一次突破都深化了我们对深度、宽度、连接模式与训练动态之间关系的理解。本小节系统梳理六个里程碑式架构的设计哲学与核心贡献。

3.1.3.1 AlexNet ——深度学习的破冰者

AlexNet [krizhevsky2012imagenet] 由 Alex Krizhevsky、Ilya Sutskever 和 Geoffrey Hinton 于 2012 年提出，在 ImageNet ILSVRC 挑战赛中以 **15.3%** 的 Top-5 错误率夺冠（第二名 26.2%），被公认为深度学习复兴的起点。

3.1.3.1.1 架构设计 AlexNet 包含 8 层可学习参数：5 层卷积层和 3 层全连接层。其核心设计包括：

- **ReLU 激活函数**: 首次在大规模网络中使用 ReLU（而非 Tanh），训练速度提升约 6 倍。ReLU 的稀疏激活特性也缓解了梯度消失问题。
- **局部响应归一化**: 在相邻通道间进行归一化，抑制过大的激活值，增强泛化能力。
- **重叠池化**: 步长小于池化窗口大小（步长 2，窗口 3×3），增加特征图的冗余性。
- **数据增强**: 随机裁剪、水平翻转和颜色扰动，将 120 万张训练图像扩充至 2048 倍。
- **Dropout**: 在全连接层中使用 Dropout（概率 0.5），有效防止过拟合。
- **GPU 并行**: 将网络分布在两块 GTX 580 GPU 上，采用分组卷积策略，突破当时显存限制。

3.1.3.1.2 历史意义与局限 AlexNet 证明了深度（8 层）和大规模数据（ImageNet）结合的有效性，并确立了 GPU 训练的标准范式。其局限在于：全连接层参数量巨大（约 5800 万），占总参数的 95%，导致过拟合风险高且计算冗余。

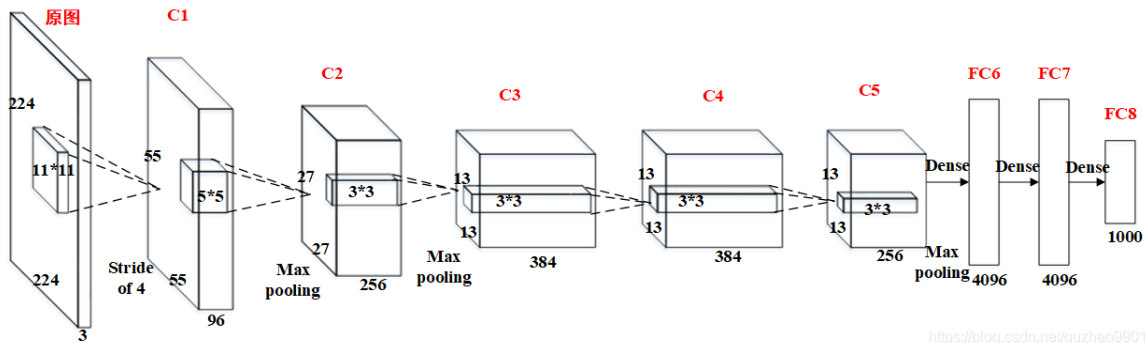


图 3.1: AlexNet 架构示意图（括号内为输出通道数）

3.1.3.2 VGG ——深度与简洁的极致统一

VGGNet [simonyan2014very] 由牛津大学视觉几何组（Visual Geometry Group）于 2014 年提出，在 ILSVRC 2014 中获得定位任务冠军和分类任务亚军（Top-5 错误率 7.3%）。

3.1.3.2.1 核心设计哲学 VGG 的最大贡献在于：用统一的 3×3 卷积核堆叠来替代大卷积核。

- 两层 3×3 卷积堆叠的等效感受野为 5×5 ，参数量为 $2 \times 3^2 = 18$ ，小于单个 5×5 卷积的 $5^2 = 25$ 。
- 三层 3×3 卷积堆叠的等效感受野为 7×7 ，参数量为 $3 \times 9 = 27$ ，远小于单个 7×7 卷积的 49。
- 更深的堆叠引入了更多的非线性变换（每层后接 ReLU），增强了网络的表达能力。

3.1.3.2.2 训练策略与参数初始化 VGG 的深层结构（如 VGG-16/19）在收敛后展现出清晰的层次化特征表达。一个关键的训练技巧是**层次参数继承**：将已收敛模型的前 4 个卷积层和后 3 个全连接层的权重，作为更深神经网络对应层的初始值，而其余卷积层则采用随机初始化。

- **前 4 层卷积**：捕获边缘、纹理等通用底层特征，这些特征在不同视觉任务中具有高度可迁移性，继承其权重可为深层网络提供稳定的梯度流起点。
- **后 3 个全连接层**：承载了高级语义组合的决策映射，若目标任务分类数与源任务一致，保留这部分权重可显著加速收敛并抑制初始损失震荡。
- **中间层随机初始化**：由于深层卷积特征具有任务特异性（如物体部件组合），随机初始化使网络能根据新数据重学从底层到语义的映射，避免源域偏见的干扰。

该策略本质上是**迁移学习中的部分微调**，在 VGG 的原始训练及后续微调任务中被广泛采用，有效缓解了深层网络在数据不足时的过拟合与收敛困难问题。

3.1.3.2.3 局限性与影响

- **参数量巨大**：VGG-16 全连接层参数量高达 1.24 亿（占总数 90%），存储和推理成本极高。
- **训练困难**：19 层网络已经面临梯度消失风险（需配合 He 初始化 [he2015delving] 和精细的学习率调度）。上述层次继承策略一定程度上减轻了极深模型从零训练的难度。
- **架构遗产**：VGG 证明了**网络深度比宽度更重要**，并确立了“小卷积核 + 大深度”的设计范式，影响深远。

3.1.3.3 Inception-v1 (GoogLeNet) ——多尺度卷积的并行探索

Inception-v1 [szegedy2015going] 由 Google 团队于 2014 年提出，获得 ILSVRC 2014 分类任务冠军（Top-5 错误率 6.7%）。其核心创新在于**Inception 模块**——在同一层并行执行不同尺度的卷积。

3.1.3.3.1 Inception 模块的设计动机 传统方法需要人工选择卷积核尺寸（ 3×3 、 5×5 、 7×7 ）。Inception 模块将选择权交给网络：

- 并行执行 1×1 、 3×3 、 5×5 卷积和 3×3 最大池化。
- 将所有输出在通道维度拼接。
- 网络自动学习“在哪个位置使用哪种尺度”最有效。

3.1.3.3.2 1×1 卷积的三大作用 Inception 模块中大量使用 1×1 卷积，其功能远非“逐点卷积”那么简单：

1. **降维**：压缩通道数，大幅降低计算量（通道数从 192 降至 96，减少 50%）。
2. **跨通道信息整合**：在通道维度上做线性组合，增强特征表达。
3. **增加非线性**： 1×1 卷积后接 ReLU，在不改变空间维度的前提下增加网络深度。

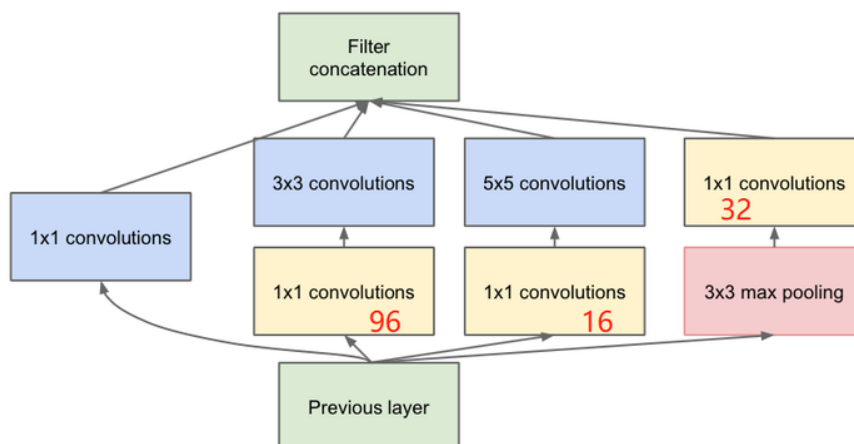


图 3.2: Inception 模块结构示意图 (含 1×1 瓶颈层降维)
A

3.1.3.3.3 辅助分类器 GoogLeNet 在中间层 (Inception-4a 和 Inception-4d) 添加了两个辅助分类器:

- 将中间层的特征图进行池化、卷积和全连接后输出分类结果。
- 训练时, 辅助损失乘以 0.3 加入总损失。
- 作用: 缓解梯度消失, 提供额外的正则化, 增强低层特征的判别性。

3.1.3.3.4 架构特点总结

- 深度 22 层 (含参数层), 参数量仅约 500 万 (AlexNet 的 1/12)。
- 计算量约 15 亿次浮点运算 (AlexNet 的 1/4)。
- 证明了 更高效的结构设计比单纯堆叠层数更重要。

3.1.3.4 BN-Inception —— 批量归一化带来的训练革命

BN-Inception [ioffe2015batch] 由 Sergey Ioffe 和 Christian Szegedy 于 2015 年提出, 在 Inception-v1 基础上引入 批量归一化 (Batch Normalization), 将 ImageNet 分类准确率提升至 4.8% Top-5 错误率。

3.1.3.4.1 批量归一化的核心思想 批量归一化对每一层的输入进行标准化, 使分布保持稳定:

$$\hat{x}^{(k)} = \frac{x^{(k)} - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y^{(k)} = \gamma^{(k)} \hat{x}^{(k)} + \beta^{(k)}$$

其中 μ_B 和 σ_B^2 为 mini-batch 的均值和方差, γ 和 β 为可学习的缩放和偏移参数。

3.1.3.4.2 BN 的五大贡献

1. 解决内部协变量偏移: 每层输入的分布变化得到抑制, 训练更稳定。
2. 允许使用更大的学习率: 梯度被限制在可控范围内, 训练加速。
3. 降低对初始化的敏感性: 即使使用较差的初始化 (如原始 Xavier), 网络仍能收敛。
4. 提供轻微的正则化效果: mini-batch 的随机统计量引入噪声, 类似 Dropout。
5. 允许移除 Dropout: BN-Inception 中部分层移除 Dropout 后, 性能反而提升。

3.1.3.4.3 在 Inception 中的部署

- BN 插入在每个卷积层之后、激活函数之前。
- 使用 BN 后, 学习率从 0.001 提高到 0.01。
- 训练收敛速度提升约 14 倍。

- 模型达到相同精度所需的迭代次数减少 60%。
- BN-Inception 标志着 归一化技术成为深度网络训练的标配组件。

3.1.3.5 Inception-v3

Inception-v3 [szegedy2016rethinking] 是 Inception 系列中应用最广泛的版本之一，它在 Inception-v2（即 BN-Inception）的基础上，进一步对卷积操作进行分解与优化，在不显著增加计算量的前提下大幅提升了模型表达能力。其核心设计哲学可概括为：将大尺寸卷积分解为小尺寸卷积的组合，同时利用批量归一化保持训练稳定性。

3.1.3.5.1 核心改进一：卷积分解（Factorized Convolutions） Inception-v3 对 Inception 模块内部的大卷积核进行了两种关键分解：

1. **5×5 卷积分解为两个 3×3 卷积堆叠**：原始的 Inception-v1 中，5×5 卷积核参数量为 $5 \times 5 = 25$ ，而将其替换为两个串联的 3×3 卷积后，参数量降为 $2 \times (3 \times 3) = 18$ ，减少了约 28% 的参数量，同时由于多了一层非线性变换（每层后均接 BN 和 ReLU），模型的表征能力反而有所提升。
2. **大尺寸非对称卷积分解（ $n \times n$ 分解为 $1 \times n$ 和 $n \times 1$ ）**：对于较大感受野（如 7×7、5×5 等），Inception-v3 将其分解为两个非对称的卷积核： $1 \times n$ 和 $n \times 1$ 。例如，将 3×3 卷积进一步分解为 1×3 和 3×1 的串联组合。这种分解在保持相同感受野的同时，将参数量从 n^2 减少到 $2n$ ，在深层网络中尤其有效。

3.1.3.5.2 核心改进二：三种 Inception 模块的混合使用 Inception-v3 不再使用单一的 Inception 模块结构，而是根据特征图的尺寸和深度，设计了三种不同形态的 Inception 模块，分别应用于网络的不同阶段：

表 3.1: Inception-v3 的三种模块类型及其适用阶段

模块类型	特征图尺寸	核心结构	使用次数
Type-A	35×35	1×1 、 3×3 （分解前）、 3×3 池化	3 次
Type-B	17×17	$1 \times 3 + 3 \times 1$ 分解卷积	5 次
Type-C	8×8	1×3 、 3×1 膨胀分解	2 次

这种分阶段设计的思想在于：浅层使用标准卷积保留空间细节，中层使用非对称卷积减少计算量，深层使用膨胀卷积扩大感受野。

3.1.3.5.3 核心改进三：高效的下采样策略（Reduction Block） Inception-v3 摒弃了传统直接使用池化层进行下采样的方式，而是设计了专门的 **Reduction Block** 用于缩减特征图的尺寸。

该模块通过将池化分支与卷积分支并行，再将结果拼接，在降低空间维度的同时避免了信息瓶颈（Information Bottleneck）问题。Reduction Block 中同样在每个卷积层后使用 BN 层，确保梯度的稳定流动。

表 3.2: Inception-v3 与 BN-Inception 的核心差异

特性	BN-Inception (V2)	Inception-v3
卷积分解	仅将 5×5 分解为两个 3×3	进一步将 3×3 分解为 $1 \times 3 + 3 \times 1$
非对称卷积	未使用	大量使用（ 1×7 、 7×1 等）
模块类型	单一模块重复堆叠	三种模块混合使用
下采样方式	使用 MaxPooling 直接下采样	专用 Reduction Block
辅助分类器	移除	重新引入（带 BN）但权重降低

3.1.3.5.4 与 BN-Inception 的结构对比

3.1.3.5.5 训练策略 Inception-v3 的训练策略继承了 BN-Inception 的优点，并进一步强化：

- **批量归一化**：每个卷积层之后紧跟 BN 层，顺序为 Conv → BN → ReLU，卷积层的偏置项统一设置为 False。

- **标签平滑 (Label Smoothing)**: 将硬标签 (one-hot) 替换为软标签, 即 $y_{\text{true}} = (1 - \epsilon) \cdot y_{\text{one-hot}} + \epsilon/K$, 其中 K 为类别数。该策略有效防止模型过拟合, 并提高了泛化能力。
- **辅助分类器**: Inception-v3 重新引入了辅助分类器, 但对其进行了 BN 化处理, 并降低了其损失权重 (约 0.3), 使其仅作为正则化手段而非梯度传播主干。
- **RMSProp 优化器**: 配合 BN 的高学习率特性, 采用 RMSProp 优化器, 初始学习率设为 0.045, 每两个 epoch 以 0.94 的指数衰减率进行衰减。

3.1.3.5.6 性能表现 在 ImageNet-1K 数据集上, Inception-v3 达到了当时的最优水平: Top-1 准确率达 78.8%, Top-5 准确率达 94.4%, 参数量约为 23.8M, 计算量约为 5.7 亿次乘加运算 (MAdds), 在精度与效率之间取得了良好的平衡。由于其轻量化和高精度的特点, Inception-v3 至今仍是许多计算机视觉任务的首选骨干网络之一。

3.1.3.5.7 小结 Inception-v3 通过卷积分解、模块多样化、下采样优化和精细化训练策略, 在 BN-Inception 的基础上实现了显著的性能跃升。其设计思想——**在保持计算效率的前提下, 通过结构分解提升网络深度和宽度**——深刻影响了后续如 Xception、MobileNet 等轻量级网络的设计。Inception-v3 证明了: 在 BN 的稳定下, 极致的结构分解不仅不会损害性能, 反而能释放网络的潜在表达能力。

3.1.3.6 ResNet

残差网络 (Residual Network, ResNet) 由 He 等人于 2015 年提出, 其核心创新在于通过**残差连接 (Skip Connection)**有效解决了极深网络的**退化问题 (Degradation Problem)**, 使得网络深度可扩展至百层乃至千层 [citation:8][citation:12]。

3.1.3.6.1 退化问题与残差学习的动机 传统卷积神经网络 (如 VGG) 的性能随层数增加先提升后饱和, 继续加深反而导致训练误差升高, 且该现象并非由过拟合引起, 而是深层网络难以优化。ResNet 的提出源于一个关键假设: 若将网络设计为学习**残差映射 (Residual Mapping)**而非直接映射, 则优化难度将大幅降低。具体地, 令堆叠的非线性层拟合残差函数 $\mathcal{F}(\mathbf{x}) = \mathcal{H}(\mathbf{x}) - \mathbf{x}$, 则原始映射 $\mathcal{H}(\mathbf{x})$ 可重构为 $\mathcal{F}(\mathbf{x}) + \mathbf{x}$ 。当最优映射接近恒等时, 残差映射趋近于零, 更易优化。

3.1.3.6.2 残差块的基本结构 一个标准残差块可表示为:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{\mathcal{W}_i\}) + \mathbf{x}$$

其中 $\mathbf{x}$ 为该残差块的输入, $\mathcal{F}$ 为待学习的残差函数 (通常由两个或三个卷积层堆叠而成), $\mathbf{x}$ 通过**捷径连接 (Shortcut Connection)**与 $\mathcal{F}$ 的输出逐元素相加, 随后通过激活函数 (如 ReLU) 得到输出 $\mathbf{y}$ 。

3.1.3.6.3 梯度传播的数学分析 残差结构对梯度传播具有显著优势。设第 l 个残差单元的输入为 $\mathbf{x}_l$, 输出为 $\mathbf{x}_{l+1}$, 则:

$$\mathbf{x}_{l+1} = \mathbf{x}_l + \mathcal{F}(\mathbf{x}_l, \mathcal{W}_l)$$

递归展开至深层单元 L , 可得:

$$\mathbf{x}_L = \mathbf{x}_l + \sum_{i=l}^{L-1} \mathcal{F}(\mathbf{x}_i, \mathcal{W}_i)$$

该形式表明, 任意深层单元的特征 $\mathbf{x}_L$ 均可表示为浅层单元特征 $\mathbf{x}_l$ 与一系列残差函数的叠加。在反向传播中, 损失函数 $\mathcal{E}$ 对浅层特征的梯度为:

$$\frac{\partial \mathcal{E}}{\partial \mathbf{x}_l} = \frac{\partial \mathcal{E}}{\partial \mathbf{x}_L} \left(1 + \frac{\partial}{\partial \mathbf{x}_l} \sum_{i=l}^{L-1} \mathcal{F}(\mathbf{x}_i, \mathcal{W}_i) \right)$$

式中， $\frac{\partial \mathcal{E}}{\partial \mathbf{x}_L}$ 项可实现梯度从极深处到浅层的**直接传播**，不经过任何权重层，有效缓解了梯度消失问题，使得极深网络的训练成为可能。

3.1.3.6.4 两种残差块类型 根据网络深度的不同，ResNet 采用两种残差块设计：

1. **基本块 (Basic Block)**: 包含两个 3×3 卷积层，每个卷积层后接批量归一化 (Batch Normalization, BN) 与 ReLU 激活。该结构用于 ResNet-18 与 ResNet-34 等较浅网络。
2. **瓶颈块 (Bottleneck Block)**: 包含三个卷积层——先通过 1×1 卷积将通道数压缩为输入通道的 $1/4$ ，再经 3×3 卷积提取特征，最后通过 1×1 卷积将通道数恢复至原始维度。该设计大幅减少了参数量，适用于 ResNet-50、ResNet-101 及 ResNet-152 等深层网络。例如，对于 2048 通道的瓶颈块，参数量约为 4.5M，而同等宽度的基本块则需 75.5M 参数，计算效率提升近 17 倍。

3.1.3.6.5 快捷连接的维度匹配 残差块的输入 $\mathbf{x}$ 与输出 $\mathcal{F}$ 需保持相同维度方可逐元素相加。当特征图尺寸减半或通道数翻倍时，快捷连接需进行维度调整，主流方式有两种 [citation:7]：

- **零填充 (Zero-padding Shortcut)**: 在新增通道上补零，不引入额外参数。
- **投影连接 (Projection Shortcut)**: 使用 1×1 卷积调整通道数，引入少量可学习参数，在维度不匹配时使用。

3.1.3.6.6 小结 ResNet 通过引入残差连接，从架构层面解决了深层网络的退化问题。其残差结构不仅使梯度的直接传播成为可能，还通过瓶颈块设计在深层网络中保持了计算可行性。ResNet 在 ILSVRC 2015 中夺得分类、检测、定位等多项冠军 [citation:12]，其思想深度影响了后续的 DenseNet、ResNeXt、Inception-ResNet 等架构，是现代深度卷积神经网络的基石之一。

3.1.4 应用于图像目标检测的卷积神经网络

3.1.4.1 背景知识：评测指标

3.1.4.1.1 IoU 交并比 (Intersection over Union, IoU) 是目标检测中最基础的评估指标，用于衡量预测边界框 (Predicted Bounding Box) 与真实边界框 (Ground Truth Box) 之间的重合程度。其定义为两个边界框交集面积与并集面积的比值：

$$\text{IoU} = \frac{|\mathcal{B}_{\text{pred}} \cap \mathcal{B}_{\text{gt}}|}{|\mathcal{B}_{\text{pred}} \cup \mathcal{B}_{\text{gt}}|}$$

其中 $\mathcal{B}_{\text{pred}}$ 为预测框， $\mathcal{B}_{\text{gt}}$ 为真实框。IoU 的取值范围为 $[0, 1]$ ，值越大表示两个框的重合度越高。在目标检测中，IoU 通常用作判断检测结果正负样本的阈值标准——当预测框与某个真实框的 IoU 超过预设阈值（如 0.5）时，该预测被视为正确检测 (True Positive)，否则视为错误检测 (False Positive)。

3.1.4.1.2 mAP 平均精度均值 (mean Average Precision, mAP) 是目标检测任务中最核心的评价指标，用于综合衡量模型在多个类别上的检测性能。其计算过程涉及精度 (Precision) 与召回率 (Recall) 两个基本概念：

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

其中 TP (True Positive) 为正确检测的正样本数，FP (False Positive) 为错误检测的负样本数，FN (False Negative) 为未被检测出的正样本数。对于单个类别，可通过调整置信度阈值绘制精度-召回率曲线 (Precision-Recall Curve)，曲线下的面积即为该类别的平均精度 (Average Precision, AP)：

$$AP = \int_0^1 p(r) dr$$

在实际计算中，AP 通常采用近似数值积分方式，如 VOC 数据集采用的 11 点插值法或 COCO 数据集采用的 101 点插值法。mAP 则是对所有类别的 AP 值取平均：

$$mAP = \frac{1}{C} \sum_{c=1}^C AP_c$$

其中 C 为类别总数。mAP 综合反映了模型在多个类别上的定位精度与分类精度，数值越高表示检测性能越好。

3.1.4.2 R-CNN

区域卷积神经网络（Regions with CNN features, R-CNN）是由 Girshick 等人于 2014 年提出的开创性工作，首次将深度学习引入目标检测领域，奠定了“基于区域提议的两阶段检测范式”的基础。

R-CNN 的核心流程分为三个步骤：

1. **区域提议生成**：采用选择性搜索（Selective Search）算法，在输入图像上通过层次化区域融合生成大量候选框。为在送入卷积神经网络之前控制候选数量并增加空间多样性，算法对融合过程中产生的所有候选框按“层级 × RND”指标排序，并执行第一次非极大值抑制（NMS），去除冗余重叠框，最终保留约 2000 个候选区域（Region Proposals）作为后续处理的对象。
2. **特征提取**：将每个候选区域缩放到固定的尺寸（ 227×227 ），然后分别输入卷积神经网络（如 AlexNet）提取固定长度的特征向量。此处每个区域独立经过 CNN，存在大量的重复计算。
3. **分类与回归**：将提取到的特征向量输入支持向量机（SVM）进行二分类，每个候选框获得关于每个类别的置信度得分。随后，对每个类别执行第二次非极大值抑制（NMS）——在属于同一类别的候选框中，仅保留得分最高的框，抑制所有与其交并比（IoU）超过阈值（通常为 0.5）的冗余框，确保每个目标最终仅由一个检测框表示。完成 NMS 后，对保留的候选框进一步通过线性回归器进行位置精调（Bounding Box Regression），以提升定位精度。

命题 3.17 (选择性搜索的层次化分组算法)

R-CNN 采用选择性搜索（Selective Search）作为底层区域提议算法，其核心是一个多策略层次化分组框架，通过以下机制生成高质量的候选区域：

1. **初始区域生成**：采用基于图的图像分割方法（Felzenszwalb-Huttenlocher 算法），在不同分割阈值下对输入图像进行过分割，生成初始的超像素区域集合，作为层次化融合的原子单元。
2. **层次化融合过程**：对于每一组初始分割结果，迭代执行以下操作直至整张图像合并为单一区域：
 - (a). 计算所有相邻区域间的综合相似度（综合考虑颜色、纹理、大小、重叠度等互补度量）；
 - (b). 每次合并相似度最高的两个相邻图像区域；
 - (c). 计算合并后的新区域与其所有相邻区域的相似度；
 - (d). 重复上述步骤，直到所有图像区域合并为一张完整图像。
3. **多策略并行枚举**：为充分覆盖目标可能出现的各种模式，该算法在多个维度上并行调用上述层次化分组过程：
 - 不同分割阈值：调整初始过分割的粒度，以捕获不同尺度的物体；
 - 不同色彩空间：使用 RGB、HSV、Lab 等多种颜色空间，增强对光照变化的鲁棒性；
 - 不同相似度组合：动态调整颜色、纹理、大小、形状（重叠度）四种相似性度量的权重，适应不同类型的目标。
4. **候选框排序与去冗余**：将所有合并策略下产生的全部位置框（包含从局部超像素到全局图像的所有

尺度) 按以下规则处理:

- (a). 按层级 (**Hierarchical Level**) 排序: 定义覆盖整张图像的区域层级为 1, 随着区域不断细分级递增, 越底层的区域层级越大;
 - (b). 计算排序指标: 层级 $\times$ RND (RND 为随机扰动项, 用于打破同层级间的固定顺序, 增加多样性);
 - (c). 按该指标升序排列所有候选框, 并依次执行非极大值抑制 (NMS) 去除冗余重叠框。
5. 固定数量采样: 从排序去重后的候选框列表中, 取固定数量 (R-CNN 中取约 2000 个) 的顶层候选区域, 作为后续卷积神经网络的特征提取输入。

该算法的核心优势在于: 通过多策略并行 (多种分割阈值、色彩空间、相似度组合) 与层次化融合的结合, 实现了对目标位置、尺度、形变的高召回率覆盖; 同时通过层级排序与 **NMS** 去冗余, 将候选区域数量从滑动窗口的百万级压缩至约 2000 个, 使得深度学习模型在目标检测中的高效应用成为可能。

命题 3.18 (非极大值抑制 (NMS))

在目标检测算法中, 无论是区域提议生成阶段还是最终检测输出阶段, 候选框之间普遍存在严重的空间重叠现象——同一目标往往被数十个位置、尺度各异的边框同时覆盖。为消除这种冗余, 非极大值抑制 (**Non-Maximum Suppression, NMS**) 被用作标准的后处理策略, 其核心思想可概括为: 对于高度重叠的候选框集合, 仅保留得分最高者, 抑制其余冗余框。

NMS 的具体执行流程如下:

1. 排序: 将所有候选框按照给定的得分 (在检测阶段为分类置信度, 在提议生成阶段可为层级与随机项的复合指标) 从高到低排列。
2. 循环选择与抑制:
 - (a). 从当前集合中取取得分最高的候选框, 将其移入最终保留集合;
 - (b). 计算该框与集合中所有剩余框的交并比 (Intersection over Union, IoU):

$$\text{IoU}(b_i, b_j) = \frac{\text{area}(b_i \cap b_j)}{\text{area}(b_i \cup b_j)}$$
 - (c). 剔除所有 IoU 大于预设阈值 (通常设为 0.5 ~ 0.7) 的候选框, 因为它们与已保留框在空间上高度重合, 极大概率指向同一目标。
3. 终止条件: 重复上述选择-抑制循环, 直至候选集合为空, 最终保留的框即为经过非极大值抑制后的代表性检测结果。

NMS 的本质是一种贪心的局部最优选择策略: 在重叠簇中, 它选择得分最高的框作为该簇的代表, 同时丢弃簇内其他成员。该操作确保了每个目标最终仅由一个检测框表示, 从而有效降低误检率并减少后续计算开销, 已成为两阶段目标检测框架中不可或缺的标准组件。

R-CNN 在 VOC 2012 数据集上将 mAP 从 35.1% 大幅提升至 53.3%, 具有里程碑意义。然而, 其存在明显缺陷:

- **训练速度慢:** 需要分阶段训练 CNN、SVM 和回归器, 流程复杂。
- **推理速度慢:** 对每张图像的约 2000 个候选区域分别进行 CNN 前向传播, 在 GPU 上仍需约 47 秒/张, 无法满足实时应用需求。
- **存储开销大:** 所有候选区域的特征需写入磁盘, 占用大量存储空间。

3.1.4.3 Fast R-CNN

Fast R-CNN [girshick2015fast] 于 2015 年提出, 通过两项关键创新解决了 R-CNN 的核心效率瓶颈:

1. **共享卷积计算 (Shared Convolutional Computation):** Fast R-CNN 不再对每个候选区域独立进行 CNN 前向传播, 而是将整张图像一次性输入 CNN, 生成全图的特征图 (Feature Map)。随后, 对每个候选区域,

通过感兴趣区域池化（**Region of Interest Pooling, RoI Pooling**）从共享特征图上提取固定尺寸的特征向量。这一设计使得卷积计算在全图范围内共享，大幅减少了计算量。

2. **多任务联合训练（Multi-task Training）**: Fast R-CNN 将分类损失与边界框回归损失合并为一个统一的损失函数，实现了端到端的联合训练：

$$\mathcal{L}(p, u, t^u, v) = \mathcal{L}_{\text{cls}}(p, u) + \lambda[u \geq 1]\mathcal{L}_{\text{loc}}(t^u, v)$$

其中 $\mathcal{L}_{\text{cls}}$ 为交叉熵分类损失， $\mathcal{L}_{\text{loc}}$ 为 Smooth L1 回归损失， λ 为权衡系数。分类与回归共享同一组特征，避免了 R-CNN 的分阶段训练。

命题 3.19 (感兴趣区域池化 (RoI Pooling))

Fast R-CNN 的核心效率突破来源于感兴趣区域池化（**Region of Interest Pooling, RoI Pooling**）操作，它解决了 R-CNN 中因候选区域独立过 CNN 而导致的大量重复计算问题。RoI Pooling 的本质是一种基于共享特征图的非均匀采样与尺寸归一化机制，其输入为一张经过 CNN 计算得到的整图特征图（尺寸为 $H \times W \times C$ ）以及一个候选框在原图上的坐标 (x_1, y_1, x_2, y_2) ，输出为一个固定尺寸（如 $7 \times 7 \times C$ ）的特征图。

RoI Pooling 的具体执行流程如下：

1. 坐标映射：将候选框在原图上的坐标按 CNN 的总下采样倍数（通常记为 s ，如 AlexNet 的 $s = 16$ ）映射到特征图坐标系中：

$$x'_1 = \left\lfloor \frac{x_1}{s} \right\rfloor, \quad y'_1 = \left\lfloor \frac{y_1}{s} \right\rfloor, \quad x'_2 = \left\lfloor \frac{x_2}{s} \right\rfloor, \quad y'_2 = \left\lfloor \frac{y_2}{s} \right\rfloor$$

映射后的 RoI 区域在特征图上的尺寸为 $h = y'_2 - y'_1$ ， $w = x'_2 - x'_1$ 。

2. 网格划分：将映射后的 $h \times w$ 区域划分为目标尺寸 $H_{\text{out}} \times W_{\text{out}}$ 个网格（bin）。每个网格的高度和宽度分别为：

$$\text{bin_h} = \left\lceil \frac{h}{H_{\text{out}}} \right\rceil, \quad \text{bin_w} = \left\lceil \frac{w}{W_{\text{out}}} \right\rceil$$

使用向上取整（ $\lceil \cdot \rceil$ ）确保网格能够完整覆盖整个 RoI 区域，且各网格大小允许不一致以适应不可整除的情形。

3. 最大池化：对每个网格内的所有特征值独立执行最大池化（Max Pooling）操作，取该网格内的最大值作为输出。该操作在每个通道上独立进行，通道间互不混合。最终得到 $H_{\text{out}} \times W_{\text{out}} \times C$ 的固定尺寸输出特征图。

RoI Pooling 的关键特性与贡献如下：

- 计算共享：所有候选框复用同一张全图特征图，CNN 的卷积计算仅执行一次，将 Fast R-CNN 的训练与推理速度相较 R-CNN 提升数十倍；
- 尺寸统一：无论候选框在原图中的尺寸如何变化，经 RoI Pooling 后均输出固定维度的特征图，从而无缝衔接后续全连接层；
- 空间信息保留：通过网格划分与最大池化，每个输出单元对应原图中一个特定空间区域，保留了粗粒度的空间结构信息。

值得注意的是，RoI Pooling 在坐标映射与网格划分过程中涉及两次量化取整操作，这会在空间定位上引入轻微的偏差。对于分类任务（需平移不变性），该偏差可接受；但对于像素级分割任务，后续工作（如 Mask R-CNN）提出了 RoI Align 以消除量化误差。但在 Fast R-CNN 的框架中，RoI Pooling 以其简洁高效的设计，成为连接共享卷积特征与任务特定网络之间的关键桥梁。

命题 3.20 (Fast R-CNN 中的背景判定机制)

在目标检测任务中，区域提议算法（如 Selective Search）生成的约 2000 个候选框中，通常仅有 10% 30% 包含完整物体（前景），其余均为不包含任何完整目标的背景区域。因此，检测模型必须显式地学习判别候选框属于前景（**Foreground**）还是背景（**Background**），以在推理阶段过滤大量无效检测，避免误报。

Fast R-CNN 通过以下机制实现前景-背景的判别：

1. 类别空间扩展：将背景视为一个独立的语义类别，纳入多分类框架。假设数据集中共有 K 类待检测物体，则模型输出的类别数为 $K + 1$ ，其中第 0 类固定为背景类，第 1 至 K 类对应具体物体类别。
2. 训练阶段的标签分配：在训练过程中，根据候选框与所有真实标注框 (Ground Truth) 的交并比 (IoU)，为每个候选框分配类别标签 u ：
 - 正样本（前景）：若某候选框与某一真实框的 IoU 超过阈值（通常设为 0.5），则将其标签 u 设为该真实框对应的物体类别 ID ($u \geq 1$)；
 - 负样本（背景）：若候选框与所有真实框的 IoU 均低于低阈值（通常为 0.1）或介于 0.1 与 0.5 之间，则将其标签设为 $u = 0$ （背景类）。
3. 分类损失中的背景监督：分类损失 $\mathcal{L}_{\text{cls}}(p, u) = -\log p_u$ 对所有候选框（包括前景和背景）均计算。对于背景框 ($u = 0$)，该损失推动模型输出 p_0 （背景概率）尽可能接近 1，从而学会识别“这不是任何物体”。
4. 推理阶段的背景过滤：在测试时，模型对每个候选框输出 $K + 1$ 维概率分布 $p = (p_0, p_1, \dots, p_K)$ 。若 p_0 （背景概率）高于所有物体类别的概率，则该候选框被视为背景并被丢弃；反之，取最大概率对应的物体类别作为检测结果。

该机制的核心贡献在于：将背景判别从 R-CNN 中 SVM 二分类器的独立建模，转变为统一 Softmax 多分类框架中的一类，使背景与物体类别在同一特征空间下共同学习，避免了分阶段训练带来的特征割裂问题。

命题 3.21 (Fast R-CNN 中的多任务联合训练机制)

Fast R-CNN 的另一核心创新在于将物体分类与边界框回归整合为同一网络框架下的多任务联合训练，实现端到端的参数优化。本节从网络结构、训练数据构建、损失函数设计与推理策略四个层面展开论述。

(a) 分类器与回归器的网络结构关系

分类器与回归器并非同一个网络，但共享了绝大部分参数。其结构关系如下：

- 共享部分：CNN 卷积层、RoI Pooling 层以及若干全连接层 (FC 层) 为分类与回归任务所共用。所有候选框先经过共享网络提取统一的高维特征向量；
- 独立分支：在共享 FC 层之后，网络分叉为两个独立的全连接层分支：
 - 分类分支：输出维度为 $K + 1$ ，后接 Softmax 函数，输出各类别（含背景）的概率分布 $p = (p_0, p_1, \dots, p_K)$ ；
 - 回归分支：输出维度为 4，直接输出边界框偏移量 $t^u = (t_x, t_y, t_w, t_h)$ ，不经过 Softmax。
- 参数更新：两个分支的梯度分别反向传播，共同更新共享网络参数，而各分支独立的全连接层仅由对应任务的梯度更新。

(b) 回归目标的构建：候选框与真实框的匹配策略

回归器的训练需要为每个候选框定义回归目标 v ，其前提是建立候选框与真实框的对应关系。由于候选框数量（约 2000）与真实框数量 (N) 不相等，且两者坐标不同，必须通过匹配策略建立对应：

1. 计算所有候选框与所有真实框之间的 IoU 矩阵，尺寸为 $2000 \times N$ ；
2. 正样本匹配：若某候选框与某一真实框的 $\text{IoU} > 0.5$ ，则将该候选框标记为该真实框对应的正样本（类别 ID $u \geq 1$ ），并计算回归目标 v ；
3. 强制匹配：对于每个真实框，若不存在任何候选框与其 $\text{IoU} > 0.5$ ，则强制选择与其 IoU 最大的候选框作为正样本，确保每个真实物体至少有一个候选框负责学习；
4. 负样本标记：IoU 介于 0.1 与 0.5 之间的候选框标记为背景 ($u = 0$)， $\text{IoU} < 0.1$ 的候选框直接丢弃；
5. 多对一关系：同一真实框可匹配多个候选框（所有 $\text{IoU} > 0.5$ 的候选框均为正样本），这为回归器提供了不同偏移程度下的多样本训练，增强泛化能力。

对于每个正样本，回归目标 v 由候选框 $P = (P_x, P_y, P_w, P_h)$ 与匹配的真实框 $G = (G_x, G_y, G_w, G_h)$ 计算：

$$v_x = \frac{G_x - P_x}{P_w}, \quad v_y = \frac{G_y - P_y}{P_h}, \quad v_w = \log\left(\frac{G_w}{P_w}\right), \quad v_h = \log\left(\frac{G_h}{P_h}\right)$$

(c) 损失函数中的回归建模

回归损失仅对正样本计算，形式为 Smooth L1 损失：

$$\mathcal{L}_{\text{loc}}(t^u, v) = \sum_{i \in \{x, y, w, h\}} \text{smooth}_{L1}(t_i^u - v_i)$$

其中：

$$\text{smooth}_{L1}(x) = \begin{cases} 0.5x^2 & \text{if } |x| < 1 \\ |x| - 0.5 & \text{otherwise} \end{cases}$$

完整的多任务损失函数为：

$$\mathcal{L}(p, u, t^u, v) = \underbrace{-\log p_u}_{\mathcal{L}_{\text{cls}}} + \lambda \underbrace{[u \geq 1] \sum_i \text{smooth}_{L1}(t_i^u - v_i)}_{\mathcal{L}_{\text{loc}}}$$

其中指示函数 $[u \geq 1]$ 确保背景框不参与回归训练。

(d) 推理阶段的 NMS 策略

训练与推理阶段在候选框处理策略上存在本质差异：

- 训练阶段 不 使用 NMS：多个候选框匹配同一真实框（多对一）是有益的，它为回归器提供了不同偏移程度下的多样本，使模型学会从各种初始位置修正到真实位置，增强泛化能力。若在训练时做 NMS 去重，将大幅减少正样本数量，损害模型性能；
- 推理阶段 必须 使用 NMS：测试时，同一物体可能被多个候选框同时检测到（每个框均输出高置信度）。为避免冗余输出，需对每个类别独立执行 NMS：按分类置信度降序排列候选框，保留得分最高的框，抑制所有与其 IoU 超过阈值（通常为 0.5）的其他框，确保每个物体最终仅由一个检测框表示。

多任务联合训练的核心优势在于：分类与回归共享同一组卷积特征，通过统一的损失函数和联合反向传播，两个任务相互促进——分类任务提供语义信息指导回归，回归任务提供空间信息辅助分类，最终学习到的特征兼具判别力与定位精度，从根本上解决了 R-CNN 中三个阶段独立训练所导致的特征割裂与训练流程繁琐的问题。



Fast R-CNN 的推理速度提升约 200 倍（从 47 秒/张降至 0.32 秒/张），训练速度提升约 9 倍，同时 mAP 进一步提升至 68.4%（VOC 2012）。然而，其区域提议仍依赖外部算法（选择性搜索），成为新的性能瓶颈——区域提议的生成时间甚至超过了模型推理本身。

3.1.4.4 Faster R-CNN

Faster R-CNN [ren2015faster] 于 2015 年提出，通过引入区域提议网络（Region Proposal Network, RPN），将区域提议的生成过程也纳入神经网络框架，实现了完整的端到端目标检测。

RPN 的核心思想是锚框（Anchors）机制：在特征图的每个位置预先定义多个不同尺度和长宽比的锚框（如在每个位置设置 3 种尺度 $\times$ 3 种长宽比 = 9 个锚框）。RPN 通过一个滑动窗口的卷积层，同时预测每个锚框的目标性得分（Objectness Score）（判断该锚框是否包含物体）和边界框偏移量（用于调整锚框位置）。RPN 的输出经过非极大值抑制（NMS）后，筛选出约 300 个高质量候选框，输入到 Fast R-CNN 检测头进行最终的分类与回归。

命题 3.22 (Faster R-CNN 中的提议框生成与 RoI 特征提取链条)

设输入图像为 $\mathcal{I} \in \mathbb{R}^{H \times W \times 3}$, 主干网络 (如 VGG16) 的总下采样步长为 S (通常 $S = 16$)。则从原始图像到检测头固定尺寸特征输入的完整数据变换链条, 由以下六个顺序执行的原子操作构成, 其中除 Step 4 中的 NMS (用于筛选) 外, 其余步骤均为可微操作。

Step 0: 共享卷积特征提取. 将整张图像 $\mathcal{I}$ 一次性输入主干卷积网络 (如 VGG16 去掉最后的全连接层), 执行前向传播:

$$\mathcal{F}_{\text{shared}} \leftarrow \text{Backbone}(\mathcal{I}; \Theta_{\text{backbone}}),$$

其中 $\mathcal{F}_{\text{shared}} \in \mathbb{R}^{\frac{H}{S} \times \frac{W}{S} \times C}$ 为空间分辨率降低 S 倍的共享特征图。该特征图同时服务于 RPN 提议生成与后续检测头的分类与回归, 是计算共享的物理基础。

Step 1: 锚框生成. 在 $\mathcal{F}_{\text{shared}}$ 的每个空间位置 (i, j) (对应原图上以 $(i \cdot S, j \cdot S)$ 为中心的 $S \times S$ 感受野区域), 预设 K 个锚框 (Anchor Boxes), 作为初始候选框的先验参考。通常配置为 $K = 9$, 即 3 种不同尺度 (如 $128^2, 256^2, 512^2$) 与 3 种不同长宽比 (如 $1:2, 1:1, 2:1$) 的组合。所有锚框的坐标被映射到原始图像空间:

$$\mathcal{A} \leftarrow \{(x_a, y_a, w_a, h_a)_m\}_{m=1}^M, \quad M = \frac{H}{S} \cdot \frac{W}{S} \cdot K,$$

其中 (x_a, y_a) 为锚框中心坐标, (w_a, h_a) 为锚框的宽度与高度。

Step 2: RPN 前向预测. 在共享特征图 $\mathcal{F}_{\text{shared}}$ 上, RPN 头部 (一个小型卷积网络, 通常为 3×3 卷积 + 两个 1×1 卷积) 并行预测两个输出:

- 分类得分: $\text{Score}_m \in [0, 1]$, 对每个锚框输出其为前景的概率, 输出维度为 $M \times 2$ (二分类: 前景/背景);
- 边界框偏移量: $(\Delta x, \Delta y, \Delta w, \Delta h)_m \in \mathbb{R}^4$, 对每个锚框输出 4 个编码偏移量, 输出维度为 $M \times 4$ 。

形式化地:

$$\text{Score}_m, (\Delta x, \Delta y, \Delta w, \Delta h)_m \leftarrow \text{RPN_Head}(\mathcal{F}_{\text{shared}}; \Theta_{\text{rpn}}).$$

Step 3: 边界框解码. 利用 RPN 预测的偏移量对锚框进行调整, 生成原始提议框集合 $\mathcal{P}_{\text{raw}}$ 。解码变换为:

$$\begin{aligned} x &= \Delta x \cdot w_a + x_a, & y &= \Delta y \cdot h_a + y_a, \\ w &= \exp(\Delta w) \cdot w_a, & h &= \exp(\Delta h) \cdot h_a. \end{aligned} \quad (3.32)$$

其中 $\Delta x, \Delta y$ 控制框中心的平移 (相对于锚框宽高进行归一化), $\Delta w, \Delta h$ 控制宽高的对数空间缩放。解码后得到 M 个提议框, 坐标仍位于原始图像空间。

Step 4: 裁剪、排序与 NMS 筛选. 对 $\mathcal{P}_{\text{raw}}$ 执行空间裁剪, 将所有提议框的边界限制在图像尺寸 $[0, W] \times [0, H]$ 内, 排除无效的越界框。随后依据前景得分 Score_m 降序排序, 取前 N_{pre} 个高置信度提议框 (训练时通常取 12000, 测试时取 6000)。最后执行非极大值抑制 (NMS, IoU 阈值通常为 0.7), 去除与更高得分框高度重叠的冗余框:

$$\mathcal{P}_{\text{final}} = \text{NMS}(\text{TopK}(\mathcal{P}_{\text{raw}}, \text{Score})),$$

最终筛选得到 N 个高质量提议框:

$$\mathcal{P}_{\text{final}} = \{(x_1, y_1, x_2, y_2)_n\}_{n=1}^N,$$

其中训练阶段 $N = 2000$, 测试阶段 $N = 300$, 坐标仍保留在原始图像空间。

Step 5: 坐标映射至特征空间. 由于 $\mathcal{P}_{\text{final}}$ 位于原始图像空间, 而特征图 $\mathcal{F}_{\text{shared}}$ 的空间分辨率仅为图像的 $1/S$, 因此需将每个提议框的坐标除以步长 S , 映射到特征图坐标系:

$$(x_{\text{feat}}, y_{\text{feat}}, w_{\text{feat}}, h_{\text{feat}}) = \left(\frac{x_1}{S}, \frac{y_1}{S}, \frac{x_2 - x_1}{S}, \frac{y_2 - y_1}{S} \right). \quad (3.33)$$

此处映射结果为浮点数，保留小数精度以减小量化误差。该映射操作确保了后续 RoI 特征提取能够正确定位到共享特征图上对应的空间区域。

Step 6: RoI Align / Pooling 提取固定尺寸特征。对共享特征图 $\mathcal{F}_{\text{shared}}$ 执行 RoI 特征提取操作，将每个尺寸不一的提议框统一转换为固定大小（如 7×7 ）的特征张量：

$$\mathcal{F}_{\text{roi}}[n] = \text{RoIAlign}(\mathcal{F}_{\text{shared}}, (x_{\text{feat}}, y_{\text{feat}}, w_{\text{feat}}, h_{\text{feat}})_n), \quad (3.34)$$

最终输出特征张量 $\mathcal{F}_{\text{roi}} \in \mathbb{R}^{N \times 7 \times 7 \times C}$ ，供后续检测头全连接层进行分类与边界框回归。

注 上述链条中，Step 0 与 Step 5、Step 6 共同构成了共享卷积特征复用的物理基础：

- 全图仅经过一次卷积计算（Step 0），所有提议框的特征提取均基于同一张特征图，避免了 R-CNN 中 2000 次独立前向传播的巨大冗余；
- Step 6 若采用原版 RoI Pooling，会经历两次量化误差（坐标映射取整与网格划分取整），导致空间定位偏差；现代实现采用 RoI Align，通过双线性插值在浮点坐标上采样，保留空间精度，显著提升小目标检测性能；
- Step 1 至 Step 4 构成了 RPN 的核心逻辑：RPN 本质是一个全卷积网络，通过锚框机制与回归解码，实现了从滑动窗口先验到数据驱动提议的端到端可学习变换，彻底替代了 Fast R-CNN 中不可微的 Selective Search 算法。

命题 3.23 (Faster R-CNN 双阶段检测流程中的分类与回归复用机制)

Faster R-CNN 作为两阶段目标检测框架，其核心结构特征在于：分类与回归任务在级联的两个阶段中各出现一次，但两阶段的任务粒度、输入对象、网络分支与优化目标存在本质区别。具体而言：

第一阶段：RPN 粗筛与粗调

该阶段以共享特征图 $\mathcal{F}_{\text{shared}} \in \mathbb{R}^{H_f \times W_f \times C}$ 上的全部锚框 $\mathcal{A} = \{a_m\}_{m=1}^M$ （通常 $M \approx 60\,000$ ）为处理对象，RPN 头部网络执行如下双重预测：

- 二分类：对每个锚框输出前景/背景的二分类得分 $\text{Score}_m \in [0, 1]$ ，用于判断该锚框是否包含任意类别物体；
- 边界框粗回归：对每个锚框输出编码偏移量 $(\Delta x, \Delta y, \Delta w, \Delta h)_m \in \mathbb{R}^4$ ，并通过解码将锚框变换为提议框。

最终经前景得分排序与非极大值抑制，筛选出 N 个高质量提议框 $\mathcal{P}_{\text{final}} = \{p_n\}_{n=1}^N$ （训练时 $N = 2000$ ，测试时 $N = 300$ ）。本阶段的分类损失为二分类交叉熵，回归损失为 Smooth L1，仅对正样本锚框计算。

第二阶段：检测头细分类与精调

该阶段以第一阶段输出的 N 个提议框 $\mathcal{P}_{\text{final}}$ 为处理对象，首先通过 RoI Align（或 RoI Pooling）在共享特征图 $\mathcal{F}_{\text{shared}}$ 上为每个提议框提取固定尺寸特征张量（如 $7 \times 7 \times C$ ），展平后送入检测头全连接网络，执行如下双重预测：

- 多分类：输出 $K + 1$ 维向量（ K 个物体类别 + 1 个背景类），经 Softmax 得到概率分布 $p = (p_0, p_1, \dots, p_K)$ ，用于判定提议框所属的具体类别或背景；
- 边界框精回归：输出 4 维偏移量 $t^u = (t_x, t_y, t_w, t_h)$ ，用于对提议框进行最终位置精调，提升定位精度。

本阶段的分类损失为多分类交叉熵，回归损失为 Smooth L1，且回归损失仅对前景提议框（ $u \geq 1$ ）计算。

两阶段分类/回归的对比关系

对比维度	第一阶段 (RPN)	第二阶段 (检测头)
处理对象	全部锚框 ($M \approx 60\,000$)	RPN 筛选后的提议框 ($N \approx 300$)
分类粒度	二分类 (前景 / 背景)	多分类 (K 个物体类 + 背景)
回归粒度	粗调 (锚框 $\rightarrow$ 提议框)	精调 (提议框 $\rightarrow$ 最终检测框)
网络分支	RPN 头部 (卷积层)	检测头 (全连接层)
分类损失	二分类交叉熵	多分类交叉熵
回归损失	Smooth L1 (仅正样本锚框)	Smooth L1 (仅前景提议框)

参数共享与梯度回传机制

两阶段共享同一组 Backbone 卷积特征 ($\mathcal{F}_{\text{shared}}$), 但各自拥有独立的头部参数:

- RPN 头部参数 (3×3 卷积 + 两个 1×1 卷积) 仅由第一阶段损失 $\mathcal{L}_{\text{RPN}}$ 的梯度更新;
- 检测头参数 (全连接层) 仅由第二阶段损失 $\mathcal{L}_{\text{det}}$ 的梯度更新;
- Backbone 参数同时接收两个阶段损失的梯度之和, 即:

$$\nabla \Theta_{\text{backbone}} = \nabla \Theta_{\mathcal{L}_{\text{RPN}}} + \nabla \Theta_{\mathcal{L}_{\text{det}}},$$

使得共享特征同时受到“是否有物体”与“具体是什么物体”的双重监督, 学习到兼具召回率与判别力的通用视觉表征。

注 该双阶段设计体现了“由粗到精”的检测策略: 第一阶段以高召回率为目标, 在大量锚框中快速筛选候选; 第二阶段以高精度为目标, 在少量候选中精细判别。两阶段的分类与回归任务虽然结构对称, 但粒度与优化目标不同, 不可互相替代。RPN 的二分类器回答“这里有没有东西”, 检测头的多分类器回答“这东西到底是什么”——两者级联, 构成了两阶段检测范式的核心逻辑。



Faster R-CNN 将区域提议的生成时间从选择性搜索的数秒级缩短至 10 毫秒级, 在 VOC 2007 上达到 73.2% 的 mAP, 推理速度提升至 5 帧/秒 (FPS), 首次使两阶段检测器接近实时性能。RPN 与检测网络共享卷积特征, 其损失函数为:

$$\mathcal{L}_{\text{RPN}} = \mathcal{L}_{\text{cls}}(\{p_i\}, \{p_i^*\}) + \lambda \cdot [p_i^* = 1] \cdot \mathcal{L}_{\text{reg}}(\{t_i\}, \{t_i^*\})$$

其中 p_i 为锚框 i 的目标性预测, p_i^* 为真实标签 (正/负样本), t_i 与 t_i^* 分别为预测的偏移量与真实偏移量。

Faster R-CNN 奠定了两阶段检测器的基本框架, 后续的 Mask R-CNN、Cascade R-CNN 等工作均在其基础上扩展。

3.1.4.5 YOLO

YOLO (You Only Look Once) [redmon2016you] 于 2016 年提出, 开创了单阶段检测 (One-Stage Detection) 范式, 将目标检测重新定义为单一的回归问题: 输入一张图像, 直接输出所有目标的位置和类别。

YOLO 将输入图像划分为 $S \times S$ 的网格 (如 7×7)。每个网格单元负责预测其中心落入该网格内的目标。每个网格单元预测 B 个边界框 (每个框包含 x, y, w, h 坐标和置信度得分) 以及 C 个类别概率, 最终输出的张量维度为 $S \times S \times (B \times 5 + C)$ 。

YOLO 的损失函数综合考虑了边界框定位误差、置信度误差和分类误差:

$$\mathcal{L} = \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{I}_{ij}^{\text{obj}} [(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2] + \dots + \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{I}_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 + \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{I}_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2 + \sum_{i=0}^{S^2} \mathbb{I}_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2$$

其中 λ_{coord} 用于平衡定位误差与分类误差的权重, $\mathbb{I}_{ij}^{\text{obj}}$ 指示第 i 个网格的第 j 个边界框是否负责预测该目标。

YOLO 的核心优势在于极快的推理速度——在 Titan X GPU 上可达 45 FPS, 快速版本可达 155 FPS, 真正实

现了实时目标检测。然而，YOLO 的定位精度相对较低，尤其对小物体和密集场景检测效果不佳，主要受限于网格划分的稀疏性。

3.1.4.6 SSD

SSD (Single Shot MultiBox Detector) [liu2016ssd] 于 2016 年提出，融合了 YOLO 的单阶段推理速度与 Faster R-CNN 的锚框机制，在保证实时性的同时显著提升了检测精度。

SSD 的核心设计可概括为两点：

- 1. **多尺度特征图检测 (Multi-scale Feature Maps):** SSD 在卷积网络的不同层级（即不同分辨率的特征图）上分别进行目标检测。浅层特征图（高分辨率）负责检测小目标，深层特征图（低分辨率）负责检测大目标。这一设计有效缓解了 YOLO 对小目标检测能力不足的问题。
- 2. **默认框 (Default Boxes) 机制:** SSD 借鉴了 Faster R-CNN 的锚框思想，在每个特征图的每个位置预定义多个不同尺度和长宽比的默认框。与 YOLO 不同的是，SSD 在不同的特征图层级上使用不同尺寸的默认框，以适配不同尺度的目标，并同时预测边界框的偏移量 (cx, cy, w, h) 和类别置信度。

SSD 的损失函数由定位损失 (Smooth L1) 与分类损失 (Softmax 交叉熵) 加权求和构成：

$$\mathcal{L} = \frac{1}{N} (\mathcal{L}_{\text{conf}} + \alpha \mathcal{L}_{\text{loc}})$$

其中 N 为匹配正样本的默认框数量， α 为权衡系数（通常设为 1）。

SSD 在 VOC 2007 上达到 74.3% 的 mAP，在 Titan X 上推理速度达到 46 FPS，在精度与速度之间取得了良好的平衡，成为当时最受工业界欢迎的实时检测器之一。其后续变体 DSSD、RefineDet、M2Det 等进一步提升了检测性能。

表 3.3: 经典目标检测方法性能对比 (VOC 2007 或 COCO 基准)

方法	范式	mAP (VOC 2007)	速度 (FPS)
R-CNN	两阶段	58.5%	0.02
Fast R-CNN	两阶段	70.0%	0.5
Faster R-CNN	两阶段	73.2%	5
YOLO	单阶段	63.4%	45
SSD	单阶段	74.3%	46

注：具体数值可能因论文版本和实验配置略有差异，此处为典型参考值。

3.1.5 应用于图像生成的神经网络

3.1.5.1 生成式对抗网络 GAN

生成式对抗网络 (Generative Adversarial Network, GAN) 由 Ian Goodfellow 于 2014 年首次提出，其核心思想是通过**博弈对抗**实现隐式概率分布的拟合，而不需要显式建模似然函数。GAN 的提出开辟了一条有别于传统概率生成模型的崭新路径，成为图像生成领域最具影响力的框架之一。

3.1.5.1.1 博弈框架与目标函数 GAN 由两个神经网络构成——生成器 G 与判别器 D ，二者在训练过程中进行零和博弈：

- **生成器 G :** 接受随机噪声 $z \sim p_z(z)$ （通常为标准正态分布），输出伪样本 $G(z)$ ，目标是让判别器无法区分真伪。
- **判别器 D :** 接受真实样本 $x \sim p_{\text{data}}$ 或伪样本 $G(z)$ ，输出标量 $D(\cdot) \in [0, 1]$ ，表示输入为真实样本的概率。目标是最小化判别误差。

命题 3.24 (GAN 生成器的标准结构：转置卷积堆叠上采样)

设随机噪声向量为 $z \in \mathbb{R}^d$ (通常 $d = 100$ 或 512)，从标准正态分布 $\mathcal{N}(0, I)$ 中采样。GAN 生成器 $G: \mathbb{R}^d \rightarrow \mathbb{R}^{H \times W \times 3}$ 通过 L 层转置卷积堆叠实现从低维隐空间到高维图像空间的映射。其标准架构 (以 DCGAN 为代表) 由以下三个模块串联构成：

模块一：全连接投影 (Dense Projection)

生成器的第一层为全连接层 (Linear Layer)，将 d 维噪声向量映射为一个初始特征向量，再重塑为具有空间维度的张量：

$$z \in \mathbb{R}^d \xrightarrow{\text{Linear}(d, C_0 \cdot H_0 \cdot W_0)} \text{vec} \in \mathbb{R}^{C_0 H_0 W_0} \xrightarrow{\text{Reshape}} \mathcal{F}_0 \in \mathbb{R}^{H_0 \times W_0 \times C_0},$$

其中 H_0 与 W_0 为初始特征图的空间尺寸 (通常取 4×4 或 8×8)， C_0 为初始通道数 (通常 512 或 1024)。该层后通常接 Batch Normalization 与 ReLU 激活函数。

模块二：转置卷积堆叠 (Transposed Convolution Stack)

从 $\mathcal{F}_0$ 出发，依次经过 L 层转置卷积模块，每层包含：

1. 转置卷积层 (Transposed Convolution)，参数为 ($\text{kernel_size} = k, \text{stride} = s, \text{padding} = p$)；
2. 批归一化层 (Batch Normalization)；
3. ReLU 激活函数 (输出层除外)。

第 ℓ 层 ($1 \leq \ell \leq L$) 的输入尺寸为 $H_{\ell-1} \times W_{\ell-1} \times C_{\ell-1}$ ，输出尺寸为 $H_\ell \times W_\ell \times C_\ell$ ，其中：

$$H_\ell = (H_{\ell-1} - 1) \times s_\ell - 2 \times p_\ell + k_\ell + \text{output_padding}_\ell,$$

$$W_\ell = (W_{\ell-1} - 1) \times s_\ell - 2 \times p_\ell + k_\ell + \text{output_padding}_\ell.$$

标准配置 (DCGAN, 目标 64×64)：

令 $d = 100$ ， $H_0 = W_0 = 4$ ， $C_0 = 1024$ ， $L = 4$ ，各层转置卷积参数如下表所示：

代入公式验证第 1 层： $O = (4 - 1) \times 2 - 2 \times 1 + 4 = 6 - 2 + 4 = 8$ ，输出尺寸精确为 8×8 。同理后续各层均实现 2 倍上采样，最终输出 $64 \times 64 \times 3$ 。

模块三：输出层映射 (Output Projection)

最后一层转置卷积输出尺寸已等于目标图像尺寸，通道数为 3 (RGB)。该层后接 Tanh 激活函数，将像素值逐元素压缩至 $(-1, 1)$ 区间：

$$\mathcal{F}_L \in \mathbb{R}^{H \times W \times 3} \xrightarrow{\text{Tanh}} \mathcal{I}_{\text{gen}} \in (-1, 1)^{H \times W \times 3}.$$

设计原则与工程约束：

1. 核大小与步长的整除约束：为保证重叠模式均匀、避免棋盘格效应，需满足 $k_\ell \bmod s_\ell = 0$ 。DCGAN 采用 $k = 4, s = 2$ ($4 \bmod 2 = 0$) 即是此约束的典型实现。
2. 输出层禁用 BN：Batch Normalization 在输出层会破坏单个样本的像素统计分布，导致色彩漂移 (色偏)，因此输出层仅使用 Tanh，不加 BN。
3. 初始特征图尺寸： $H_0 = W_0 = 4$ 是经验最优值，过小 (1×1) 无法承载足够的空间先验，过大 (8×8) 则增加计算负担且不利深部语义的汇聚。
4. 通道数逐层递减：从 $1024 \rightarrow 512 \rightarrow 256 \rightarrow 128 \rightarrow 3$ ，遵循“空间维度扩张、通道维度压缩”的设计范式，与编码器的“空间压缩、通道扩张”形成对称结构。

输出尺寸递推总览 (以 64×64 目标为例)：

$$4 \times 4 \xrightarrow{\text{ConvT}(4,2,1)} 8 \times 8 \xrightarrow{\text{ConvT}(4,2,1)} 16 \times 16 \xrightarrow{\text{ConvT}(4,2,1)} 32 \times 32 \xrightarrow{\text{ConvT}(4,2,1)} 64 \times 64.$$

目标函数为极小极大博弈 (Minimax Game)：

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))].$$

命题 3.25 (GAN 的极小极大博弈目标函数)

生成式对抗网络 (GAN) 的训练被形式化为一个二人零和博弈 (Two-Player Zero-Sum Game), 其中生成器 G 与判别器 D 互为对手, 共同优化以下目标函数:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]. \quad (3.35)$$

博弈参与者与策略空间:

- 判别器 D (最大化玩家): $D: \mathcal{X} \rightarrow [0, 1]$, 是一个二分类器, 接受图像 x 并输出该图像为真实样本的概率。判别器的策略空间为所有可微函数 (由神经网络参数化)。其目标是最大化目标函数 $V(D, G)$, 即:
 1. 对真实样本 $x \sim p_{\text{data}}$, 使 $D(x) \rightarrow 1$ (正确识别真实);
 2. 对生成样本 $G(z) \sim p_g$, 使 $D(G(z)) \rightarrow 0$ (正确识别伪造)。
- 生成器 G (最小化玩家): $G: \mathcal{Z} \rightarrow \mathcal{X}$, 接受噪声向量 $z \sim p_z$ 并生成伪图像。生成器的策略空间为所有可微生成函数 (由神经网络参数化)。其目标是最小化目标函数 $V(D, G)$, 即使 $D(G(z)) \rightarrow 1$ (让判别器认为生成图像是真实的)。

博弈的时序结构:

GAN 的训练遵循交替优化 (Alternating Optimization) 范式, 而非同时博弈:

1. 判别器步骤: 固定生成器 G , 更新判别器 D 以最大化 $V(D, G)$:

$$D^* = \arg \max_D V(D, G).$$

此时判别器学习区分真实分布 p_{data} 与当前生成分布 p_g 。

2. 生成器步骤: 固定判别器 D , 更新生成器 G 以最小化 $V(D, G)$:

$$G^* = \arg \min_G V(D, G).$$

此时生成器学习调整其参数, 以降低判别器的识别准确率。

训练生成器时, 梯度确实流过了判别器, 但“流过”不等于“更新”——优化器只更新自己管辖的参数。判别器虽然收到了梯度, 但由于它不在生成器优化器的管理范围内, 其权重纹丝不动。

上述两步交替执行, 直至达到纳什均衡 (Nash Equilibrium)。

纳什均衡的解析解:

在理想情况下, 当生成器完美拟合真实数据分布时, 即 $p_g = p_{\text{data}}$, 博弈达到唯一的纳什均衡点。此时:

1. 最优判别器: 对于任意固定的生成器 G , 最优判别器为:

$$D_G^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}. \quad (3.36)$$

推导: 对 $V(D, G)$ 关于 $D(x)$ 逐点求导并令其为零:

$$\begin{aligned} \frac{\partial}{\partial D(x)} [p_{\text{data}}(x) \log D(x) + p_g(x) \log(1 - D(x))] &= 0, \\ \frac{p_{\text{data}}(x)}{D(x)} - \frac{p_g(x)}{1 - D(x)} &= 0 \implies D_G^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}. \end{aligned}$$

2. 均衡点处的判别器输出: 当 $p_g = p_{\text{data}}$ 时, 代入式 (3.36):

$$D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_{\text{data}}(x)} = \frac{1}{2}, \quad \forall x.$$

即判别器无法区分真实与伪造, 输出恒为 $1/2$ (随机猜测)。

3. 均衡点处的目标函数值: 将 $D^* = 1/2$ 代入式 (3.35):

$$V(D^*, G) = \mathbb{E}_{x \sim p_{\text{data}}} \log \frac{1}{2} + \mathbb{E}_{x \sim p_g} \log \left(1 - \frac{1}{2}\right) = -2 \log 2.$$

这是目标函数在纳什均衡处的全局最小值 (对于生成器而言) 与全局最大值 (对于判别器而言) 的

统一交汇点。

目标函数的双重角色：

式 (3.35) 中的两项具有明确的语义分工：

生成器虽然只能直接影响第二项，但通过优化 G 迫使 p_g 趋近于 p_{data} ，间接影响了第一项中 $D(x)$ 的取值（因为在均衡点处 $D(x) = 1/2$ ）。

训练动态与工程挑战：

尽管上述理论框架优雅且完备，实际训练中存在两个核心挑战：

1. **梯度消失 (Vanishing Gradients)**：当判别器过强 ($D(G(z)) \rightarrow 0$) 时， $\log(1 - D(G(z)))$ 的梯度趋近于零，生成器难以获得有效的更新信号。工程对策：采用非饱和损失 (Non-saturating Loss)，即令生成器最小化 $-\log D(G(z))$ 而非 $\log(1 - D(G(z)))$ 。
2. **模式崩溃 (Mode Collapse)**：生成器可能坍缩到少数几个模式，仅生成高度相似的样本，丧失多样性。工程对策：采用 WGAN、Unrolled GAN、Minibatch Discrimination 等技术，或引入多样性促进正则项。

理论意义：

极小极大博弈框架为 GAN 提供了坚实的理论基础：

- 将无监督学习问题转化为有监督的二分类问题，使得生成模型训练可利用成熟的分类工具。
- 通过博弈均衡保证了生成分布 p_g 在理想情况下收敛于真实分布 p_{data} 。
- 该框架启发了后续大量变体（如 f-GAN、WGAN、LSGAN），每种变体本质上是改变了度量两个分布之间距离的散度 (Divergence) 函数。



在纳什均衡条件下，生成器的输出分布 p_g 完美拟合真实数据分布 p_{data} ，判别器对所有输入的输出均为 $1/2$ ，即：

$$p_g = p_{\text{data}} \quad \text{且} \quad D(x) = \frac{1}{2}, \forall x.$$

3.1.5.1.2 WGAN 与 Wasserstein 距离 WGAN (Wasserstein GAN, Arjovsky et al., 2017) 是 GAN 训练稳定性的一次重大飞跃。其核心洞察是将 JS 散度替换为 **Wasserstein 距离 (Earth Mover's Distance)**，该距离即使在两个分布没有重叠时仍能提供平滑的梯度。

WGAN 的目标函数为：

$$\min_G \max_{w \in \mathcal{W}} \mathbb{E}_{x \sim p_{\text{data}}} [f_w(x)] - \mathbb{E}_{z \sim p_z(z)} [f_w(G(z))],$$

其中 f_w 是受 Lipschitz 约束的判别器（此时称为“评价器”，Critic）。WGAN 通过权值裁剪 (Weight Clipping) 强制 Lipschitz 约束，后续 WGAN-GP (Gulrajani et al., 2017) 改用梯度惩罚 (Gradient Penalty) 进一步优化。

WGAN 的主要优势：

- 训练稳定性显著提升，几乎不再出现模式崩溃。
- 损失函数数值与生成质量具有相关性（损失越低，图像质量越高）。
- 允许使用更激进的网络架构，无需精心调参。

3.1.5.1.3 DCGAN：将卷积引入 GAN DCGAN (Deep Convolutional GAN, Radford et al., 2015) 是 GAN 历史上首个大规模成功应用的卷积架构。其提出的设计原则至今仍被广泛遵循：

1. **全卷积架构**：生成器与判别器中不使用全连接层（除输入/输出投影外），全部使用步长卷积/转置卷积。
2. **批归一化 (BN)**：除生成器输出层与判别器输入层外，所有层均使用 BN，有效防止内部协变量偏移。
3. **激活函数规范**：生成器除输出层使用 Tanh 外，其余层使用 ReLU；判别器所有层使用 LeakyReLU（斜率 0.2）。
4. **步长卷积代替池化**：使用 $s = 2$ 的卷积/转置卷积实现下采样/上采样，让网络自行学习空间压缩策略。

DCGAN 在 LSUN 卧室数据集上生成了 64×64 的清晰图像，证明了 GAN 具备学习复杂图像分布的能力，为后续 StyleGAN、BigGAN 等高性能生成器奠定了基础。

3.1.5.1.4 条件 GAN (Conditional GAN) 条件 GAN (cGAN) 由 Mirza & Osindero 在 2014 年提出，其核心思想是在生成器和判别器中同时引入额外条件信息 y （如类别标签、文本描述或属性向量），从而实现对生成过程的主动控制。

目标函数扩展为：

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D(x|y)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z|y)|y))]. \tag{3.37}$$

此处条件 y 与噪声 z 拼接

3.1.5.1.5 GAN 的评价指标 评估 GAN 生成质量是学术界的一大难题。目前主流评价指标包括：

- **Inception Score (IS):** 同时度量生成图像的质量（Inception 网络分类置信度高）与多样性（类别分布均匀），但 IS 对样本内多样性不敏感。
- **Fréchet Inception Distance (FID):** 在 Inception 网络的特征空间中，计算真实样本分布与生成样本分布之间的 Fréchet 距离（即均值的马氏距离 + 协方差矩阵的差异）。FID 更低表示生成图像更逼真且更多样，是目前最常用的评价指标。
- **Precision & Recall:** 由 Kynkäänniemi 等人在 2019 年提出，精确率衡量生成图像是否保真（Quality），召回率衡量生成图像是否覆盖了真实分布的全部模式（Diversity），将两者解耦。

表 3.4: 主流 GAN 架构对比

模型	核心创新	上采样方案	关键局限
原始 GAN	极小极大博弈	全连接 + 转置卷积	训练极不稳定，易模式崩溃
DCGAN	卷积架构 + BN	转置卷积 ($k = 4, s = 2$)	仍存在模式崩溃风险
WGAN-GP	Wasserstein 距离 + 梯度惩罚	依赖于具体架构	训练较慢，需调参
StyleGAN	解耦隐空间 + 自适应实例归一化	双线性上采样 + 卷积	计算量大，参数极多

3.1.5.1.6 当前发展趋势 GAN 的研究正朝以下方向深化：

- **条件生成 (Conditional GAN):** 通过标签、文本或属性向量的引入，实现对生成内容的外部控制（典型工作：Pix2Pix, StyleGAN 的调整）。
- **大尺度高分辨率生成:** BigGAN 在 ImageNet 上生成了 512×512 的高清图像，证明 GAN 具备处理超大规模分布的能力。
- **与扩散模型的融合:** Diffusion GAN 等工作尝试将 GAN 的快速采样优势与扩散模型的多步去噪机制结合，兼具速度与质量。

小结

上采样是生成模型将隐变量投射到像素空间的桥梁，其实现方式经历了从“插值”到“转置卷积”再到“插值 + 卷积组合”的螺旋演进，核心目标是平衡参数效率与伪影控制。而 GAN 则通过对抗博弈实现了高质量的隐式生成，其训练稳定性问题和架构设计原则（如 DCGAN 规范）至今仍是生成模型研究的基石。

后输入生成器，同时与图像 x 拼接后输入判别器。cGAN 的提出催生了大量条件生成任务，如图像翻译 (Pix2Pix)、文本到图像生成等。

3.1.5.2 扩散模型 (Diffusion Models)

扩散模型是近年来图像生成领域最具突破性的进展，其生成质量已超越 GAN（如 DALL-E 2、Stable Diffusion 等），并展现出更强的可控性与分布覆盖能力。

扩散模型的核心框架包含前向扩散过程（Forward Process）与反向去噪过程（Reverse Process）两个马尔可夫链。

3.1.5.2.1 前向过程：从数据到噪声 给定真实图像 $x_0 \sim q(x)$ ，逐步添加高斯噪声，直至图像变为纯噪声 $x_T \sim \mathcal{N}(0, I)$ ：

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t I), \quad (3.38)$$

其中 $\{\beta_t\}_{t=1}^T$ 为方差调度（Variance Schedule）。利用重参数化技巧，可直接从 x_0 采样任意时刻 t 的噪声图像：

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I), \quad \alpha_t = 1 - \beta_t, \quad \bar{\alpha}_t = \prod_{i=1}^t \alpha_i. \quad (3.39)$$

3.1.5.2.2 反向过程：从噪声生成数据 反向过程训练一个神经网络 ϵ_θ （通常为 U-Net 结构）来预测添加的噪声 ϵ ，从而逐步去噪还原图像：

$$\min_{\theta} \mathbb{E}_{t, x_0, \epsilon} [\|\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t)\|^2]. \quad (3.40)$$

训练完成后，从 $x_T \sim \mathcal{N}(0, I)$ 出发，迭代 T 步逐步去噪，最终生成高质量图像。

3.1.5.2.3 Palette Palette (Saharia et al., 2022) 是 Google Research 提出的一类通用扩散模型框架，旨在统一图像生成、修复、上色和去模糊等多种底层视觉任务。其核心贡献在于：

- 采用条件扩散范式，将输入图像（如低分辨率图、灰度图）作为条件 c ，引导反向去噪过程。
- 使用 Transformer-based U-Net 架构，支持 256×256 分辨率下的高保真生成。
- 通过分析发现，扩散模型对条件信号具有较强的鲁棒性，多种任务只需调整条件输入即可共享同一套权重。

3.1.5.2.4 DALL-E 2 DALL-E 2 (Ramesh et al., 2022) 是 OpenAI 推出的文本到图像生成系统，其架构摒弃了直接扩散于像素空间的策略，转而采用两阶段级联方案：

1. **CLIP 视觉编码**：使用 CLIP 模型将文本编码为语义嵌入，同时将图像编码为视觉特征。
2. **先验模块 (Prior)**：一个自回归或扩散模型，以文本嵌入为条件，生成 CLIP 图像嵌入。
3. **解码器 (Decoder)**：一个扩散模型，以 CLIP 图像嵌入为条件，在图像空间生成 1024×1024 高分辨率图像。

该“文本 → 语义嵌入 → 图像”的解耦设计使得 DALL-E 2 能够生成极其丰富、风格多样的图像，且支持图像编辑（如替换物体、调整风格）等高级功能。

3.1.5.2.5 Stable Diffusion Stable Diffusion (Rombach et al., 2022) 是目前开源社区影响力最广的扩散模型，由 Stability AI 发布。其核心创新为潜在扩散模型（Latent Diffusion Model, LDM）：

- 不直接在像素空间进行扩散，而是先用 VAE 编码器将图像压缩至潜在空间（Latent Space），在该低维空间执行扩散与去噪过程，再通过 VAE 解码器还原为像素图像。
- 此举将计算复杂度从 $O(n^2)$ 降至 $O((n/k)^2)$ （ k 为下采样倍数，通常 $k = 8$ ），实现了在单张消费级 GPU（如 NVIDIA 2080 Ti）上秒级生成 512×512 图像。
- 引入交叉注意力机制（Cross-Attention），将文本条件以注意力方式注入 U-Net 的每一层，实现精细化的文本语义控制。

3.1.5.2.6 ControlNet ControlNet (Zhang & Agrawala, 2023) 是扩散模型空间可控性的突破性工作。其核心思想是：在预训练的扩散模型（如 Stable Diffusion）之上，添加一个可训练的零卷积（Zero-Convolution）分支，以接收额外的空间条件输入（如边缘图、姿态骨骼、深度图或涂鸦），从而精确控制生成图像的空间结构。

定义 3.6 (ControlNet 的架构设计)

ControlNet 复制了原扩散模型 U-Net 中编码器的可训练副本，并将其与原始特征通过零卷积层逐层融合。零卷积层的初始权重为 0，确保在训练初期网络完全保留预训练能力；随着训练进行，条件信号逐步被激活，实现对生成过程的精细化控制。



ControlNet 的出现使得扩散模型从“随机生成”走向了“可控编辑”，被广泛应用于姿态迁移、室内设计、线稿上色等专业创作场景。

表 3.5: 主流扩散模型对比

模型	空间维度	条件模态	核心创新
Palette	像素空间	图像（低质量输入）	统一底层视觉任务
DALL-E 2	像素空间	文本	CLIP 解耦级联
Stable Diffusion	潜在空间	文本	潜在扩散 + 交叉注意力
ControlNet	潜在空间	空间控制图	零卷积分支 + 条件注入

小结

GAN 与扩散模型分别代表了“对抗式学习”与“似然式学习”两种范式。前者擅长快速生成与风格多样性，但训练不稳定且易模式崩溃；后者训练稳定、覆盖全分布且可控性强，但推理速度较慢。当前趋势显示，扩散模型正在接管高质量图像生成的主阵地，而 GAN 则在实时应用、小样本生成等场景中继续保持竞争力。二者融合（如扩散 GAN）也成为新兴研究方向。

3.2 自然语言处理发展长廊

正如卷积神经网络（CNN）之于计算机视觉，自然语言处理（NLP）的核心基石则是循环神经网络（Recurrent Neural Network, RNN）。与图像数据具有天然的空间结构不同，文本数据本质上是**时间序列**——每个词的出现都依赖于其之前的上下文。RNN 的设计正是为了捕捉这种时序依赖关系，它为后续 BERT、GPT 等大规模预训练语言模型铺平了道路。

3.2.1 循环神经网络 RNN

3.2.1.1 网络结构

RNN 的核心设计思想是**参数共享**（Parameter Sharing）——在不同时间步使用相同的权重矩阵处理输入，从而将序列长度这一维度“折叠”进循环计算中。

定义 3.7 (RNN 的循环结构)

给定输入序列 $\mathbf{x} = (x_1, x_2, \dots, x_T)$ ，RNN 通过以下递推关系计算隐状态序列：

$$\mathbf{h}_t = \sigma(\mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h), \quad (3.41)$$

$$\mathbf{y}_t = \sigma(\mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y), \quad (3.42)$$

其中 $\mathbf{h}_t \in \mathbb{R}^{d_h}$ 为 t 时刻的隐状态， $\mathbf{W}_{xh} \in \mathbb{R}^{d_h \times d_x}$ 、 $\mathbf{W}_{hh} \in \mathbb{R}^{d_h \times d_h}$ 、 $\mathbf{W}_{hy} \in \mathbb{R}^{d_y \times d_h}$ 为权重矩阵， σ 为非线性激活函数（通常为 $\tanh$ 或 ReLU ）。



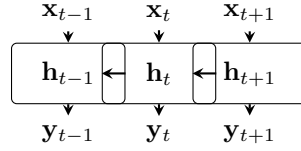


图 3.3: RNN 的循环结构（展开图）。每个时间步共享相同的权重矩阵 $\mathbf{W}_{xh}$ 、 $\mathbf{W}_{hh}$ 、 $\mathbf{W}_{hy}$ 。

命题 3.26 (Elman 网络与 Jordan 网络：两种经典的 RNN 拓扑结构)

在循环神经网络（RNN）的发展历程中，Elman 网络（1990）和 Jordan 网络（1986）是最早被提出的两种具有循环连接的神经网络架构。它们奠定了现代 RNN 的基本范式，并深刻影响了后续 LSTM、GRU 等复杂结构的设计。

1. Elman 网络（Simple Recurrent Network）

Elman 网络由 Jeffrey L. Elman 于 1990 年提出，其结构是最朴素、最直接的循环神经网络形式。

3.2.1.1.1 网络结构 Elman 网络包含三个主要的层：

- 输入层：接收当前时间步的输入 $\mathbf{x}_t \in \mathbb{R}^{d_x}$ 。
- 隐层（上下文层）：计算当前时间步的隐状态 $\mathbf{h}_t \in \mathbb{R}^{d_h}$ ，其输入来自当前输入 $\mathbf{x}_t$ 和上一时刻的隐状态 $\mathbf{h}_{t-1}$ 。
- 输出层：从隐状态生成输出 $\mathbf{y}_t \in \mathbb{R}^{d_y}$ 。

$$\begin{aligned} \mathbf{h}_t &= \sigma_h(\mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h), \\ \mathbf{y}_t &= \sigma_y(\mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y), \end{aligned} \quad (3.43)$$

其中 $\mathbf{W}_{xh} \in \mathbb{R}^{d_h \times d_x}$ 、 $\mathbf{W}_{hh} \in \mathbb{R}^{d_h \times d_h}$ 、 $\mathbf{W}_{hy} \in \mathbb{R}^{d_y \times d_h}$ 为权重矩阵， $\mathbf{b}_h$ 、 $\mathbf{b}_y$ 为偏置项， σ_h 通常为 $\tanh$ 或 ReLU ， σ_y 通常为 softmax 或 linear 。

3.2.1.1.2 Elman 网络的核心特征

1. 隐层到隐层的循环：循环连接发生在隐层内部（ $\mathbf{h}_{t-1} \rightarrow \mathbf{h}_t$ ）。
2. 上下文记忆：隐状态 $\mathbf{h}_t$ 充当了序列的“记忆单元”，编码了从 $t=1$ 到 t 的所有历史信息。
3. 训练算法：采用时间反向传播（BPTT），但在 Elman 的原始工作中，由于梯度消失问题尚未被充分认识，他使用的是更简单的前向传播 + 固定循环权重的方案。

2. Jordan 网络（Jordan Recurrent Network）

Jordan 网络由 Michael I. Jordan 于 1986 年提出（早于 Elman 网络），其结构理念与 Elman 网络有所不同。

3.2.1.1.3 网络结构 Jordan 网络的主要特点是，循环连接来自于输出层而非隐层：

- 输入层：接收当前输入 $\mathbf{x}_t \in \mathbb{R}^{d_x}$ 。
- 隐层：计算 $\mathbf{h}_t$ ，其输入来自当前输入 $\mathbf{x}_t$ 和上一时刻的输出 $\mathbf{y}_{t-1}$ 。
- 输出层：生成当前输出 $\mathbf{y}_t$ 。

$$\begin{aligned} \mathbf{h}_t &= \sigma_h(\mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{yh}\mathbf{y}_{t-1} + \mathbf{b}_h), \\ \mathbf{y}_t &= \sigma_y(\mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y), \end{aligned} \quad (3.44)$$

其中 $\mathbf{W}_{yh} \in \mathbb{R}^{d_h \times d_y}$ 是将上一时刻输出映射到当前隐层的权重矩阵， $\mathbf{y}_{t-1}$ 为上一时刻的输出值。

3.2.1.1.4 Jordan 网络的核心特征

1. 输出到隐层的循环：循环连接来自输出层 ($y_{t-1} \rightarrow h_t$)。
2. 输出作为记忆： y_{t-1} 充当了网络的记忆信号，因此网络对输出值本身具有更强的记忆依赖。
3. 应用于控制任务：Jordan 最初的设计动机是用于机器人控制和动态系统建模，输出记忆更符合控制信号平滑变化的需求。

3. Elman 网络与 Jordan 网络的对比

4. 两种网络的变体与扩展

1. **Elman+Jordan 混合网络**：同时包含隐层到隐层和输出到隐层的循环连接，表达更为丰富。
2. **Teacher Forcing**：在训练 Jordan 网络时，将真实目标值 y_{t-1}^{true} 而非模型预测值 y_{t-1}^{pred} 作为循环输入，可加速收敛。该技术后来被广泛应用于所有 RNN 训练中。
3. 上下文层的物理含义：Elman 网络中的隐状态 h_{t-1} 常被解释为网络的“短期记忆”或“工作记忆”，而 Jordan 网络中的输出延迟 y_{t-1} 则是“时间平滑约束”的体现。

5. 历史意义与局限

定义 3.8 (两种网络的历史定位)

Elman 网络和 Jordan 网络是 RNN 发展史上最早的具有可训练循环连接的神经网络架构。它们证明了通过循环连接可以使神经网络具备序列建模能力，但其均受限于：

- 循环连接长度有限（通常不超过 10-20 步）。
- 梯度消失/爆炸问题（Elman 网络尤为严重）。
- 无法建模长期依赖（这是 LSTM 后来解决的核心问题）。

尽管如此，Elman 网络仍被视为现代 RNN 的标准原型，其基本公式 $h_t = \sigma(W_{xh}x_t + W_{hh}h_{t-1})$ 至今仍是所有 RNN 变体的基础表达式。



命题 3.27 (双向循环神经网络 (Bidirectional RNN))

双向循环神经网络 (Bidirectional RNN, BiRNN) 由 Schuster & Paliwal 于 1997 年提出，其核心思想是：在标准 RNN 只能利用过去上下文的基础上，引入一个反向传播的 RNN 来利用未来上下文，从而实现对整个输入序列的双向建模。

1. 结构设计：两个独立的方向

BiRNN 由两个独立的 RNN 堆叠而成，分别沿时间轴的两个方向处理序列：

- **前向 RNN (Forward RNN)**：按时间顺序 $t = 1, 2, \dots, T$ 读取序列，产生前向隐状态序列 $\vec{h}_1, \vec{h}_2, \dots, \vec{h}_T$ 。
- **反向 RNN (Backward RNN)**：按时间逆序 $t = T, T-1, \dots, 1$ 读取序列，产生反向隐状态序列 $\overleftarrow{h}_1, \overleftarrow{h}_2, \dots, \overleftarrow{h}_T$ 。

两个 RNN 在每一时间步的输出被拼接 (Concatenate) 或相加 (Sum)，作为该时间步的最终表示。

2. 数学形式

设输入序列为 $\mathbf{x} = (\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T)$ ，其中 $\mathbf{x}_t \in \mathbb{R}^{d_x}$ 。

3.2.1.1.5 前向 RNN：从 $t = 1$ 到 $t = T$ 递推

$$\vec{h}_t = \sigma_h \left(\mathbf{W}_{xh}^f \mathbf{x}_t + \mathbf{W}_{hh}^f \vec{h}_{t-1} + \mathbf{b}_h^f \right), \quad (3.45)$$

其中 $\vec{h}_0 = \mathbf{0}$ 。

3.2.1.1.6 反向 RNN : 从 $t = T$ 到 $t = 1$ 递推

$$\overleftarrow{\mathbf{h}}_t = \sigma_h \left(\mathbf{W}_{xh}^b \mathbf{x}_t + \mathbf{W}_{hh}^b \overleftarrow{\mathbf{h}}_{t+1} + \mathbf{b}_h^b \right), \quad (3.46)$$

其中 $\overleftarrow{\mathbf{h}}_{T+1} = \mathbf{0}$ 。

3.2.1.1.7 输出层 : 将两个方向的隐状态拼接

$$\mathbf{h}_t = \left[\overrightarrow{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t \right] \in \mathbb{R}^{2d_h}, \quad (3.47)$$

$$\mathbf{y}_t = \sigma_y (\mathbf{W}_{hy} \mathbf{h}_t + \mathbf{b}_y). \quad (3.48)$$

定义 3.9 (拼接 vs 求和)

最常见的融合方式是拼接 (**Concatenation**)，将两个 d_h 维向量拼成 $2d_h$ 维向量。在某些场景下也可采用逐元素相加 (**Element-wise Sum**)，但拼接保留了两个方向的独立信息，表达能力更强。

**3. 双向 RNN 的三大优势**

1. 上下文完整性：每个时间步的输出同时感知了左侧（过去）和右侧（未来）的上下文信息。对于自然语言处理任务，这至关重要——例如，判断“苹果”一词的含义需要同时看到前后的词语（“吃苹果” vs “苹果公司”）。
2. 表征能力更强：由于拼接了两个独立 RNN 的隐状态，输出的特征维度翻倍 ($2d_h$)，能够编码更丰富的语义信息。
3. 任务性能提升：在命名实体识别 (NER)、词性标注 (POS Tagging)、情感分类等序列标注任务中，双向 RNN 显著优于单向 RNN。



RNN 的三种模式：

模式	输入长度	输出长度	典型应用
多对一 (Many-to-One)	T	1	情感分类、文本分类
一对一 (One-to-One)	1	1	标准前馈网络（非序列任务）
多对多 (Many-to-Many)	T	T	语言模型、机器翻译
多对多（不等长）	T	S	编码器-解码器架构

3.2.1.2 正向传播与反向传播**3.2.1.2.1 正向传播 (Forward Propagation)** RNN 的正向传播遵循时间顺序，从 $t = 1$ 到 $t = T$ 依次计算：

1. 初始化隐状态 $\mathbf{h}_0 = \mathbf{0}$ （或随机初始化）。
2. 对每个时间步 $t = 1, 2, \dots, T$ ：

$$\mathbf{a}_t = \mathbf{W}_{xh} \mathbf{x}_t + \mathbf{W}_{hh} \mathbf{h}_{t-1} + \mathbf{b}_h, \quad (3.49)$$

$$\mathbf{h}_t = \tanh(\mathbf{a}_t), \quad (3.50)$$

$$\mathbf{o}_t = \mathbf{W}_{hy} \mathbf{h}_t + \mathbf{b}_y, \quad (3.51)$$

$$\hat{\mathbf{y}}_t = \text{softmax}(\mathbf{o}_t) \quad (\text{用于分类任务}). \quad (3.52)$$

3.2.1.2.2 反向传播 (Backpropagation Through Time, BPTT) RNN 的反向传播需要在时间维度上展开, 称为时间反向传播 (BPTT)。损失函数定义为所有时间步损失之和:

$$\mathcal{L} = \sum_{t=1}^T \mathcal{L}_t(\mathbf{y}_t, \hat{\mathbf{y}}_t).$$

对于第 t 步的隐状态 $\mathbf{h}_t$, 其梯度不仅来自当前时间步的输出损失 $\partial \mathcal{L}_t / \partial \mathbf{h}_t$, 还来自后续所有时间步传递回来的梯度:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} = \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} + \frac{\partial \mathcal{L}_{t+1}}{\partial \mathbf{h}_{t+1}} \cdot \frac{\partial \mathbf{h}_{t+1}}{\partial \mathbf{h}_t} + \cdots + \frac{\partial \mathcal{L}_T}{\partial \mathbf{h}_T} \cdot \frac{\partial \mathbf{h}_T}{\partial \mathbf{h}_{T-1}} \cdots \frac{\partial \mathbf{h}_{t+1}}{\partial \mathbf{h}_t}. \quad (3.53)$$

输入: 输入序列 $\{\mathbf{x}_1, \dots, \mathbf{x}_T\}$, 目标序列 $\{\mathbf{y}_1, \dots, \mathbf{y}_T\}$

输出: 梯度 $\partial \mathcal{L} / \partial \mathbf{W}_{xh}$, $\partial \mathcal{L} / \partial \mathbf{W}_{hh}$, $\partial \mathcal{L} / \partial \mathbf{W}_{hy}$

步骤 1: 正向传播;

计算所有时间步的 $\mathbf{h}_t$ 、 $\mathbf{o}_t$ 、 $\hat{\mathbf{y}}_t$, 保存中间值;

步骤 2: 初始化;

$\nabla_{\mathbf{h}_T} \mathcal{L} = \partial \mathcal{L}_T / \partial \mathbf{h}_T$;

步骤 3: 反向遍历时间步;

for $t = T, T-1, \dots, 1$ do

$\nabla_{\mathbf{h}_{t-1}} \mathcal{L} = \nabla_{\mathbf{h}_t} \mathcal{L} \cdot \partial \mathbf{h}_t / \partial \mathbf{h}_{t-1}$;

 累积对权重矩阵的梯度: $\partial \mathcal{L} / \partial \mathbf{W}_{hh} += \nabla_{\mathbf{h}_t} \mathcal{L} \cdot \partial \mathbf{h}_t / \partial \mathbf{W}_{hh}$;

end

Algorithm 1: RNN 的时间反向传播算法 (BPTT)

3.2.1.3 梯度消失与梯度爆炸

RNN 在实际训练中面临最严峻的挑战——梯度消失 (Vanishing Gradients) 与梯度爆炸 (Exploding Gradients)。这两个问题直接限制了 RNN 捕捉长距离依赖的能力。

3.2.1.3.1 数学根源 从 BPTT 的梯度链式法则中, 相邻时间步的梯度传播因子为:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} = \mathbf{W}_{hh}^\top \cdot \text{diag}(\sigma'(\mathbf{a}_t)).$$

当梯度从 t 传播到 $t-k$ 时 (跨 k 步), 需要乘以 k 次该因子:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_{t-k}} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} \cdot \prod_{i=t-k+1}^t \frac{\partial \mathbf{h}_i}{\partial \mathbf{h}_{i-1}}.$$

定义 3.10 (梯度问题的临界条件)

设 λ 为矩阵 $\mathbf{W}_{hh}^\top \cdot \text{diag}(\sigma'(\mathbf{a}_t))$ 的谱半径 (最大特征值绝对值):

- 若 $\lambda > 1$: 梯度指数级放大 $\rightarrow$ 梯度爆炸。
- 若 $\lambda < 1$: 梯度指数级衰减 $\rightarrow$ 梯度消失。



3.2.1.3.2 梯度消失的后果

- 早期时间步的梯度趋近于 0, 网络无法学习长距离依赖。

- 例如：在句子 “*The cat, which was chased by the dog, was hungry.*” 中，RNN 难以将主语 *cat* 与谓语 *was* 关联起来（间隔多个词）。
- 隐状态 $\mathbf{h}_t$ 主要由最近的输入决定，丢失了早期上下文信息。

3.2.1.3.3 梯度爆炸的后果

- 梯度值变得极大（可达 10^8 量级），导致参数更新步长过大。
- 损失函数值出现剧烈震荡，甚至产生 NaN（数值溢出）。
- 网络权重被“炸飞”，无法继续训练。

问题	策略	具体做法
梯度爆炸	梯度裁剪（Gradient Clipping）	将梯度的范数限制在阈值 θ 内： $\mathbf{g} \leftarrow \mathbf{g} \cdot \min(1, \theta/\ \mathbf{g}\)$
梯度消失	激活函数选择	使用 ReLU 替代 $\tanh$ （ σ' 在正半轴为 1）
梯度消失	参数初始化	使用正交初始化（Orthogonal Initialization）
梯度消失	架构改进	引入 LSTM 或 GRU（见 §2.2）

3.2.2 长短期记忆网络 LSTM

长短期记忆网络（Long Short-Term Memory, LSTM）由 Hochreiter & Schmidhuber 于 1997 年提出，是解决 RNN 梯度消失问题最具里程碑意义的架构创新。LSTM 通过引入门控机制（Gating Mechanism）和细胞状态（Cell State），实现了对信息流的精确控制。

3.2.2.1 LSTM 的核心创新

LSTM 与标准 RNN 的本质区别在于：RNN 的隐状态 $\mathbf{h}_t$ 在每个时间步都被完全覆盖（旧信息被新信息完全替换）；而 LSTM 引入了一个独立的细胞状态 $\mathbf{c}_t$ ，通过三个门控单元选择性地读写信息。

3.2.2.2 LSTM 的数学形式

LSTM 的核心计算公式如下：

1. 遗忘门（Forget Gate）：决定丢弃多少旧信息。

$$\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f), \quad (3.54)$$

其中 σ 为 Sigmoid 函数，输出 $[0, 1]$ 之间的值。

2. 输入门（Input Gate）：决定写入多少新信息。

$$\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i), \quad (3.55)$$

3. 候选值（Candidate Value）：当前时间步的候选信息。

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c), \quad (3.56)$$

4. 细胞状态更新：结合遗忘门和输入门更新长期记忆。

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \quad (3.57)$$

5. 输出门（Output Gate）：决定输出多少信息到隐状态。

$$\mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o), \quad (3.58)$$

6. 隐状态输出：

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t). \quad (3.59)$$

其中 $[\mathbf{h}_{t-1}, \mathbf{x}_t]$ 表示两个向量的拼接（维度为 $d_h + d_x$ ）， $\odot$ 表示逐元素乘法（Hadamard 积）。

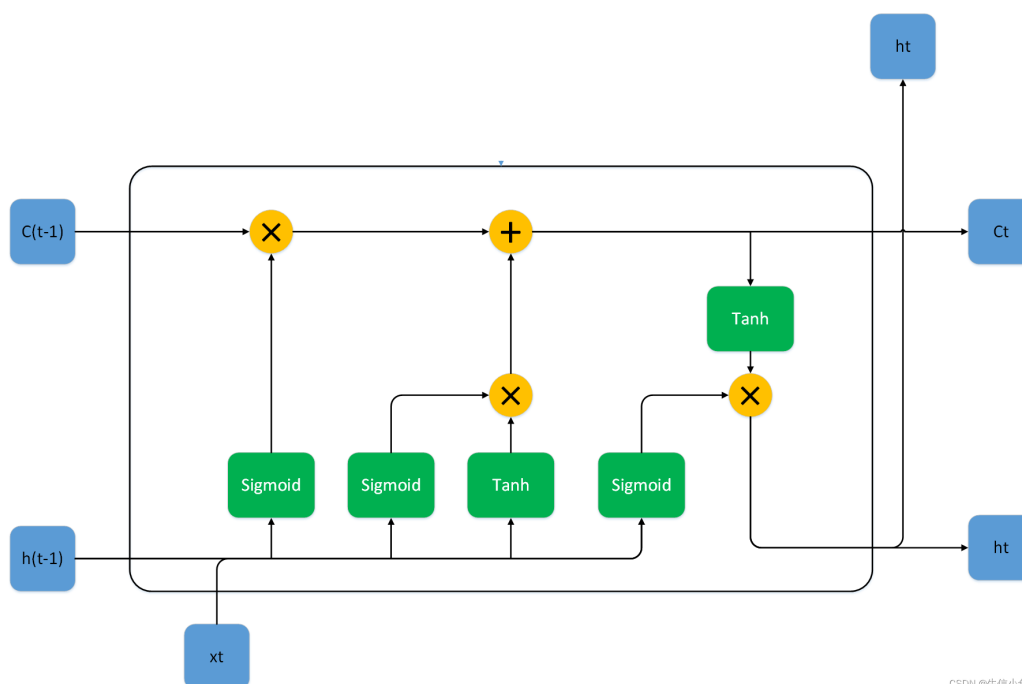


图 3.4: LSTMCell

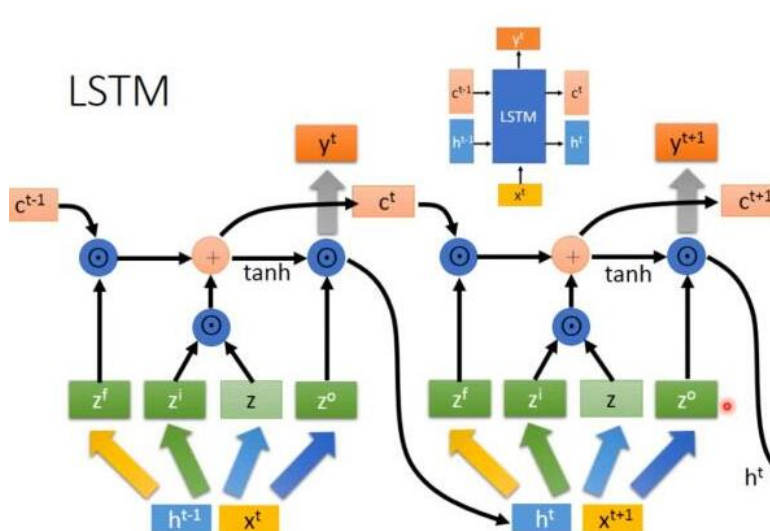


图 3.5: LSTMLayer

3.2.2.3 LSTM 如何解决梯度消失？

LSTM 通过两条关键设计解决了 RNN 的梯度消失问题：

1. 细胞状态 c_t 的恒等传播路径

在细胞状态的更新公式 $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$ 中，存在一条从 c_{t-1} 到 c_t 的直通路径：

- 当遗忘门 $f_t \approx 1$ 时，旧信息以接近 1 的比例保留。
- 梯度在传播时， $\partial c_t / \partial c_{t-1} = f_t$ 。
- 当 f_t 接近 1 时，梯度几乎无损地沿着细胞状态回传。

2. 门控值的饱和特性

遗忘门 f_t 由 Sigmoid 函数控制，输出值在 $[0, 1]$ 之间：

- 当 $f_t \rightarrow 1$ 时, $\partial c_t / \partial c_{t-1} = 1$, 梯度无衰减。
- 当 $f_t \rightarrow 0$ 时, 梯度被“切断”, 网络自动选择遗忘。
- 相比 RNN 中固定的 $\mathbf{W}_{hh}$ 矩阵 (梯度随步长指数衰减), LSTM 让网络自主决定保留多少梯度。

例题 3.1 LSTM 与 RNN 的梯度传播对比 RNN 传播 k 步的梯度因子: $\prod_{i=1}^k \mathbf{W}_{hh}^\top \text{diag}(\sigma'(\mathbf{a}_i))$ (指数衰减或爆炸)
LSTM 传播 k 步的梯度因子: $\prod_{i=1}^k \mathbf{f}_i$ (每个 $\mathbf{f}_i \in [0, 1]$, 由网络学习控制)

命题 3.28 (LSTM 的 Peephole Connection 与门控变体)

长短期记忆网络 (Long Short-Term Memory, LSTM) 通过精巧的门控机制解决了循环神经网络中的长期依赖问题。在其演进历程中, **Peephole Connection** (窥视孔连接) 与 耦合输入门与遗忘门是两项重要的结构变体, 分别从“信息可见性”与“参数精简性”两个维度对原始 LSTM 进行了改进。

一、标准 LSTM 回顾 (基线模型)

为便于对比, 首先给出标准 LSTM (Hochreiter & Schmidhuber, 1997) 在时刻 t 的更新方程。设 $x_t \in \mathbb{R}^d$ 为当前时刻输入, $h_{t-1} \in \mathbb{R}^H$ 为上一时刻隐状态, $c_{t-1} \in \mathbb{R}^H$ 为上一时刻细胞状态。标准 LSTM 的计算过程为:

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f), \quad (\text{遗忘门}) \quad (3.60a)$$

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i), \quad (\text{输入门}) \quad (3.60b)$$

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c), \quad (\text{候选细胞状态}) \quad (3.60c)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \quad (\text{细胞状态更新}) \quad (3.60d)$$

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o), \quad (\text{输出门}) \quad (3.60e)$$

$$h_t = o_t \odot \tanh(c_t). \quad (\text{隐状态输出}) \quad (3.60f)$$

其中 $\sigma(\cdot)$ 为 Sigmoid 函数 (输出范围 $(0, 1)$), $\tanh(\cdot)$ 为双曲正切函数 (输出范围 $(-1, 1)$), $\odot$ 表示逐元素乘法 (Hadamard 积)。

关键观察: 所有门控信号 f_t, i_t, o_t 仅依赖于当前输入 x_t 与上一时刻隐状态 h_{t-1} , 与细胞状态 c_{t-1} 或 c_t 无直接连接。这意味着门控单元在决定“遗忘多少旧信息”或“输出多少新信息”时, 对细胞状态本身的内容是“盲目”的。

二、Peephole Connection (窥视孔连接)

Peephole Connection 由 Gers & Schmidhuber (2000) 提出, 其核心思想是: 允许门控单元在计算时“窥视”细胞状态, 从而基于细胞状态的实际内容来调节信息流动, 而非仅依赖隐状态 h_{t-1} 的间接表示。

定义 3.11 (窥视孔连接的动机)

在标准 LSTM 中, 隐状态 h_{t-1} 是细胞状态 c_{t-1} 经过输出门过滤后的“压缩视图”, 丢失了细胞状态中的精细信息。Peephole Connection 直接将细胞状态的完整向量连接到门控单元, 使门控决策能基于细胞状态的精确数值进行细粒度调节。



Peephole 连接的具体实现

Peephole LSTM 的更新方程在标准 LSTM 基础上, 为三个门控单元各增加一项“细胞状态”的线性投影:

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + P_f c_{t-1} + b_f), \quad (\text{遗忘门窥视上一时刻细胞状态}) \quad (3.61a)$$

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + P_i c_{t-1} + b_i), \quad (\text{输入门窥视上一时刻细胞状态}) \quad (3.61b)$$

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c), \quad (\text{候选细胞状态, 无窥视}) \quad (3.61c)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \quad (\text{细胞状态更新}) \quad (3.61d)$$

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + P_o c_t + b_o), \quad (\text{输出门窥视当前时刻细胞状态}) \quad (3.61e)$$

其中 $P_f, P_i, P_o \in \mathbb{R}^{H \times H}$ 为对角矩阵（在实际实现中通常使用向量 $p_f, p_i, p_o \in \mathbb{R}^H$ 进行逐元素乘，以减少参数量）。使用对角矩阵而非稠密矩阵的原因有二：

1. 参数效率：将参数量从 $3H^2$ 降至 $3H$ ，避免引入过多的额外参数。
2. 防止过拟合：特别是在小规模数据集上，稠密的窥视连接容易导致过拟合。

关键差异：输出门的窥视目标

值得注意的是，遗忘门与输入门窥视的是上一时刻细胞状态 c_{t-1} ，而输出门窥视的是当前时刻（已更新后）的细胞状态 c_t 。这种设计差异的逻辑是：

- 遗忘门与输入门在更新 c_t 之前执行，只能基于旧状态 c_{t-1} 决策。
- 输出门在 c_t 更新之后执行，可以基于全新的细胞状态来决定输出内容，这使得输出门能更精确地调控哪些“新信息”被暴露给隐状态 h_t 。

三、耦合输入门与遗忘门（Coupled Input & Forget Gates）

耦合输入门与遗忘门是另一种常见的 LSTM 变体，其动机源于对标准 LSTM “冗余门控” 的观察：在许多任务中，输入门与遗忘门的行为往往呈现此消彼长的互补关系——当网络决定更多地保留旧信息时（遗忘门接近 1），通常意味着较少需要新信息（输入门接近 0），反之亦然。

定义 3.12 (门控耦合的假设)

标准 LSTM 中，输入门 i_t 与遗忘门 f_t 各自独立训练，但二者在功能上天然互补。将它们耦合为 $i_t = 1 - f_t$ 可以强制这种互补关系，同时将参数减少一个门控单元，降低过拟合风险。



耦合变体的数学形式

耦合输入门与遗忘门的 LSTM (Gers et al., 2000; 亦见于 Cho et al., 2014 的 GRU 设计) 将更新方程简化为：

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f), \quad (\text{仅保留遗忘门}) \quad (3.62a)$$

$$i_t = 1 - f_t, \quad (\text{输入门为遗忘门的补数}) \quad (3.62b)$$

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c), \quad (\text{候选细胞状态}) \quad (3.62c)$$

$$c_t = f_t \odot c_{t-1} + (1 - f_t) \odot \tilde{c}_t, \quad (\text{细胞状态更新}) \quad (3.62d)$$

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o), \quad (\text{输出门保持不变}) \quad (3.62e)$$

$$h_t = o_t \odot \tanh(c_t). \quad (\text{隐状态输出}) \quad (3.62f)$$

核心约束： $i_t + f_t = 1$ 逐元素成立。这强制网络在“遗忘”与“写入”之间进行明确的权衡：有多少旧信息被遗忘，就有多少空间被新信息填充。

与 GRU 的关系

Cho 等人在 2014 年提出的 GRU (Gated Recurrent Unit) 将这一思想推向极致: GRU 彻底合并了细胞状态 c_t 与隐状态 h_t , 并使用单一的更新门 (Update Gate) 同时控制遗忘与写入, 进一步精简了参数。



3.2.2.4 GRU: LSTM 的轻量级变体

门控循环单元 (Gated Recurrent Unit, GRU) 由 Cho 等人在 2014 年提出, 是 LSTM 的简化版本:

特性	LSTM	GRU
门控单元数量	3 个门 (遗忘门、输入门、输出门)	2 个门 (重置门、更新门)
细胞状态	有独立的 c_t 和 h_t	无独立细胞状态, 仅 h_t
参数数量	更多 (约 4 倍于 RNN)	更少 (约 3 倍于 RNN)
计算复杂度	较高	较低
表达能力	更强大 (但可能过拟合)	足够应对大多数任务

GRU 的核心公式:

$$\begin{aligned}
 \mathbf{z}_t &= \sigma(\mathbf{W}_z[\mathbf{h}_{t-1}, \mathbf{x}_t]), \\
 \mathbf{r}_t &= \sigma(\mathbf{W}_r[\mathbf{h}_{t-1}, \mathbf{x}_t]), \\
 \tilde{\mathbf{h}}_t &= \tanh(\mathbf{W}_h[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t]), \\
 \mathbf{h}_t &= (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t.
 \end{aligned}$$

3.2.2.5 LSTM 的工程实践建议

1. **梯度裁剪**: 即使在 LSTM 中, 梯度爆炸仍可能发生 (特别是在长序列上)。通常设置梯度范数阈值为 1.0。
2. **丢弃机制 (Dropout)**: 在 LSTM 层之间使用 Dropout (而非层内), 常用变体为 `nn.LSTM(..., dropout=0.3)`。
3. **双向 LSTM**: 对于分类任务, 使用双向 LSTM (BiLSTM) 可同时捕获前向和后向上下文。
4. **堆叠 LSTM**: 深层 LSTM (2-4 层) 通常优于单层, 但需注意过拟合。
5. **注意力机制**: 在 LSTM 顶部添加注意力层可进一步提升长序列性能 (见后续章节)。

表 3.6: RNN / LSTM 关键参数配置建议

超参数	推荐值	说明
隐状态维度 d_h	128-512	取决于任务复杂度和数据量
层数	2-4	超过 4 层收益递减
Dropout 率	0.2-0.5	在层间使用
梯度裁剪阈值	1.0	防止梯度爆炸
学习率	10^{-4} - 10^{-3}	使用 Adam 优化器

小结

从 RNN 到 LSTM 的演进反映了深度学习发展的一个核心主题: ** 如何让神经网络在长时间跨度上有效地传递信息 **。RNN 通过参数共享和循环连接建立了处理序列的基本框架, 但其固有的梯度消失/爆炸问题限制了其建模长距离依赖的能力。LSTM 通过引入门控机制和细胞状态, 为信息流开辟了一条“高速公路”, 使得网络能够自主决定保留或遗忘哪些信息, 从而将有效序列长度从约 10 个时间步扩展到数百甚至上千步。

LSTM 的提出深刻地影响了机器翻译、语音识别、文本生成等众多 NLP 任务。然而，RNN 及 LSTM 均具有串行计算的本质特性（ t 时刻必须等待 $t-1$ 时刻计算完成），这限制了其在现代大规模数据上的训练效率。这一瓶颈最终催生了基于自注意力机制（Self-Attention）的 Transformer 架构——这也是我们下一节将要讲述的内容。

3.3 从自然语言处理到大语言模型

自然语言处理（Natural Language Processing, NLP）是人工智能领域中最富挑战性的分支之一。从早期的统计语言模型，到神经网络时代的 Seq2Seq 框架，再到注意力机制的诞生与 Transformer 架构的统治，NLP 在短短十年间经历了跨越式的发展。本节沿着这条技术演进的主线，从最基础的序列到序列模型讲起，逐步揭示现代大语言模型（Large Language Model, LLM）的核心技术基石。

3.3.1 普通的 Seq2Seq 模型

序列到序列（Sequence-to-Sequence, Seq2Seq）模型由 Sutskever 等人在 2014 年提出，是第一个真正意义上实现“任意长度序列到任意长度序列”映射的端到端神经网络框架。它为机器翻译、文本摘要、对话生成等任务提供了统一的建模范式。

3.3.1.1 架构总览：编码器-解码器（Encoder-Decoder）

Seq2Seq 模型的核心架构可概括为两个协同工作的循环神经网络：

1. **编码器（Encoder）**：逐词读取输入序列 $X = (x_1, x_2, \dots, x_T)$ ，并将其压缩为一个固定长度的上下文向量 c （Context Vector），即语义编码。
2. **解码器（Decoder）**：以 c 为初始状态，自回归地逐词生成输出序列 $Y = (y_1, y_2, \dots, y_{T'})$ 。

编码器与解码器通常采用 RNN 或其变体（LSTM、GRU），以建模序列的时序依赖关系。训练时使用教师强制（Teacher Forcing）策略，即解码器在每一步接收真实的前一词作为输入，而非自己的预测结果，从而加速收敛并稳定训练。

3.3.1.2 信息瓶颈问题（Information Bottleneck）

Seq2Seq 模型最为人诟病的缺陷在于其固定长度的上下文向量 c 。无论输入序列长度如何，编码器都必须将所有信息压缩到一个定长向量中。当输入序列较长时（如包含 50 个以上单词的句子），编码器难以将全部语义信息无损地压缩进 c ，导致解码器生成质量急剧下降。

定义 3.13 (信息瓶颈)

编码器输出的最后一个隐状态 h_T （或所有隐状态的某种池化结果）作为唯一的上下文载体。当序列长度 T 增大时， h_T 的容量成为生成质量的硬约束，网络被迫“遗忘”早期输入的信息。



这一问题直接催生了注意力机制的诞生。

3.3.2 注意力机制（Attention Mechanism）

注意力机制由 Bahdanau 等人在 2015 年首次引入 Seq2Seq 框架，其核心思想是：在解码的每一步，动态地“关注”编码器输出中与当前生成词最相关的部分，而非机械地依赖单一的固定向量。

3.3.2.1 注意力机制的核心公式

以 Bahdanau 注意力（Additive Attention）为例，假设编码器输出了全部隐状态序列 $\mathbf{H} = (h_1, h_2, \dots, h_T)$ ，解码器当前步的隐状态为 s_{t-1} 。注意力机制的计算流程如下：

1. 对齐分数 (Alignment Scores): 计算 s_{t-1} 与每个编码器隐状态 h_j 的关联强度:

$$e_{t,j} = \text{score}(s_{t-1}, h_j) = v_a^T \tanh(W_a s_{t-1} + U_a h_j),$$

其中 v_a, W_a, U_a 为可学习参数。

2. 归一化权重 (Attention Weights): 通过 Softmax 将分数转换为概率分布:

$$\alpha_{t,j} = \frac{\exp(e_{t,j})}{\sum_{k=1}^T \exp(e_{t,k})}, \quad \sum_{j=1}^T \alpha_{t,j} = 1.$$

3. 上下文向量 (Context Vector): 对所有编码器隐状态进行加权求和:

$$c_t = \sum_{j=1}^T \alpha_{t,j} h_j.$$

4. 解码器更新: 将 c_t 与当前输入 y_{t-1} 拼接后送入 RNN 单元, 计算 s_t , 进而预测输出词 y_t 。

命题 3.29 (交叉注意力的两种经典实现: 加性注意力与乘性注意力)

在序列到序列 (Seq2Seq) 模型中, 解码器通过交叉注意力 (Cross-Attention) 从编码器输出中提取与当前生成相关的信息。学术界存在两种经典的实现范式——Bahdanau 加性注意力 (Additive Attention) 与 Luong 乘性注意力 (Multiplicative Attention)。二者在数学形式和计算效率上存在差异, 但在“查询-键-值”的宏观框架下本质相通。

一、Bahdanau 注意力 (加性注意力, 2015)

Bahdanau 等人在 2015 年首次将注意力机制引入 Seq2Seq, 其对齐分数 (Alignment Score) 的计算方式为:

$$e_{t,j} = v_a^T \tanh(W_a s_{t-1} + U_a h_j),$$

其中:

- $s_{t-1} \in \mathbb{R}^{d_{\text{dec}}}$: 解码器上一时刻的隐状态 (作为查询 Q 的来源);
- $h_j \in \mathbb{R}^{d_{\text{enc}}}$: 编码器第 j 个位置的隐状态 (作为键 K 的来源);
- $W_a \in \mathbb{R}^{d_a \times d_{\text{dec}}}$ 、 $U_a \in \mathbb{R}^{d_a \times d_{\text{enc}}}$ 、 $v_a \in \mathbb{R}^{d_a}$: 可学习参数;
- $\tanh(\cdot)$: 引入非线性变换。

随后通过 Softmax 归一化得到注意力权重:

$$\alpha_{t,j} = \frac{\exp(e_{t,j})}{\sum_{k=1}^T \exp(e_{t,k})}.$$

最终上下文向量为编码器隐状态的加权和:

$$c_t = \sum_{j=1}^T \alpha_{t,j} h_j.$$

特点

- 参数化方式: 通过一个前馈网络 ($v_a^T \tanh(\cdot)$) 来学习 s_{t-1} 与 h_j 之间的关联, 属于可学习的非线性对齐。
- 计算开销: 每步需对所有 T 个编码器隐状态执行前馈网络计算, 复杂度为 $\mathcal{O}(T \cdot d_a)$ 。
- 历史地位: 这是历史上第一个注意力机制, 首次证明了“动态对齐”的有效性。

二、Luong 注意力（乘性注意力，2015）

Luong 等人在同年提出了更为简洁的乘性注意力（Multiplicative Attention），其对齐分数直接计算为解码器隐状态与编码器隐状态之间的点积：

$$e_{t,j} = s_t^T W_a h_j,$$

其中 $W_a \in \mathbb{R}^{d_{\text{dec}} \times d_{\text{enc}}}$ 为可学习投影矩阵（如果 $d_{\text{dec}} = d_{\text{enc}}$ ，则可简化为 $s_t^T h_j$ ，无需额外参数）。

你描述的流程正是 Luong 注意力的标准实现：

1. 将编码器隐状态 $\mathbf{H} \in \mathbb{R}^{T \times d_{\text{enc}}}$ 通过线性变换 W_K 映射为键（Keys）： $K = HW_K$ ；
2. 将解码器隐状态 s_t 通过线性变换 W_Q 映射为查询（Query）： $Q = s_t W_Q$ ；
3. 计算点积 QK^T 并除以 $\sqrt{d_k}$ 进行缩放（Scaled Dot-Product）；
4. 通过 Softmax 归一化得到注意力权重 α_t ；
5. 将编码器隐状态通过线性变换 W_V 映射为值（Values）： $V = HW_V$ ；
6. 上下文向量为 $c_t = \sum_j \alpha_{t,j} V_j$ （或直接使用 H 不加变换，取决于实现）。

三、两种范式的对比

四、Transformer 使用的是哪种？

Transformer 使用的是缩放点积注意力（Scaled Dot-Product Attention），它本质上是 Luong 乘性注意力的一种推广和标准化：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V.$$

其中 Q, K, V 分别通过独立的线性投影层得到：

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V.$$

与 Luong 注意力的继承关系：

- Luong 的查询只来自解码器当前步的单个隐状态 s_t ，而 Transformer 的查询来自序列中所有位置（多头注意力中的每一头）。
- Transformer 引入了 $\sqrt{d_k}$ 缩放因子，以控制点积值的数量级，防止 Softmax 梯度进入饱和区。
- Transformer 将 Q, K, V 分别投影到不同的子空间，多头机制允许模型关注不同语义层面。

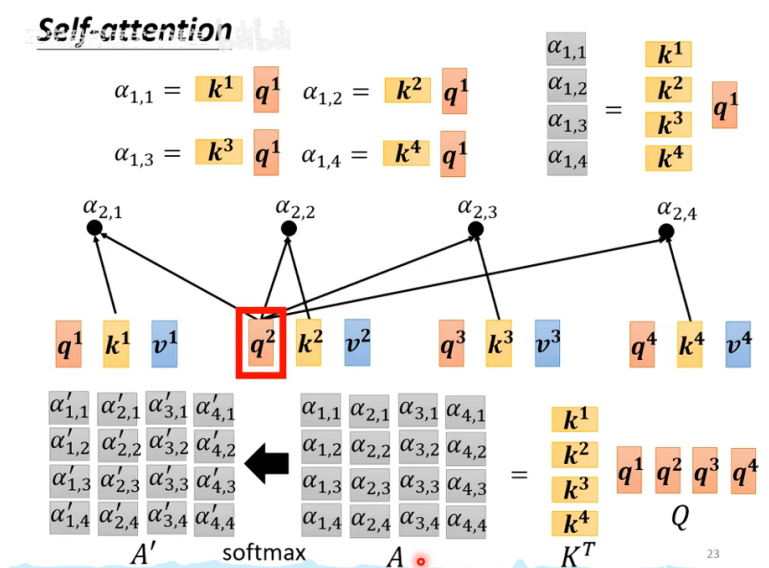
3.3.2.2 注意力机制的三层递进理解

1. 从“只看一眼”到“看全貌”：传统 Seq2Seq 的解码器只看一眼编码器的最终隐状态；注意力机制允许解码器在每一步“回头重读”整个输入序列。
2. 权重即“对齐”：注意力权重 $\alpha_{t,j}$ 可视作输入词与输出词之间的软对齐（Soft Alignment）。在机器翻译中， $\alpha_{t,j}$ 往往自然地对应于输入序列中与当前目标词语义对应的源词位置。
3. 计算复杂度：注意力机制的引入将 Seq2Seq 的时间复杂度从 $\mathcal{O}(T + T')$ 提升至 $\mathcal{O}(T \cdot T')$ ，但在当时的长序列任务中带来了显著的性能提升。

3.3.2.3 自注意力（Self-Attention）的引入

Bahdanau 注意力属于编码器-解码器注意力（Cross-Attention），即查询来自解码器，键与值来自编码器。Vaswani 等人在 2017 年进一步提出了自注意力（Self-Attention）：查询、键、值全部来自同一序列内部，通过注

图 3.6: 注意力机制的可视化示意（软对齐权重矩阵）



注意力机制捕获序列内任意两个位置之间的长距离依赖关系。

自注意力的核心优势在于：

- **全局感受野**：RNN 的依赖传递需要 $\mathcal{O}(T)$ 步，自注意力仅需一步即可建立任意两词之间的直接连接。
- **可并行化**：自注意力的计算完全基于矩阵乘法，无需像 RNN 那样逐时序依赖，从而充分利用 GPU 的并行计算能力。

定义 3.14 (缩放点积注意力)

自注意力的标准形式为缩放点积注意力（Scaled Dot-Product Attention）：

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V,$$

其中 Q, K, V 分别为查询矩阵、键矩阵与值矩阵， d_k 为键向量的维度。缩放因子 $\sqrt{d_k}$ 用于防止点积过大导致 Softmax 梯度过小。

3.3.3 Transformer

Transformer 由 Vaswani 等人在 2017 年发表于论文 *Attention Is All You Need*，是 NLP 历史上最具颠覆性的架构创新。它彻底抛弃了 RNN 与 CNN，完全基于自注意力机制构建编码器与解码器，奠定了当前所有大语言模型（GPT、BERT、LLaMA 等）的架构基础。

3.3.3.1 总体架构

Transformer 遵循编码器-解码器范式，但编码器与解码器均由 N 个相同的层堆叠而成（原论文中 $N = 6$ ）。每层包含两个或三个核心子模块，每个子模块后均接残差连接（Residual Connection）与层归一化（Layer Normalization）。

1. 编码器层（每层）：

- 多头自注意力（Multi-Head Self-Attention）
- 逐位置前馈网络（Position-wise Feed-Forward Network）

2. 解码器层（每层）：

- 掩蔽多头自注意力（Masked Multi-Head Self-Attention）——防止关注未来位置
- 编码器-解码器注意力（Cross-Attention）——连接编码器输出

- 逐位置前馈网络

3.3.3.2 多头注意力（Multi-Head Attention）

多头注意力是 Transformer 的核心组件，其思想是：将查询、键、值分别投影到 h 个不同的子空间，在每个子空间中独立执行注意力计算，最后将所有头的输出拼接并线性投影。

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O, \quad (3.63)$$

其中：

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V). \quad (3.64)$$

多头机制的意义：不同的注意力头可以关注序列中不同类型的语义关系（如语法结构、指代关系、局部上下文等），类似于 CNN 中不同卷积核提取不同的特征模式。

3.3.3.3 位置编码（Positional Encoding）

由于自注意力本身是置换不变的（Permutation Invariant）——即打乱输入顺序，注意力计算结果不变——Transformer 必须显式注入位置信息。原论文采用正弦-余弦位置编码：

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \quad (3.65a)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right). \quad (3.65b)$$

其中 pos 为位置索引， i 为维度索引。这种编码方式的优点在于：任意两个位置之间的内积仅依赖于它们的相对偏移量 $pos_1 - pos_2$ ，使得模型能够泛化到训练中未见过的序列长度。

3.3.3.4 逐位置前馈网络（Position-wise FFN）

每个 Transformer 层中的前馈网络是一个简单的两层全连接网络，但在序列的每个位置上独立且相同地应用：

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2.$$

设计意图：自注意力是线性变换的组合（加权求和），缺乏非线性能力。逐位置 FFN 引入非线性激活函数（ReLU），为模型提供足够的表示能力。

3.3.3.5 掩蔽机制（Masking）

在解码器的训练中，当前词不应该看到未来词，否则会造成信息泄露。Transformer 通过未来掩蔽（Future Masking）实现：将 QK^T 中所有 $j > i$ 的位置设为 $-\infty$ （经过 Softmax 后变为 0），使得第 i 个位置只能关注到 $1, 2, \dots, i$ 位置。

3.3.3.6 Transformer 的优势与局限

主要局限：自注意力的 $\mathcal{O}(T^2)$ 复杂度限制了 Transformer 处理超长序列（如全书级别的文本）的能力。后续工作如 Longformer、BigBird、FlashAttention 等致力于突破这一瓶颈。

从 Transformer 到大语言模型

Transformer 的诞生为 NLP 带来了范式级的转变。其后的发展轨迹可概括为两条主线：

表 3.7: Transformer 相对 RNN 的优劣对比

维度	Transformer	RNN / LSTM
并行计算	完全并行（矩阵乘法）	严格时序依赖，无法并行
长距离依赖	一步到位（全局感受野）	需传递 T 步，存在衰减
空间复杂度	$\mathcal{O}(T^2)$ （注意力矩阵）	$\mathcal{O}(T)$ （隐状态）
位置建模	需显式位置编码	天然具备时序次序
训练稳定性	受 LayerNorm 保障	易梯度消失/爆炸

1. **自编码预训练（Autoencoding）**: 以 BERT 为代表，通过掩蔽语言模型（Masked Language Model）学习双向上下文表示，擅长自然语言理解（NLU）任务。
2. **自回归预训练（Autoregressive）**: 以 GPT 系列为代表，通过因果语言模型（Causal Language Model）从左到右逐词生成，擅长自然语言生成（NLG）任务。

当模型规模扩展至数十亿乃至数千亿参数，并在海量文本语料上进行预训练后，GPT-3 及其后继者展现出强大的上下文学习（In-Context Learning）与指令遵循（Instruction Following）能力，这些模型被统称为大语言模型（Large Language Model, LLM）。LLM 的核心范式已从“预训练 + 微调”转向“预训练 + 提示（Prompting）”，这一转变深刻重塑了整个 NLP 的研究与产业格局。

定理 3.1 (自注意力的置换不变性)

设 π 为输入序列的任意置换。自注意力机制 $\text{Attention}(X, X, X)$ 满足：

$$\text{Attention}(\pi(X), \pi(X), \pi(X)) = \pi(\text{Attention}(X, X, X)).$$

即自注意力的输出随输入序列的置换而等变。这一性质表明，自注意力本身不具备位置敏感能力，位置信息必须通过位置编码或相对位置偏置等外部机制注入。



3.3.4 基于 Transformer 的大模型架构

Transformer 架构的提出为自然语言处理带来了范式级的变革。在其基础上，研究者们通过扩展模型规模、创新预训练目标与优化训练策略，催生了以 BERT、GPT、Gemini 和 Claude 为代表的大语言模型（Large Language Model, LLM）家族。这些模型不仅在性能上持续刷新榜单，更从根本上重塑了人与 AI 交互的方式。

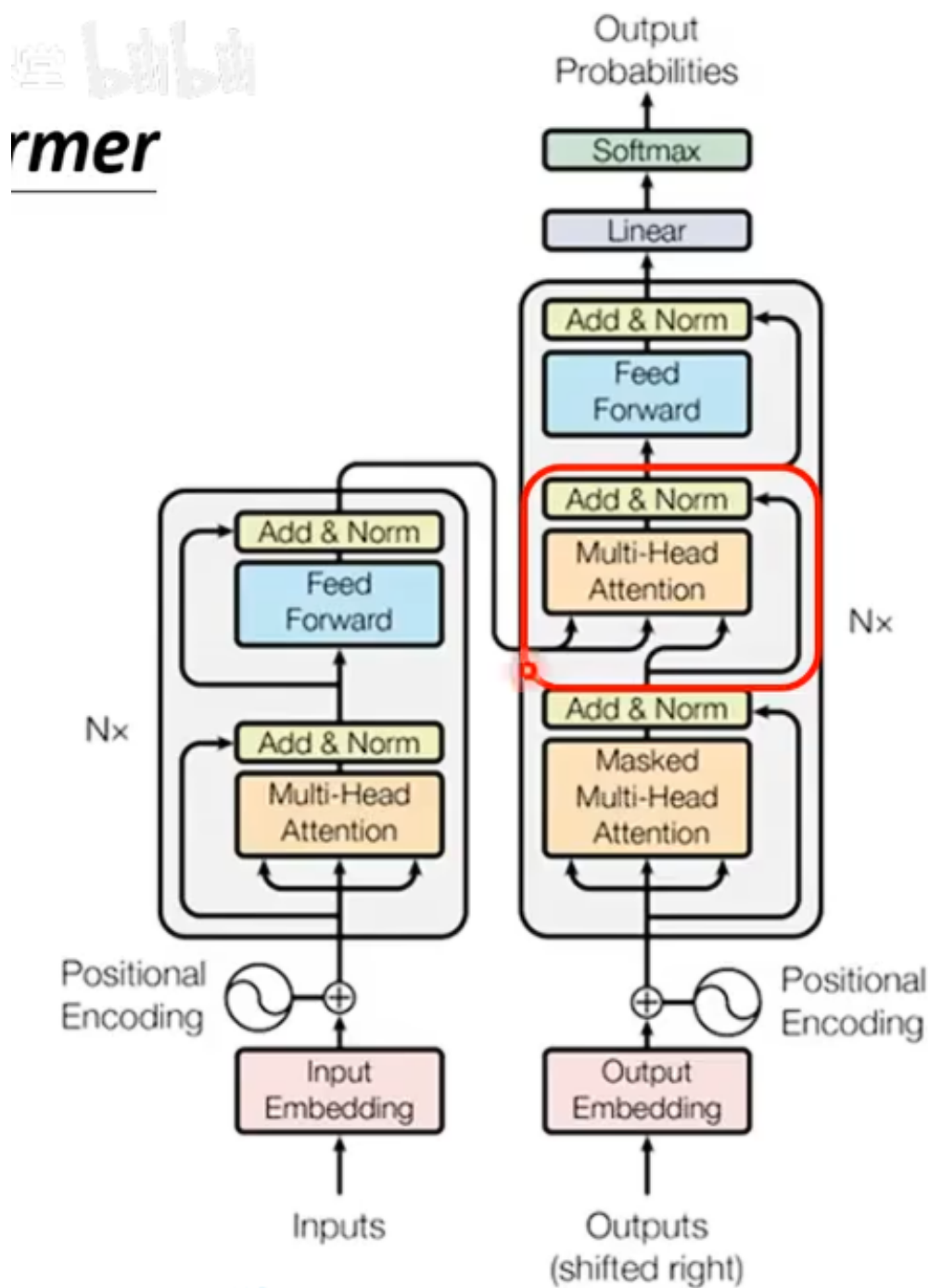
3.3.4.1 BERT：双向编码器表示的里程碑

BERT（Bidirectional Encoder Representations from Transformers）由 Devlin 等人在 2018 年提出，是第一个真正意义上大规模双向预训练语言模型。其核心贡献在于证明了双向上下文建模对自然语言理解（NLU）任务的至关重要性。

3.3.4.1.1 预训练任务 BERT 采用两个联合训练的预训练目标：

1. **掩蔽语言模型（Masked Language Model, MLM）**:
 - 随机掩蔽输入序列中 15% 的 Token，其中 80% 替换为 [MASK]，10% 替换为随机 Token，10% 保持不变。
 - 模型需要根据上下文预测被掩蔽位置的原始 Token。
 - 这一设计使得模型能够学习双向上下文表示，打破了 GPT 等自回归模型的单向限制。
2. **下一句预测（Next Sentence Prediction, NSP）**:
 - 输入为两个句子 A 和 B（以 [SEP] 分隔），模型判断 B 是否为 A 的下一句。
 - 正样本来自语料中的连续句子对，负样本随机采样。
 - 该任务旨在让模型学习句子级别的语义连贯性，为问答（QA）和自然语言推理（NLI）等下游任务提供支持。

图 3.7: 注意力机制的可视化示意（软对齐权重矩阵）



3.3.4.1.2 模型变体与影响力 BERT 发布时提供了两个版本：

- **BERT-Base:** $L = 12$ 层, $H = 768$, $A = 12$ 个注意力头, 参数量 110M。
- **BERT-Large:** $L = 24$ 层, $H = 1024$, $A = 16$ 个注意力头, 参数量 340M。

BERT 在 GLUE 基准测试中取得了 80.5% 的准确率（较之前最优提升 7.7%），在 SQuAD 问答任务上也大幅超越人类基线。更重要的是，BERT 开创了“预训练 + 微调（Pre-train + Fine-tune）”的 NLP 范式，该范式统治了 2018-2021 年的 NLP 领域。

定义 3.15 (掩蔽语言模型的数学形式)

给定输入序列 $X = (x_1, x_2, \dots, x_T)$ ，掩蔽位置集合为 $\mathcal{M}$ ，则 MLM 的训练目标为：

$$\mathcal{L}_{\text{MLM}} = - \sum_{i \in \mathcal{M}} \log p(x_i | X_{\setminus i}),$$

其中 $X_{\setminus i}$ 表示将第 i 个位置替换为 [MASK] 后的序列。



3.3.4.2 GPT 系列：自回归生成的技术演进

GPT (Generative Pre-trained Transformer) 系列由 OpenAI 推出，是自回归语言模型的代表。与 BERT 的双向编码不同，GPT 采用因果 (Causal) 解码器架构，从左到右逐词生成，天然适合自然语言生成 (NLG) 任务。

3.3.4.2.1 GPT-1: 奠基之作 (2018) GPT-1 首次提出“预训练 + 微调”的通用框架，采用 12 层 Transformer 解码器，参数量 117M。其关键创新在于：

- 在不同下游任务中，通过任务特定的输入变换将各类问题统一为序列生成格式，无需修改模型架构。
- 预训练使用标准的自回归语言建模目标：

$$\mathcal{L}_{\text{LM}} = - \sum_{i=1}^T \log p(x_i | x_1, \dots, x_{i-1}).$$

3.3.4.2.2 GPT-2: 规模扩展与零样本泛化 (2019) GPT-2 将参数量扩展至 1.5B，并在 40GB 的 WebText 数据上预训练。其最引人注目的发现是零样本 (Zero-Shot) 学习能力：在未经微调的情况下，GPT-2 在多个任务上展现出一定的下游迁移能力，预示着“规模即能力”的潜力。

例题 3.2 GPT-2 的零样本表现 在阅读理解任务 LAMBADA 上，GPT-2 的零样本准确率达到 52.7%，显著优于基于 N-gram 的基线模型（仅为 19.3%）。这一结果表明，大规模预训练模型能够隐式地学习任务模式。

3.3.4.2.3 GPT-3: 上下文学习的革命 (2020) GPT-3 是 NLP 史上的一个里程碑，参数量达到 175B，是 GPT-2 的 100 倍以上。其核心贡献在于提出了上下文学习 (In-Context Learning, ICL) 能力：

- 通过在 Prompt 中提供少数示例 (Few-Shot)，GPT-3 可以在不更新任何参数的情况下完成新任务。
- 在 SuperGLUE、TriviaQA 等多个基准上，GPT-3 的 Few-Shot 性能达到甚至超越当时的微调 SOTA。
- 这一发现催生了“提示工程 (Prompt Engineering)”这一新兴领域。

定义 3.16 (上下文学习 (In-Context Learning))

设模型参数 θ 在训练后固定。给定一个任务描述 Prompt P (包含 k 个示例和 1 个待预测样本)，上下文学习要求模型：

$$\hat{y} = \arg \max_y p_{\theta}(y | P, x_{\text{test}}),$$

即在条件概率分布中直接采样输出，无需任何梯度更新。



3.3.4.2.4 GPT-4: 多模态与对齐的飞跃 (2023) GPT-4 进一步扩展至多模态输入（接受图像与文本混合输入），并在 HumanEval 编程任务、SAT 数学考试等复杂基准上达到人类水平。其关键突破在于：

- 通过基于人类反馈的强化学习（RLHF）进行模型对齐，大幅减少有害输出，提升指令遵循能力。
- 上下文窗口扩展至 32K（后升级至 128K），能够处理整本书籍级别的长文档。
- 在标准基准测试中，GPT-4 的 MMLU 准确率达到 86.4%，显著优于 GPT-3.5 的 70.0%。

3.3.4.3 Gemini：原生多模态的统一架构

Gemini 由 Google DeepMind 于 2023 年底发布，是首个从零开始为原生多模态设计的模型系列。其核心理念是：不将文本、图像、音频等模态作为孤立的输入拼接，而是在模型架构层面实现多模态的统一表征。

3.3.4.3.1 架构特点

1. 原生多模态训练（Native Multimodal Training）：
 - 模型从预训练阶段即同时处理文本、图像、视频、音频和点云等多种模态。
 - 使用统一的多模态 Tokenizer 将不同模态的数据映射到共享的表示空间。
 - 这使得 Gemini 能够理解跨模态的复杂推理任务（如图表解读、视频帧间逻辑推断）。
2. 高效注意力机制：
 - 采用 FlashAttention 等高效注意力实现，支持 10^6 级超长上下文窗口。
 - 通过 混合专家（MoE, Mixture of Experts）架构，在保持推理效率的同时扩大模型容量。
3. 版本分级：
 - **Gemini Ultra**：最强版本，在 32 个基准测试中的 30 个上超越 GPT-4。
 - **Gemini Pro**：平衡性能与效率，服务于大多数生产场景。
 - **Gemini Nano**：轻量级版本，可在移动设备端侧运行。

3.3.4.3.2 多模态基准表现 Gemini Ultra 在 MMLU（多任务语言理解）上达到 90.0% 的准确率，首次超越人类专家基线（89.8%）。在多模态数学推理（MathVista）和视频理解（VATEX）等任务上，Gemini 也展现出显著优势。

表 3.8: Gemini 系列主要版本对比

版本	参数量	上下文窗口	主要应用场景
Gemini Ultra	未公开（估计 > 1T）	1M Token	科研、复杂推理
Gemini Pro	未公开（估计 > 100B）	128K Token	企业服务、API
Gemini Nano-1	1.8B	4K Token	端侧部署
Gemini Nano-2	3.25B	4K Token	端侧部署

3.3.4.4 Claude 系列：聚焦对齐与安全

Claude 系列由 Anthropic 公司开发，其核心差异化优势在于宪法 AI（Constitutional AI）框架，即通过明确的道德准则而非纯人类反馈来实现模型对齐。

3.3.4.4.1 宪法 AI（Constitutional AI） 宪法 AI 包含两个阶段：

1. 监督学习阶段：
 - 模型根据一份书面的“宪法准则”（包含联合国人权宣言、Anthropic 自研原则等）对自身生成的回复进行批评与修正。
 - 通过“自我批评 + 修订”的循环，生成符合准则的示范数据。
2. 强化学习阶段：
 - 使用 AI 反馈（而非人类反馈）进行强化学习（RLAIF）。
 - 训练一个偏好模型（Preference Model）来评估生成回复是否符合宪法准则，以此作为奖励信号。

这一框架的核心优势在于：大幅减少对人类标注数据的依赖，实现可扩展的模型对齐，且对齐规则透明可审计。

3.3.4.4.2 Claude 3 系列（2024）

Claude 3 系列包含三个版本：

- **Claude 3 Haiku**：速度最快，成本最低，适合实时交互场景。
- **Claude 3 Sonnet**：平衡智能与速度，适合大多数企业工作负载。
- **Claude 3 Opus**：最智能版本，在复杂推理、数学、编程等任务上达到 SOTA。

Claude 3 系列的关键技术亮点包括：

- **200K 上下文窗口**（部分场景可扩展至 1M Token），能够处理长篇小说、研究报告等超长文档。
- 在 MMLU、GSM-8K（数学推理）、HumanEval（代码生成）等基准上超越 GPT-4 和 Gemini Ultra。
- 更强的多语言能力与指令遵循精度，在复杂结构化输出任务中表现优异。

小结：从架构到智能的跃迁

从 BERT 到 GPT，再到 Gemini 和 Claude，大模型架构的演进呈现出三大趋势：

1. **规模持续扩张**：从百万级参数（BERT-Base）到万亿级参数（Gemini Ultra），规模是能力跃迁的第一驱动力。
2. **从单模态到原生多模态**：模型不再是“文本专用”，而是从一开始就设计为处理多模态信息的通用智能体。
3. **对齐技术的精细化**：从简单的“预训练 + 微调”到 RLHF、宪法 AI 等复杂对齐框架，模型的行为安全性和可控性得到显著提升。

这些趋势共同指向一个未来：大模型正在从“语言模型”进化为“通用智能模型”，其影响力将远不止于自然语言处理领域。

3.3.5 Transformer 在图像处理中的应用

Transformer 在自然语言处理领域取得统治地位后，研究者们开始探索将其应用于计算机视觉任务。从 ViT 将图像视为“词序列”开始，到 DETR 将目标检测重构为集合预测问题，再到 Swin Transformer 引入层次化设计以适配视觉任务的多尺度特性，Transformer 正在深刻重塑计算机视觉的架构格局。

3.3.5.1 ViT：将图像视为词序列

ViT（Vision Transformer）由 Dosovitskiy 等人在 2020 年提出，是首个在图像分类任务上媲美甚至超越 CNN（ResNet）的纯 Transformer 架构。其核心思想是：将图像分割为固定大小的 Patch，将每个 Patch 视为一个“Token”，从而将图像分类问题完全转化为 Transformer 的标准序列分类问题。

3.3.5.1.1 ViT 的完整处理流程

1. Patch 分割与线性投影：

- 输入图像 $\mathbf{X} \in \mathbb{R}^{H \times W \times C}$ ，分割为 N 个 Patch，每个 Patch 大小为 $P \times P$ （通常 $P = 16$ ）。
- 每个 Patch 展平为 $P^2 \cdot C$ 维向量，通过线性投影（可训练的嵌入层）映射为 D 维嵌入向量：

$$\mathbf{x}_i = \mathbf{E} \cdot \text{Flatten}(\text{Patch}_i) + \mathbf{p}_i, \quad i = 1, 2, \dots, N,$$

其中 $\mathbf{E} \in \mathbb{R}^{D \times (P^2 \cdot C)}$ 为可学习的 Patch 嵌入矩阵， $\mathbf{p}_i$ 为可学习的位置编码。

2. [CLS] Token 的引入：

- 在序列开头添加一个特殊的可学习 [CLS] Token，其最终输出状态作为整张图像的全局表示，送入分类头。
- 这一设计借鉴了 BERT 的做法，避免了对所有 Patch 输出做平均池化可能带来的信息损失。

3. Transformer 编码器堆叠：

- 将 $N + 1$ 个 Token 输入标准 Transformer 编码器（多头自注意力 + 前馈网络 + 残差连接 + LayerNorm）。
- 编码器层数 L 通常为 12（Base）或 24（Large）。

4. 分类输出：

- 取 [CLS] Token 的最终输出状态，通过一个线性分类层得到类别预测。

表 3.9: ViT 相对 CNN 的优劣对比

维度	ViT	CNN (ResNet)
全局感受野	自注意力一步覆盖全图	需堆叠多层才能扩大感受野
归纳偏置 数据效率	极少（需大规模数据学习） 低（需亿级图像预训练）	强（平移不变性 + 局部性） 高（可在百万级数据集上训练）
计算复杂度	$\mathcal{O}(N^2)$ (N 为 Patch 数)	$\mathcal{O}(HW)$ （与图像尺寸线性相关）
可解释性	注意力权重可视化	特征图可解释性较弱

3.3.5.1.2 ViT 的核心优势与局限

3.3.5.1.3 ViT 的变体与演进 ViT 的成功催生了大量视觉 Transformer 变体：

- **DeiT (Data-efficient Image Transformers)**: 通过知识蒸馏从 CNN 教师模型中学习，显著降低了对预训练数据量的需求（仅需 ImageNet-1K 即可训练）。
- **MAE (Masked Autoencoders)**: 采用类 BERT 的掩蔽自编码策略进行自监督预训练，大幅提升 ViT 的数据效率。

3.3.5.2 DETR: 目标检测的端到端革命

DETR (Detection Transformer) 由 Carion 等人在 2020 年提出，是首个将目标检测重构为集合预测 (**Set Prediction**) 问题的端到端架构。它彻底摒弃了锚框 (Anchors)、非极大值抑制 (NMS) 等传统检测器的繁杂组件，实现了真正的“输入图像 $\rightarrow$ 输出边界框集合”的端到端映射。

3.3.5.2.1 DETR 的核心架构 DETR 的架构由三个核心组件构成：

1. CNN 骨干网络 (Backbone):

- 使用标准 CNN（如 ResNet-50）提取图像的二维特征图。
- 特征图经过 1×1 卷积降维后，展平为序列输入 Transformer。

2. Transformer 编码器-解码器:

- 编码器对图像特征序列进行自注意力编码，捕获全局上下文信息。
- 解码器接收 N 个可学习的对象查询 (**Object Queries**)，每个查询负责预测一个目标对象。
- N 为固定常数（通常 $N = 100$ ），远大于图像中可能出现的最大目标数。

3. 预测头 (Prediction Heads):

- 每个对象查询对应的解码器输出通过前馈网络预测：
 - 类别概率（包含 $\emptyset$ “无对象”类）
 - 边界框坐标（归一化的中心点 + 宽高）

3.3.5.2.2 集合预测与匈牙利匹配损失 DETR 最核心的创新在于其训练时的集合匹配策略。由于模型输出的是无序集合，无法直接与标注框计算损失。DETR 使用匈牙利算法 (**Hungarian Algorithm**) 在预测框与真实框之间进行二分图最优匹配：

$$\hat{\sigma} = \arg \min_{\sigma \in \mathbb{S}_N} \sum_{i=1}^N \mathcal{L}_{\text{match}}(y_i, \hat{y}_{\sigma(i)}),$$

其中匹配损失 $\mathcal{L}_{\text{match}}$ 综合考虑了类别预测和边界框的 L1 损失与 GIOU 损失。匹配完成后，再计算最终的训练损失：

$$\mathcal{L}_{\text{DETR}} = \sum_{i=1}^N \left[\mathcal{L}_{\text{cls}}(\hat{p}_{\hat{\sigma}(i)}, c_i) + \mathbf{1}_{\{c_i \neq \emptyset\}} \mathcal{L}_{\text{box}}(b_i, \hat{b}_{\hat{\sigma}(i)}) \right].$$

定义 3.17 (GIOU 损失 (Generalized IoU Loss))

GIOU 在标准 IoU 的基础上引入了一个惩罚项，用于衡量两个边界框之间的“最小包围框”面积，从而即使两个框完全不重叠时也能提供有效的梯度信息：

$$\text{GIOU} = \text{IoU} - \frac{|C \setminus (A \cup B)|}{|C|},$$

其中 C 为包含 A 与 B 的最小凸包围框。



3.3.5.2.3 DETR 的局限与改进

- 收敛速度慢：DETR 需要 500 个 Epoch 才能收敛，远慢于传统检测器（通常 12-36 Epoch）。
- 小目标检测性能不佳：Transformer 的自注意力在高分辨率特征图上计算量过大，限制了输入分辨率。

后续改进工作 **Deformable DETR** (Zhu et al., 2021) 通过引入可变形注意力 (Deformable Attention)，使每个查询只关注特征图中的稀疏采样点，将收敛速度提升 10 倍，并显著改善了小目标检测性能。

3.3.5.3 Swin Transformer：层次化视觉 Transformer

Swin Transformer (Shifted Window Transformer) 由 Liu 等人在 2021 年提出，是首个可作为通用视觉骨干网络 (Backbone) 的 Transformer 架构，其性能在分类、检测、分割三大任务上均超越同期 CNN 和 ViT。

3.3.5.3.1 核心创新：移动窗口注意力 Swin Transformer 的核心设计目标是解决 ViT 的两个根本问题：

- 自注意力的计算复杂度 $\mathcal{O}(N^2)$ 随图像尺寸指数增长。
- ViT 缺乏跨尺度的层次化特征，难以适配检测、分割等需要多尺度表征的任务。

窗口注意力 (Window Attention)：

- 将图像特征图均匀划分为不重叠的局部窗口（如 7×7 大小），在每个窗口内部独立计算自注意力。
- 时间复杂度从 $\mathcal{O}(N^2)$ 降至 $\mathcal{O}(N \cdot k^2)$ ，其中 k 为窗口大小（常数），计算量随图像尺寸线性增长。

移动窗口 (Shifted Window)：

- 连续两层之间，窗口划分进行偏移（向两个方向各偏移半个窗口大小）。
- 这使得上一层的“窗口边界”在下一层变为“窗口内部”，实现了跨窗口的信息交换，弥补了局部注意力感受野受限的问题。

3.3.5.3.2 层次化特征金字塔 Swin Transformer 通过 Patch 合并 (Patch Merging) 逐层构建特征金字塔：

- **Stage 1:** Patch 大小为 4×4 ，输出下采样 4 倍的特征图。
- **Stage 2:** 通过 Patch Merging（将 2×2 邻域的 Patch 合并为 1 个），输出下采样 8 倍的特征图。
- **Stage 3:** 输出下采样 16 倍的特征图。
- **Stage 4:** 输出下采样 32 倍的特征图。

这一层次化设计使得 Swin Transformer 可以直接适配 FPN（特征金字塔网络）等现有检测/分割框架，无需额外修改。

表 3.10: ViT、DETR 与 Swin Transformer 的核心差异

模型	核心创新	注意力机制	主要应用
ViT	Patch 嵌入 + [CLS] Token	全局自注意力	图像分类
DETR	对象查询 + 匈牙利匹配	编码器-解码器注意力	目标检测
Swin Transformer	移动窗口注意力 + 层次化设计	局部窗口注意力	通用视觉骨干

3.3.5.4 DiT：扩散模型中的 Transformer

DiT (Diffusion Transformer) 由 Peebles & Xie 于 2023 年提出，首次将 Transformer 架构引入扩散模型，取代了传统扩散模型中常用的 U-Net 骨干网络。DiT 的提出标志着 Transformer 在图像生成领域正式接管了主导地位。

3.3.5.4.1 DiT 的架构设计 DiT 的核心思想是将扩散模型的输入——带噪图像 x_t 、时间步 t 以及可选的类别/文本条件——全部通过 Transformer 编码器进行处理，而非传统的卷积 U-Net。

1. 输入处理:
- 将带噪图像 x_t 通过 Patch Embedding 转换为 Token 序列（与 ViT 类似）。
 - 时间步 t 通过时间步嵌入层（Sinusoidal Embedding + MLP）转换为条件向量。
 - 类别标签或文本条件通过独立的嵌入层编码。
2. 条件注入方式:
- **AdaLN (Adaptive Layer Normalization)**: 使用条件向量动态生成 LayerNorm 的缩放参数 γ 和偏移参数 β 。这是 DiT 的核心创新，实现了条件信息与 Transformer 层的精细融合。

$$\text{AdaLN}(h, c) = \gamma(c) \cdot \text{LayerNorm}(h) + \beta(c),$$

其中 $\gamma(c)$ 与 $\beta(c)$ 由条件向量 c 通过 MLP 预测得到。

3. 输出预测目标:
- DiT 预测扩散模型中添加的噪声 $\epsilon_\theta(x_t, t, c)$ ，与 U-Net 扩散模型的训练目标一致。
 - 输出张量通过线性层从 Token 序列映射回原始图像尺寸。

3.3.5.4.2 DiT 的可扩展性 (Scaling Property) DiT 最重要的发现是：****Transformer 架构在扩散模型上同样展现出如 GPT 般的规模扩展性 (Scaling Law)****。实验表明，增加 DiT 的模型参数量（从 0.3B 到 3B）和训练计算量（从 1M 到 400M 图像），生成图像的 FID 指标持续改善，且没有饱和迹象。

这一发现直接推动了 Sora 等视频生成模型的出现，标志着视觉生成正式进入“Transformer + 扩散”的双引擎时代。

定义 3.18 (扩散模型的训练目标)

给定数据 x_0 ，扩散过程在 T 步内逐步添加噪声，得到 x_t 。模型 ϵ_θ 被训练来预测添加的噪声：

$$\mathcal{L} = \mathbb{E}_{t, x_0, \epsilon \sim \mathcal{N}(0, I)} [\|\epsilon - \epsilon_\theta(x_t, t, c)\|^2],$$

其中 $x_t = \sqrt{\alpha_t}x_0 + \sqrt{1 - \alpha_t}\epsilon$ 。



3.4 多模态大模型

多模态大模型是当前人工智能领域的前沿方向，旨在打通文本、图像、视频、音频等多种模态之间的壁垒，实现跨模态的语义理解与生成。本节聚焦 CLIP、BLIP 和 BLIP2 三个具有里程碑意义的模型，梳理多模态预训练技术的演进脉络。

3.4.0.1 CLIP: 连接文本与图像的桥梁

CLIP (Contrastive Language-Image Pre-training) 由 OpenAI 于 2021 年提出, 是首个在零样本 (Zero-Shot) 图像分类任务上达到 SOTA 性能的视觉-语言模型。其核心思想是通过对比学习在 4 亿图文对数据集上进行预训练, 将图像和文本映射到共享的语义空间中。

3.4.0.1.1 CLIP 的预训练机制 CLIP 使用了一个简单的对比学习目标:

1. **数据格式:** 每个训练样本由一张图像 $\mathbf{I}_i$ 及其对应的文本描述 $\mathbf{T}_i$ 组成。
2. **双编码器架构:**
 - 图像编码器: ViT 或 ResNet, 将图像编码为固定维度的向量 $\mathbf{v}_i$ 。
 - 文本编码器: Transformer, 将文本编码为固定维度的向量 $\mathbf{u}_i$ 。
 - 两个编码器输出投影到同一个 d 维空间 (通常 $d = 768$ 或 1024)。
3. **对比损失函数:**
 - 在 Batch 中, 对于每一对正确的图文组合 $(\mathbf{I}_i, \mathbf{T}_i)$ 作为正样本, 其余 $B - 1$ 对作为负样本。
 - 使用对称的 InfoNCE 损失 (即同时优化图像到文本和文本到图像的匹配):

$$\mathcal{L} = -\frac{1}{2B} \sum_{i=1}^B \left[\log \frac{\exp(\text{sim}(\mathbf{v}_i, \mathbf{u}_i)/\tau)}{\sum_{j=1}^B \exp(\text{sim}(\mathbf{v}_i, \mathbf{u}_j)/\tau)} + \log \frac{\exp(\text{sim}(\mathbf{u}_i, \mathbf{v}_i)/\tau)}{\sum_{j=1}^B \exp(\text{sim}(\mathbf{u}_i, \mathbf{v}_j)/\tau)} \right],$$

其中 $\text{sim}(\cdot, \cdot)$ 为余弦相似度, τ 为可学习的温度参数。

3.4.0.1.2 CLIP 的零样本迁移能力 预训练完成后, CLIP 可以零样本本地迁移到任意图像分类任务:

1. 将待分类的类别标签转换为自然语言描述 (如 “一张狗的照片”、“一张猫的照片”)。
2. 将这些文本描述通过文本编码器编码为文本特征。
3. 将待分类图像通过图像编码器编码为图像特征。
4. 计算图像特征与所有文本特征的相似度, 得分最高的类别即为预测结果。

CLIP 在 ImageNet 零样本分类上达到 76.2% 的准确率, 媲美当时全监督 ResNet-50 的性能。更重要的是, CLIP 的零样本能力具备极强的泛化性, 能在分布外 (Out-of-Distribution) 数据集上保持稳定性能。

3.4.0.2 BLIP: 统一理解与生成的视觉-语言模型

BLIP (Bootstrapping Language-Image Pre-training) 由 Li 等人在 2022 年提出, 是首个统一了视觉-语言理解和视觉-语言生成的预训练模型。其核心贡献在于提出了 MED (Multimodal Mixture of Encoder-Decoder) 架构和一个基于生成器的数据自举 (CapFilter) 方法。

3.4.0.2.1 MED 架构: 多模态编码器-解码器混合体 MED 是一个灵活的多模态架构, 可以根据任务需求切换为三种不同的子架构:

1. **双编码器 (Dual Encoder):**
 - 独立的图像编码器与文本编码器, 仅通过对比学习进行交互。
 - 适用于检索任务, 计算效率高 (可预先计算所有图文特征)。
2. **融合编码器 (Fusion Encoder):**
 - 在编码器中通过自注意力与交叉注意力实现图像与文本的深度融合。
 - 适用于理解任务 (如 VQA、视觉推理)。
3. **编码器-解码器 (Encoder-Decoder):**
 - 编码器接收图像与文本前缀, 解码器自回归生成文本。
 - 适用于生成任务 (如图像描述、视觉问答中的答案生成)。

3.4.0.2.2 预训练任务三合一 BLIP 同时优化三个预训练目标，分别对应上述三种架构：

1. 图像-文本对比损失 (ITC)：与 CLIP 类似，对齐图像与文本的全局表示。
2. 图像-文本匹配损失 (ITM)：二分类任务，判断图像与文本是否匹配，用于融合编码器的训练。
3. 语言建模损失 (LM)：自回归生成任务，训练编码器-解码器进行文本生成。

例题 3.3 BLIP 的 CapFilter 自举流程 1. 使用预训练 BLIP 模型为网络上的噪声图文对生成多个候选描述 (Captioning)。2. 使用 BLIP 的匹配模块 (ITM) 筛选高质量描述 (Filtering)。3. 将筛选后的高质量图文对作为新的训练数据，用于下一轮训练。

3.4.0.2.3 CapFilter：数据自举与质量过滤 BLIP 发现网络上的图文对通常包含大量噪声（如图像与文本不匹配或描述过于简略）。为此，BLIP 提出了 CapFilter 方法：

1. 对每个图像，使用 BLIP 的生成模块生成多个候选描述。
2. 使用 BLIP 的匹配模块对候选描述进行评分，保留得分最高的描述。
3. 将原始噪声数据替换为高质量的自举数据，显著提升训练效果。

3.4.0.3 BLIP-2：高效的多模态预训练范式

BLIP-2 (Li et al., 2023) 在 BLIP 的基础上进一步突破，提出了轻量级查询 Transformer (Q-Former) 架构，实现了高效的多模态预训练。其核心思想是：预训练好的大语言模型 (LLM) 和图像编码器，只需训练一个轻量级的 Q-Former 作为“桥梁”，即可完成对齐。

3.4.0.3.1 Q-Former：查询驱动的信息提取 Q-Former 是一个轻量级 Transformer（参数量约 100M），其核心设计包含：

1. 可学习的查询 (Learnable Queries)：
 - Q-Former 包含 N 个可学习的查询向量 (N 通常取 32 或 64)，作为图像信息的“读取头”。
 - 这些查询通过交叉注意力与图像特征交互，提取与文本生成最相关的视觉信息。
2. 两阶段预训练：
 - (a). 阶段一：视觉-语言表示学习：冻结图像编码器，训练 Q-Former 对齐图像与文本表示（使用 ITC、ITM、LM 三种损失）。
 - (b). 阶段二：视觉到语言生成学习：将 Q-Former 输出的查询特征通过线性投影注入冻结的 LLM，训练 Q-Former 生成适配 LLM 的视觉特征。

定义 3.19 (Q-Former 的查询-图像交叉注意力)

给定可学习查询 $Q \in \mathbb{R}^{N \times d}$ 与图像特征 $F \in \mathbb{R}^{M \times d}$ ，Q-Former 执行：

$$\text{CrossAttn}(Q, F) = \text{softmax} \left(\frac{QW_Q(FW_K)^T}{\sqrt{d}} \right) (FW_V).$$

每个查询向量独立地提取图像中的不同“视觉概念”（如物体、场景、纹理等）。



3.4.0.3.2 BLIP-2 的效率和性能优势 BLIP-2 的核心优势在于其效率：

- 计算效率：仅训练 Q-Former（约 100M 参数），冻结超大规模的图像编码器（如 ViT-G, 1.8B）和 LLM（如 OPT-30B 或 Flan-T5-XXL）。总训练计算量仅为 BLIP 的 30%，但性能显著超越。
- 灵活组合：Q-Former 学到的图像表示可以适配任意 LLM，无需重新训练，极大提升了模型的复用性。

BLIP-2 在零样本 VQA、图像描述、图像-文本检索等任务上达到 SOTA，为后续多模态大模型的“冻结大模型 + 轻量适配器”范式奠定了基础。

多模态模型的演进趋势总结

从 CLIP 到 BLIP 再到 BLIP-2，多模态预训练经历了三代演进：

1. 第一代（CLIP）：双塔独立编码 + 对比对齐，实现模态间的表示对齐。
2. 第二代（BLIP）：融合编码-解码架构 + 统一预训练目标，同时实现理解与生成。
3. 第三代（BLIP-2）：轻量适配器（Q-Former）+ 冻结大模型，大幅降低训练成本并提升灵活性。

 **练习 3.1 多模态模型概念检测** 判断以下说法的正误，并简要说明理由：

1. CLIP 的预训练目标是让图像编码器和文本编码器生成相似的表示向量。
2. DETR 使用锚框（Anchors）和非极大值抑制（NMS）来筛选检测结果。
3. Swin Transformer 通过移动窗口（Shifted Window）机制实现跨窗口的信息交换。
4. BLIP-2 使用 Q-Former 作为可训练的适配器，并将大语言模型（LLM）冻结。

3.4.1 基于大模型的智能体系统

大语言模型（LLM）的快速发展不仅重塑了自然语言处理本身，更催生了一个全新的研究领域——** 基于大模型的智能体系统（LLM-based Agent Systems）**。这类系统将大模型视为“大脑”，赋予其感知环境、规划决策、使用工具和反思学习的能力，从而将大模型从“聊天工具”进化为“自主行动者”。

3.4.1.1 智能体的定义与核心架构

定义 3.20 (LLM 智能体)

一个基于大语言模型的智能体是一个能够在环境中自主感知、推理、规划并执行行动的计算实体。其核心特征在于：以大模型为推理引擎，通过调用外部工具（Tools）或与物理/数字环境交互来完成任务。

一个完整的 LLM 智能体系统通常包含以下四个核心模块：

1. **大语言模型（LLM）**：
 - 作为智能体的“核心大脑”，负责理解任务、进行推理、制定计划。
 - 通常使用 GPT-4、Claude 3、LLaMA 3 等高性能基础模型，并通过系统提示词（System Prompt）定义其角色与行为准则。
2. **工具集（Tools）**：
 - 扩展智能体能力的外部功能模块，如搜索引擎、计算器、代码解释器、数据库查询接口、物理机器人控制接口等。
 - 工具通过 API 调用或函数调用（Function Calling）机制与 LLM 交互。
3. **记忆模块（Memory）**：
 - **短期记忆**：当前对话轮次内的上下文信息，由 LLM 的上下文窗口承载。
 - **长期记忆**：跨会话的历史经验、知识库或向量数据库，通过检索增强（RAG）机制进行读写。
4. **规划模块（Planning）**：
 - 将复杂任务分解为可执行的子任务序列。
 - 支持动态重规划（Replanning），在执行中根据反馈调整策略。

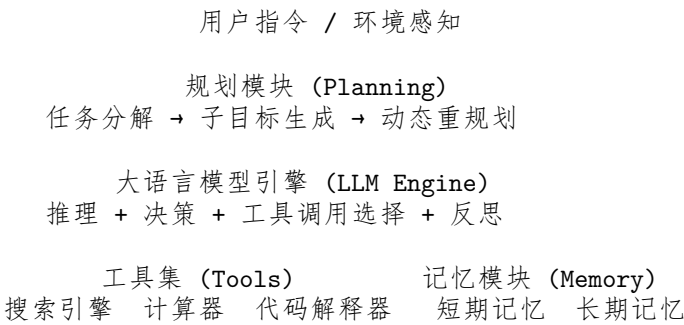
3.4.1.2 智能体的核心推理范式

3.4.1.2.1 CoT：思维链推理 思维链（Chain-of-Thought, CoT）由 Wei 等人在 2022 年提出，是智能体推理能力的基石。其核心思想是：在给出最终答案之前，引导模型逐步展示推理过程，而非直接输出结果。

例题 3.4CoT 提示示例

Q：张三有 5 个苹果，他给了李四 2 个，然后又买了 3 个，他现在有几个苹果？

图 3.8: LLM 智能体的四层架构示意图



A: 让我一步步想: 张三一开始有 5 个苹果。
给了李四 2 个后: $5 - 2 = 3$ 个。
又买了 3 个后: $3 + 3 = 6$ 个。
所以张三现在有 6 个苹果。

CoT 的有效性揭示了 LLM 在“推理时计算”(Inference-time Computation) 上的巨大潜力: 通过显式地要求模型“慢思考”, 可以显著提升其在数学、逻辑和常识推理任务上的表现。

3.4.1.2.2 ReAct: 推理与行动的交织 ReAct (Reasoning + Acting) 由 Yao 等人在 2022 年提出, 是智能体系统中最具影响力的范式之一。其核心创新在于: ** 将模型的推理过程 (Reasoning) 与实际行动 (Acting) 交替进行 **, 形成一个可观察、可干预的循环。

定义 3.21 (ReAct 循环)

ReAct 代理在一个迭代循环中执行以下步骤:

1. 思考 (Thought): LLM 分析当前状态, 规划下一步行动。
2. 行动 (Act): 执行一个外部动作 (如调用搜索引擎、执行代码)。
3. 观察 (Observation): 从环境中获得行动结果反馈。
4. 回到步骤 1, 直至任务完成或达到终止条件。

ReAct 与纯 CoT 的核心区别在于, ReAct 允许模型通过“行动”与外部世界交互, 将封闭的“内心独白”转变为开放的“与环境对话”。

例题 3.5 ReAct 轨迹示例

问题: 请告诉我 2024 年夏季奥林匹克运动会的举办城市。

Thought 1: 我需要搜索 2024 年奥运会的信息。

Act 1: 调用搜索引擎, 搜索 "2024 Summer Olympics host city"

Observation 1: 搜索结果显示 2024 年奥运会由法国巴黎举办。

Thought 2: 我已获得答案。

Act 2: 输出 "巴黎, 法国"

3.4.1.2.3 ToT: 思维树 思维树 (Tree of Thoughts, ToT) 由 Yao 等人在 2023 年提出, 将 CoT 的单路径推理扩展为树状搜索。其核心思想是: 在每一步推理中维护多个可能的“思维候选”, 通过评估函数进行剪枝和回溯, 以系统化的方式探索推理空间。

1. **思维分解 (Thought Decomposition)**: 将复杂任务分解为若干推理步骤 (中间思维节点)。
2. **思维生成 (Thought Generation)**: 在每个节点上, 通过 LLM 生成 k 个候选思维 (如 “提出 k 种可能的下一步方案”)。
3. **状态评估 (State Evaluation)**: 使用启发式函数或 LLM 自评估对每个候选状态打分。
4. **搜索算法 (Search Algorithm)**: 使用 BFS 或 DFS 进行树搜索, 对高分分支深入探索, 对低分分支提前剪枝。

ToT 在 24 点游戏、创意写作、跨词谜题等需要探索性推理的任务上显著超越了 CoT。

3.4.1.3 代表性智能体系统

3.4.1.3.1 AutoGPT: 自主任务执行的开源先锋 AutoGPT 是首个引发广泛关注的自主智能体系统。其核心理念是: 用户只需提供高层目标, 智能体自主地进行任务分解、执行和迭代。

核心机制:

- **自我追问**: AutoGPT 会不断问自己 “我还需要做什么才能完成目标?”, 并据此生成新的子任务。
- **工具集成**: 原生集成网络搜索、文件读写、代码执行等能力。
- **长期记忆**: 使用 Pinecone 等向量数据库存储历史推理和执行记录。
- **持续循环**: 系统在预设的迭代次数内持续执行 “思考-行动-观察” 循环, 直至任务完成。

例题 3.6 AutoGPT 的典型工作流程 用户输入目标: “研究 2024 年 AI 芯片市场趋势, 并生成一份报告”

1. AutoGPT 将目标分解为子任务: 搜索市场数据 → 分析竞争格局 → 生成报告大纲 → 撰写报告。
2. 调用搜索引擎获取市场数据。
3. 调用代码解释器进行数据分析。
4. 调用文件写入工具生成 Markdown 报告。
5. 自主评估报告完整性, 决定是否补充内容。

3.4.1.3.2 MetaGPT: 多智能体协作的软件开发框架 MetaGPT 由 Hong 等人在 2023 年提出, 是一个将多智能体协作应用于软件开发的全新范式。其核心创新是: 将软件开发的流水线 (需求分析 → 设计 → 编码 → 测试) 映射为多个角色的智能体 (产品经理、架构师、工程师、测试员) 之间的协作流程。

核心机制:

- **角色专用化**: 为每个智能体分配特定角色, 使用不同的系统提示词定义其职责和行为模式。
- **结构化输出**: 强制要求智能体输出标准化的中间产物 (如需求文档、API 设计文档、代码文件), 确保协作的规范性。
- **共享工作空间**: 所有智能体通过共享的文档库和工作流进行异步协作。

例题 3.7 MetaGPT 的软件开发生命周期

1. **产品经理代理**: 将用户需求转化为标准化的 PRD (产品需求文档)。
2. **架构师代理**: 根据 PRD 设计系统架构和 API 接口。
3. **工程师代理**: 根据架构设计编写代码。
4. **测试员代理**: 执行代码测试并报告缺陷。
5. **项目经理代理**: 监控整体进度, 协调各角色间的工作交接。

MetaGPT 证明了多智能体协作系统能够在复杂工程任务中产生 “1 + 1 > 2” 的涌现效应, 为未来智能体系统的规模化协作提供了重要参考。

3.4.1.3.3 Voyager: 游戏中涌现的智能体 Voyager 由 Wang 等人在 2023 年提出, 是首个在开放世界游戏 *Minecraft* 中实现持续探索与技能学习的 LLM 智能体。

核心创新:

- **自动课程 (Automatic Curriculum):** 智能体根据当前状态自主提出“下一个最合适的挑战”，形成渐进式的学习路径。
- **技能库 (Skill Library):** 成功完成的任务被封装为可复用的技能代码，存入向量数据库以备后续检索调用。
- **迭代提示 (Iterative Prompting):** 任务执行失败时，LLM 通过分析错误反馈修改代码，实现自我纠错。

Voyager 在 *Minecraft* 中进行了 160 小时的持续探索，解锁了 63 项独特技能（从采集木材到建造钻石工具），展现了基于 LLM 的智能体在开放世界中持续学习和自适应进化的能力。

3.4.1.4 智能体系统的关键能力维度

表 3.11: LLM 智能体系统的核心能力维度

能力维度	关键机制	代表性技术/系统
推理与规划	任务分解 + 动态重规划	CoT、ReAct、ToT、LLM+P
工具使用	函数调用 + API 集成	Toolformer、Gorilla、AutoGPT
多步回溯	错误检测 + 策略修正	Self-Refine、ReAct 的迭代循环
长期记忆	向量检索 + 经验重放	RAG、MemGPT、Voyager 技能库
多智能体协作	角色分配 + 通信协议	MetaGPT、ChatDev、SWE-Agent
环境交互	状态感知 + 行动执行	Voyager、WebArena、Minigrid

3.4.1.5 智能体系统的安全与对齐挑战

随着智能体获得越来越强的自主行动能力，其安全性和可控性成为关键挑战：

1. **行为可预测性:** 智能体可能在长序列推理中产生不可预见的行为链，偏离设计者的意图。
2. **工具误用风险:** 智能体可能以非预期的方式调用工具（如调用代码解释器执行危险指令）。
3. **长期目标漂移:** 在持续迭代中，智能体可能逐渐偏离原始目标。

当前应对策略：

- **沙箱执行:** 将智能体的工具调用限制在隔离的沙箱环境中，防止对外部系统造成实际影响。
- **人类介入 (Human-in-the-loop):** 关键行动前需经人类确认，如 AutoGPT 的“连续执行次数限制”设计。
- **可解释性工具:** 记录并可视化智能体的完整推理轨迹，便于事后审查和错误溯源。

3.4.1.6 智能体系统的未来趋势

1. **多模态感知与行动:** 智能体将从纯文本交互扩展为感知视觉、语音、触觉等多模态信息，并驱动物理世界中的机器人执行行动。
2. **自反思与元认知:** 智能体将具备对自身推理过程进行监控和评估的能力（Self-Reflection），实现持续的自我改进。
3. **大规模多智能体协同:** 从少数智能体的协作扩展到“智能体社会”，模拟复杂的社会经济系统，或用于大规模任务的并行分解与执行。
4. **与物理世界的融合:** 通过具身智能（Embodied AI）将 LLM 智能体部署到机器人、自动驾驶等物理系统中，实现“思考-感知-行动”的完整闭环。

 **练习 3.2 智能体系统概念检测** 判断以下说法的正误，并简要说明理由：

1. ReAct 范式要求智能体在推理和行动之间交替进行。
2. CoT（思维链）与 ReAct 的核心区别在于 CoT 允许智能体与外部环境交互。
3. AutoGPT 是一种多智能体协作框架，专为软件开发任务设计。
4. Voyager 在 *Minecraft* 中通过“自动课程”机制实现了持续的技能学习与进化。
5. ToT（思维树）将 CoT 的单路径推理扩展为树状搜索，通过评估函数进行剪枝和回溯。

3.5 神经网络量化

随着深度学习模型在参数量和计算复杂度上的指数级增长，将大模型部署到资源受限的边缘设备（如手机、嵌入式设备、IoT 传感器）面临严峻挑战。神经网络量化（Neural Network Quantization）作为一种有效的模型压缩技术，通过将模型权重和激活值从高精度浮点数（通常为 32 位）转换为低精度整数（如 8 位、4 位甚至 1 位），能够显著减少模型存储空间、降低内存带宽需求并加速推理计算，同时保持可接受的精度损失。

3.5.1 量化的动机与基本原理

3.5.1.1 为什么需要量化？

以经典的 AlexNet 为例，其原始 FP32 权重文件超过 200 MB，而现代大语言模型（如 GPT-3）的参数量已达 175B，存储需求超过 700 GB。这种规模使得模型在边缘设备上的部署几乎不可能。

量化的核心动机在于以下三重收益：

- 存储压缩：**将 FP32（32 位）转换为 INT8（8 位），模型大小缩减为原来的 1/4。极端情况下，二值化网络（1 位）可实现 32× 的压缩比 [citation:1]。
- 计算加速：**低精度整数运算比浮点运算快 2-4 倍，且可充分利用现代硬件中的专用计算单元（如 NVIDIA GPU 的 Tensor Cores） [citation:3]。
- 能耗降低：**数据从内存传输到计算单元的过程消耗能量，降低精度可使数据量减半，有效降低功耗 [citation:3][citation:4]。

表 3.12: 不同精度格式的存储与计算特性对比

精度格式	位宽	存储压缩比（相对 FP32）	典型应用场景
FP32	32 位	1.0×（基准）	训练、高精度推理
FP16/BF16	16 位	2×	混合精度训练
INT8	8 位	4×	边缘推理、移动端部署
INT4	4 位	8×	大语言模型推理
Binary (1-bit)	1 位	32×	极端资源受限场景

3.5.1.2 量化的数学本质

量化本质上是一种从连续值空间（浮点数）到离散值空间（整数）的映射。以最常见的线性量化为例，浮点实数 r 与量化整数 q 之间的转换关系为 [citation:5]：

$$r = S \cdot (q - Z),$$

其中：

- S （Scale）为缩放因子，表示实数与整数之间的比例关系；
- Z （Zero Point）为零点，表示实数 0 量化后对应的整数。

Scale 和 Zero Point 由浮点数据的动态范围 $[r_{\min}, r_{\max}]$ 和目标整数范围 $[q_{\min}, q_{\max}]$ 共同确定 [citation:5]：

$$S = \frac{r_{\max} - r_{\min}}{q_{\max} - q_{\min}}, \quad Z = q_{\min} - \frac{r_{\min}}{S}.$$

对于 INT8 量化， $q_{\min} = -128$ ， $q_{\max} = 127$ （有符号）或 $q_{\min} = 0$ ， $q_{\max} = 255$ （无符号）。

3.5.2 量化方案的分类体系

根据不同的设计维度，量化方案可按以下方式分类 [citation:2]:

3.5.2.1 均匀量化 vs. 非均匀量化

- **均匀量化 (Uniform Quantization)**: 量化后的整数在数轴上等距分布。上述线性量化即属于均匀量化，是工业界最主流的方案，因其硬件实现简单、计算效率高 [citation:5]。
- **非均匀量化 (Non-uniform Quantization)**: 量化步长随数值区间变化，通常在对数域进行。对数量化 (Log Quantization) 能更好地适应权重的长尾分布，但硬件实现复杂度较高 [citation:1]。

3.5.2.2 对称量化 vs. 非对称量化

- **对称量化 (Symmetric Quantization)**: 零点固定为 0 (即 $Z = 0$)，浮点数据的正负范围对称映射到整数范围。适用于权重分布关于 0 对称的场景，计算更为高效。
- **非对称量化 (Asymmetric Quantization)**: 零点可根据数据分布灵活调整 ($Z \neq 0$)，能够更好地适配激活值等非对称分布的数据，减少量化误差。TensorFlow Lite 等框架默认采用非对称量化 [citation:4][citation:3]。

3.5.2.3 逐层量化 vs. 逐通道量化

- **逐层量化 (Per-layer Quantization)**: 同一层内所有通道共享相同的 Scale 和 Zero Point。实现简单，但对包含异常值的通道会产生较大误差。
- **逐通道量化 (Per-channel Quantization)**: 为卷积层的每个输出通道独立计算量化参数。实验表明，逐通道量化可将量化误差减半，是卷积层的推荐默认设置 [citation:3][citation:2]。

3.5.3 量化方法体系: PTQ 与 QAT

根据量化介入训练流程的时机，业界主流方法分为两大范式: ** 训练后量化 (Post-training Quantization, PTQ) ** 与 ** 量化感知训练 (Quantization-Aware Training, QAT) ** [citation:3][citation:7][citation:12]。

3.5.3.1 训练后量化 (PTQ)

PTQ 在模型完成全精度训练后介入，无需重新训练或仅需少量校准数据即可完成量化转换 [citation:3]。

PTQ 的工作流程:

1. 使用校准数据集 (通常为 100-1000 张代表性样本) 对模型进行前向推理。
2. 收集各层权重和激活值的统计信息 (最小值、最大值、百分位数等)。
3. 根据统计信息计算量化参数 (Scale 和 Zero Point)。
4. 将模型权重从 FP32 转换为 INT8，并对激活值进行量化处理。

PTQ 的两种实现方式 [citation:3]:

- **动态 PTQ**: 在推理过程中根据实时输入数据分布动态调整激活值的量化范围。无需校准数据，但每次推理都需额外计算量化参数。
- **静态 PTQ**: 引入额外的校准步骤，使用代表性数据集预先估计激活值的量化范围。部署后量化参数固定，推理效率更高。

优点: 资源效率高、部署快速，适用于无法重新训练的场景。**局限:** 精度损失相对较大 (尤其在低比特条件下)，需精心校准 [citation:3][citation:7]。

例题 3.8PTQ 的硬件加速效果 在 PYNQ-Z1 FPGA 上对 ResNet50 进行 INT8 量化部署的实验结果表明:

- 功耗-延迟积 (PDP) 降低 94%;
- LUT 效率提升 3100%;

- DSP 利用率增益超过 420% [citation:4]。

这些数据充分证明了 PTQ 在资源受限场景下的巨大价值。

3.5.3.2 量化感知训练 (QAT)

QAT 在模型训练过程中模拟量化效应，使模型在低精度环境下“学会”如何保持高性能。其核心机制是在前向传播中引入伪量化 (Fake Quantization) 操作，而反向传播仍使用高精度梯度 [citation:3][citation:10]。

QAT 的工作流程 [citation:10]:

1. 以一个训练完毕的 FP32 浮点模型为起点。
2. 在网络中插入假量化算子 (FakeQuantize)，在 FP32 精度下模拟 INT8 的截断和舍入效果。
3. 继续训练模型 (通常为原始训练时间的 5-10%)，使权重调整以补偿量化引入的噪声。
4. 训练完成后，将伪量化模块替换为实际量化算子，导出可直接部署的 INT8 模型。

直通估计器 (Straight-Through Estimator, STE): 量化操作中的舍入函数 $\text{round}(\cdot)$ 在反向传播时梯度为 0 (或未定义)。QAT 通过 STE 技术解决这一问题——在反向传播时将舍入操作视为恒等函数 ($y = x$)，从而使梯度能够正常回传至前层权重。

$$\text{Forward: } \hat{x} = \text{round}(x/S) \cdot S, \quad \text{Backward: } \frac{\partial \mathcal{L}}{\partial x} \approx \frac{\partial \mathcal{L}}{\partial \hat{x}}.$$

优点: 精度损失最小 (可接近全精度模型)，尤其适用于对量化误差敏感的轻量级架构。**局限**: 需要额外计算资源和训练时间，实现复杂度较高 [citation:10][citation:3]。

例题 3.9 QAT 与 PTQ 的精度恢复效果对比 在 Llama Nemotron Super 模型上的实验表明，QAT (量化感知蒸馏) 能够恢复 4% 至 22% 的 PTQ 精度损失。在 Math-500 和 AIME 2024 等数学推理基准上，QAT 显著缩小了低精度推理与全精度模型之间的精度差距 [citation:10]。

3.5.4 极端量化：二值化神经网络 (BNN)

当量化精度降至 1 位时，权重和激活值仅取 $\{+1, -1\}$ 两个值，即构成二值化神经网络 (Binary Neural Network, BNN) [citation:1]。

BNN 的核心优势:

- 所有的乘法累加运算 (MAC) 可替换为 **XNOR 位运算** 和 **PopCount** (统计 1 的个数) 操作，大幅降低计算复杂度。
- 模型存储压缩比可达 32×，功耗极低，适用于可穿戴设备、IoT 传感器等极端资源受限场景 [citation:1]。

主要挑战:

- 在 ImageNet 等大规模数据集上，二值化模型相比全精度模型通常掉点 10% 以上，这在精度敏感应用中不可接受 [citation:1]。
- 当前研究重点包括: 引入缩放因子 (如 XNOR-Net)、使用多个二值模块逼近全精度分布 (如 ABC-Net)、在残差连接中保留全精度分支 (如 Bi-Real Net) 等 [citation:1]。

3.5.5 工业级量化最佳实践

小结

神经网络量化通过在存储、计算与精度之间建立灵活的权衡空间，为大模型的边缘部署提供了关键支撑。从工业界广泛采用的 INT8 PTQ，到追求极致精度的 QAT，再到面向极端场景的二值化网络，量化技术正朝着更精细、更自适应、更硬件友好的方向持续演进。当前研究前沿聚焦于极低比特 ($\leq 4 \text{ bit}$) 条件下的精度保持，以及学习驱动量化方法对异构硬件的适配优化 [citation:7]。

 **练习 3.3 量化概念检测** 判断以下说法的正误，并简要说明理由:

表 3.13: PTQ 与 QAT 的核心对比

对比维度	训练后量化（PTQ）	量化感知训练（QAT）	二值化网络（BNN）
是否需要重新训练	否（仅需校准）	是（需微调）	是（特殊训练策略）
精度保持	中（INT8 约损失 1-2%）	高（接近全精度）	低（通常掉点 10%+）
部署速度	极快	中等	极快（位运算加速）
实现复杂度	低	中	高
适用场景	快速原型验证、资源紧张	生产级部署、精度敏感	极端资源受限、低功耗

- 1. 量化感知训练（QAT）在前向传播中使用全精度计算，仅在反向传播中模拟量化效应。
- 2. 逐通道量化相比逐层量化能够更有效地降低量化误差，是卷积层的推荐默认设置。
- 3. 二值化神经网络（BNN）使用 1 位表示权重和激活值，所有乘加运算均可替换为 XNOR 位运算。
- 4. 训练后量化（PTQ）在模型训练后应用，需要完整的训练数据集进行校准。

第 4 章 编程框架使用

内容提要

❑ PyTorch 常用编程流程

❑ TensorFlow/Keras 常用编程流程

❑ 张量、模型、数据、训练、保存与部署 API

❑ 两类框架的常用 API 对照

本章只关注 PyTorch 和 TensorFlow 的**具体用法与重要 API**，不展开计算图、自动微分和运行时优化等原理。实际开发中可以按照如下顺序组织代码：

1. 导入框架并检查设备；
2. 准备数据集和批量加载器；
3. 定义模型、损失函数和优化器；
4. 编写训练、验证和测试流程；
5. 保存、加载和导出模型。

4.1 PyTorch 具体用法

4.1.1 导入与设备检查

例题 4.1 PyTorch 基础导入

```
import torch
from torch import nn
from torch.utils.data import Dataset, DataLoader

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(torch.__version__)
print(device)
```

常用设备 API：

- `torch.cuda.is_available()`：检查是否可用 CUDA。
- `torch.device("cuda")`：创建设备对象。
- `x.to(device)`：把张量或模型移动到指定设备。
- `torch.cuda.device_count()`：查看 GPU 数量。
- `torch.cuda.empty_cache()`：释放 PyTorch 缓存的显存。

4.1.2 张量创建与基本操作

例题 4.2 PyTorch 张量常用写法

```
x = torch.tensor([[1, 2], [3, 4]], dtype=torch.float32)
z = torch.zeros(2, 3)
o = torch.ones(2, 3)
r = torch.randn(4, 5)

print(x.shape)      # torch.Size([2, 2])
print(x.dtype)      # torch.float32
```

```

print(x.device)      # cpu 或 cuda

x = x.to(device)
y = x.reshape(4)
m = torch.matmul(x, x.T)
s = torch.softmax(x, dim=1)

```

表 4.1: PyTorch 张量 API 速查

API	用途
<code>torch.tensor(data)</code>	从 Python 列表或 NumPy 数组创建张量
<code>torch.zeros/ones/randn</code>	创建全零、全一或随机张量
<code>torch.arange, torch.linspace</code>	创建序列张量
<code>tensor.shape, tensor.dtype</code>	查看形状和数据类型
<code>tensor.reshape, tensor.view</code>	改变张量形状
<code>tensor.permute,</code> <code>tensor.transpose</code>	交换维度
<code>torch.cat, torch.stack</code>	拼接或堆叠张量
<code>torch.matmul, torch.mm</code>	矩阵乘法
<code>torch.sum/mean/max/argmax</code>	归约统计
<code>tensor.detach().cpu().numpy()</code>	转为 NumPy 数组

4.1.3 梯度计算用法

例题 4.3 PyTorch 梯度计算

```

w = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)
loss = (w ** 2).sum()
loss.backward()

print(w.grad) # tensor([2., 4., 6.])

with torch.no_grad():
    w -= 0.1 * w.grad
w.grad.zero_()

```

常用梯度 API:

- `requires_grad=True`: 声明张量需要梯度。
- `loss.backward()`: 执行反向传播。
- `param.grad`: 读取参数梯度。
- `optimizer.zero_grad()`: 清空旧梯度。
- `torch.no_grad()`: 关闭梯度记录, 常用于验证和推理。
- `tensor.detach()`: 得到不再参与梯度计算的张量。

4.1.4 模型定义

PyTorch 模型通常继承 `nn.Module`, 在 `__init__` 中定义层, 在 `forward` 中定义前向调用。

例题 4.4 PyTorch MLP 模型

```

class MLP(nn.Module):
    def __init__(self, in_dim=784, hidden=256, out_dim=10):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(in_dim, hidden),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(hidden, out_dim)
        )

    def forward(self, x):
        return self.net(x)

model = MLP().to(device)

```

表 4.2: PyTorch 常用网络层

API	用途
<code>nn.Linear</code>	全连接层
<code>nn.Conv1d/2d/3d</code>	一维、二维、三维卷积
<code>nn.MaxPool2d</code> , <code>nn.AvgPool2d</code>	池化层
<code>nn.BatchNorm1d/2d</code>	批归一化
<code>nn.LayerNorm</code>	层归一化
<code>nn.ReLU</code> , <code>nn.GELU</code> , <code>nn.Sigmoid</code>	激活函数
<code>nn.Dropout</code>	随机失活
<code>nn.Embedding</code>	词嵌入或离散 ID 嵌入
<code>nn.LSTM</code> , <code>nn.GRU</code>	循环神经网络层
<code>nn.TransformerEncoder</code>	Transformer 编码器组件

4.1.5 损失函数与优化器

例题 4.5 PyTorch 损失函数和优化器

```

loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)

pred = model(x)
loss = loss_fn(pred, y)

optimizer.zero_grad()
loss.backward()
optimizer.step()

```

常用 API:

- `nn.CrossEntropyLoss()`: 多分类任务常用。
- `nn.BCEWithLogitsLoss()`: 二分类或多标签任务常用。
- `nn.MSELoss()`: 回归任务常用。
- `torch.optim.SGD()`: 随机梯度下降。

- `torch.optim.Adam()`: Adam 优化器。
- `torch.optim.AdamW()`: 带权重衰减的 AdamW。
- `torch.optim.lr_scheduler.StepLR()`: 阶梯式学习率调整。
- `torch.optim.lr_scheduler.CosineAnnealingLR()`: 余弦退火学习率。

4.1.6 数据集与 DataLoader

例题 4.6 自定义 PyTorch Dataset

```
class MyDataset(Dataset):
    def __init__(self, xs, ys):
        self.xs = xs
        self.ys = ys

    def __len__(self):
        return len(self.xs)

    def __getitem__(self, idx):
        return self.xs[idx], self.ys[idx]

dataset = MyDataset(x_train, y_train)
loader = DataLoader(dataset, batch_size=64, shuffle=True, num_workers=2)
```

常用 DataLoader 参数:

- `batch_size`: 每批样本数量。
- `shuffle=True`: 训练集通常需要打乱。
- `num_workers`: 数据读取子进程数量。
- `pin_memory=True`: 配合 GPU 训练可提升数据拷贝效率。
- `drop_last=True`: 丢弃最后一个不足批量大小的批次。

4.1.7 训练、验证与测试模板

例题 4.7 PyTorch 标准训练循环

```
for epoch in range(num_epochs):
    model.train()
    total_loss = 0.0

    for x, y in train_loader:
        x, y = x.to(device), y.to(device)
        pred = model(x)
        loss = loss_fn(pred, y)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```



```

        total_loss += loss.item()

    model.eval()
    correct, total = 0, 0
    with torch.no_grad():
        for x, y in val_loader:
            x, y = x.to(device), y.to(device)
            pred = model(x)
            cls = pred.argmax(dim=1)
            correct += (cls == y).sum().item()
            total += y.size(0)

    print(epoch, total_loss / len(train_loader), correct / total)

```

4.1.8 保存与加载

例题 4.8 PyTorch 保存和加载模型

```

# 推荐保存参数
torch.save(model.state_dict(), "model.pt")

model = MLP().to(device)
model.load_state_dict(torch.load("model.pt", map_location=device))
model.eval()

# 保存训练断点
torch.save({
    "model": model.state_dict(),
    "optimizer": optimizer.state_dict(),
    "epoch": epoch
}, "checkpoint.pt")

```

4.1.9 混合精度训练

例题 4.9 PyTorch AMP 用法

```

scaler = torch.cuda.amp.GradScaler()

for x, y in train_loader:
    x, y = x.to(device), y.to(device)

    optimizer.zero_grad()
    with torch.cuda.amp.autocast():
        pred = model(x)
        loss = loss_fn(pred, y)

    scaler.scale(loss).backward()

```

```
scaler.step(optimizer)
scaler.update()
```

4.2 TensorFlow/Keras 具体用法

4.2.1 导入与设备检查

例题 4.10 TensorFlow 基础导入

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

print(tf.__version__)
print(tf.config.list_physical_devices())
print(tf.config.list_physical_devices("GPU"))
```

常用设备 API:

- `tf.config.list_physical_devices()`: 查看物理设备。
- `tf.config.list_physical_devices("GPU")`: 查看 GPU。
- `tf.device("/GPU:0")`: 指定设备上下文。
- `tf.config.experimental.set_memory_growth()`: 设置 GPU 显存按需增长。

4.2.2 张量与变量

例题 4.11 TensorFlow 张量常用写法

```
x = tf.constant([[1, 2], [3, 4]], dtype=tf.float32)
z = tf.zeros((2, 3))
o = tf.ones((2, 3))
r = tf.random.normal((4, 5))
```

```
print(x.shape)
print(x.dtype)
```

```
y = tf.reshape(x, (4,))
m = tf.matmul(x, tf.transpose(x))
s = tf.nn.softmax(x, axis=1)
```

```
w = tf.Variable(tf.random.normal((3, 2)))
w.assign_add(tf.ones_like(w))
```

4.2.3 Keras Sequential 模型

例题 4.12 Sequential 模型

```
model = keras.Sequential([
    layers.Input(shape=(784,)),
```

表 4.3: TensorFlow 张量 API 速查

API	用途
<code>tf.constant</code>	创建常量张量
<code>tf.Variable</code>	创建可训练或可更新变量
<code>tf.zeros/ones</code>	创建全零或全一张量
<code>tf.random.normal/uniform</code>	创建随机张量
<code>tf.reshape</code>	改变张量形状
<code>tf.transpose</code>	转置或交换维度
<code>tf.concat, tf.stack</code>	拼接或堆叠张量
<code>tf.matmul</code>	矩阵乘法
<code>tf.reduce_sum/mean/max</code>	归约统计
<code>tensor.numpy()</code>	转为 NumPy 数组

```

layers.Dense(256, activation="relu"),
layers.Dropout(0.2),
layers.Dense(10)
])

```

```
model.summary()
```

4.2.4 Keras 函数式模型

函数式 API 适合多输入、多输出、跳连结构。

例题 4.13 Functional API

```

inputs = keras.Input(shape=(784,))
x = layers.Dense(256, activation="relu")(inputs)
x = layers.Dropout(0.2)(x)
outputs = layers.Dense(10)(x)

model = keras.Model(inputs=inputs, outputs=outputs)

```

4.2.5 子类化模型

例题 4.14 Keras 子类化模型

```

class MLP(keras.Model):
    def __init__(self):
        super().__init__()
        self.dense1 = layers.Dense(256, activation="relu")
        self.drop = layers.Dropout(0.2)
        self.dense2 = layers.Dense(10)

    def call(self, x, training=False):
        x = self.dense1(x)
        x = self.drop(x, training=training)
        return self.dense2(x)

model = MLP()

```

表 4.4: Keras 常用网络层

API	用途
Dense	全连接层
Conv1D/2D/3D	一维、二维、三维卷积
MaxPooling2D	最大池化
BatchNormalization	批归一化
LayerNormalization	层归一化
Activation	指定激活函数
Dropout	随机失活
Embedding	嵌入层
LSTM, GRU	循环神经网络层
MultiHeadAttention	多头注意力层

4.2.6 compile、fit、evaluate、predict

Keras 标准训练流程通常使用 compile 和 fit。

例题 4.15 Keras 标准训练流程

```
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    metrics=["accuracy"]
)

history = model.fit(
    train_dataset,
    validation_data=val_dataset,
    epochs=10
)

test_loss, test_acc = model.evaluate(test_dataset)
pred = model.predict(x_test)
```

常用 API:

- model.compile(): 配置优化器、损失函数和指标。
- model.fit(): 训练模型。
- model.evaluate(): 评估模型。
- model.predict(): 执行预测。
- model.summary(): 查看模型结构。

4.2.7 损失函数、优化器与指标

4.2.8 tf.data 数据流水线

例题 4.16 TensorFlow 数据流水线

```
train_dataset = (
    tf.data.Dataset.from_tensor_slices((x_train, y_train))
    .shuffle(10000)
```

表 4.5: TensorFlow/Keras 常用训练 API

API	用途
<code>SparseCategoricalCrossentropy</code>	整数标签多分类
<code>CategoricalCrossentropy</code>	one-hot 标签多分类
<code>BinaryCrossentropy</code>	二分类或多标签分类
<code>MeanSquaredError</code>	回归任务
<code>SGD</code>	随机梯度下降
<code>Adam</code>	Adam 优化器
<code>AdamW</code>	AdamW 优化器
<code>Accuracy</code>	准确率
<code>SparseCategoricalAccuracy</code>	整数标签分类准确率

```
.batch(128)
.prefetch(tf.data.AUTOTUNE)
)
```

```
for x, y in train_dataset:
    print(x.shape, y.shape)
    break
```

常用 `tf.data` API:

- `Dataset.from_tensor_slices()`: 从张量或数组构建数据集。
- `Dataset.map()`: 对样本做预处理。
- `Dataset.shuffle()`: 打乱数据。
- `Dataset.batch()`: 组成批量。
- `Dataset.prefetch()`: 预取后续批次。
- `tf.data.AUTOTUNE`: 自动设置并行度或预取缓冲。

4.2.9 GradientTape 自定义训练

当 `model.fit()` 不够灵活时，可以使用 `tf.GradientTape` 编写训练循环。

例题 4.17 TensorFlow 自定义训练循环

```
loss_fn = keras.losses.SparseCategoricalCrossentropy(from_logits=True)
optimizer = keras.optimizers.Adam(1e-3)

for epoch in range(num_epochs):
    for x, y in train_dataset:
        with tf.GradientTape() as tape:
            pred = model(x, training=True)
            loss = loss_fn(y, pred)

        grads = tape.gradient(loss, model.trainable_variables)
        optimizer.apply_gradients(zip(grads, model.trainable_variables))
```

4.2.10 保存与加载

例题 4.18 TensorFlow/Keras 保存和加载


```
# 保存完整模型
model.save("saved_model.keras")
model = keras.models.load_model("saved_model.keras")

# 只保存权重
model.save_weights("weights.weights.h5")
model.load_weights("weights.weights.h5")

# 保存为 TensorFlow SavedModel
model.export("saved_model_dir")
```

4.2.11 回调函数

例题 4.19 Keras callbacks

```
callbacks = [
    keras.callbacks.ModelCheckpoint(
        "best.keras",
        save_best_only=True,
        monitor="val_loss"
    ),
    keras.callbacks.EarlyStopping(
        monitor="val_loss",
        patience=5,
        restore_best_weights=True
    ),
    keras.callbacks.TensorBoard(log_dir="logs")
]

model.fit(train_dataset, validation_data=val_dataset,
          epochs=50, callbacks=callbacks)
```

4.2.12 混合精度训练

例题 4.20 TensorFlow 混合精度

```
from tensorflow.keras import mixed_precision

mixed_precision.set_global_policy("mixed_float16")

model = keras.Sequential([
    layers.Input(shape=(784,)),
    layers.Dense(256, activation="relu"),
    layers.Dense(10, dtype="float32")
])
```

4.3 常见任务代码模板

4.3.1 PyTorch 图像分类模板

例题 4.21 PyTorch 分类任务最小模板

```
model = MLP().to(device)
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3)

for epoch in range(10):
    model.train()
    for x, y in train_loader:
        x = x.reshape(x.size(0), -1).to(device)
        y = y.to(device)

        pred = model(x)
        loss = loss_fn(pred, y)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

4.3.2 TensorFlow 图像分类模板

例题 4.22 TensorFlow/Keras 分类任务最小模板

```
model = keras.Sequential([
    layers.Input(shape=(784,)),
    layers.Dense(256, activation="relu"),
    layers.Dense(10)
])

model.compile(
    optimizer=keras.optimizers.Adam(1e-3),
    loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    metrics=["accuracy"]
)

model.fit(train_dataset, validation_data=val_dataset, epochs=10)
```

4.4 PyTorch 与 TensorFlow API 对照

4.5 实践检查清单

编写框架代码时，可以按以下清单快速检查：

表 4.6: 常用 API 对照表

功能	PyTorch	TensorFlow/Keras
张量创建	<code>torch.tensor</code>	<code>tf.constant</code>
随机张量	<code>torch.randn</code>	<code>tf.random.normal</code>
可训练参数	<code>nn.Parameter</code>	<code>tf.Variable</code>
模型基类	<code>nn.Module</code>	<code>keras.Model</code>
全连接层	<code>nn.Linear</code>	<code>layers.Dense</code>
卷积层	<code>nn.Conv2d</code>	<code>layers.Conv2D</code>
损失函数	<code>nn.CrossEntropyLoss</code>	<code>keras.losses.*</code>
优化器	<code>torch.optim.AdamW</code>	<code>keras.optimizers.AdamW</code>
数据加载	<code>DataLoader</code>	<code>tf.data.Dataset</code>
反向传播	<code>loss.backward()</code>	<code>tape.gradient()</code>
参数更新	<code>optimizer.step()</code>	<code>apply_gradients()</code>
关闭梯度	<code>torch.no_grad()</code>	<code>training=False</code> 或不使用 <code>tape</code>
保存模型	<code>torch.save</code>	<code>model.save</code>

1. 输入数据形状是否与模型第一层匹配。
2. 分类任务的标签格式是否与损失函数匹配。
3. PyTorch 中模型和数据是否都已经移动到同一个 device。
4. PyTorch 中每次反向传播前是否调用 `optimizer.zero_grad()`。
5. 验证和测试阶段是否关闭训练行为，例如 `model.eval()` 或 `training=False`。
6. 保存模型时，是只保存权重，还是保存完整模型。
7. 数据加载是否设置了批量、打乱和预取。
8. 训练日志是否记录 `loss`、`accuracy` 和学习率。

练习 4.1 API 使用练习 完成以下小任务：

1. 分别用 PyTorch 和 TensorFlow 写出一个两层 MLP。
2. 分别写出分类任务的训练代码，要求包含损失函数和优化器。
3. 分别保存模型权重，并重新加载用于推理。
4. 修改数据批量大小，观察训练速度和显存占用变化。
5. 将训练集划分为训练集和验证集，输出每轮验证准确率。